

Detección visual, cálculo de puntos de navegación y control autónomo para UAVs



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor: Álvaro Ros Villanueva

Director: Daniel Aláez Gómez

Pamplona, junio de 2026

upna

Universidad Pública de Navarra
Nafarroako Unibertsitate Publikoa

Resumen

Este Trabajo Fin de Grado presenta el diseño, desarrollo y validación de una arquitectura de software para la navegación autónoma de vehículos aéreos no tripulados (UAVs) en entornos de interior sin señal GPS. Utilizando un dron comercial de bajo coste (DJI Tello), el sistema es capaz de detectar puertas, representadas mediante marcadores ArUco, estimar su posición tridimensional y generar rutas de vuelo, con el objetivo de lograr una navegación autónoma.

Toda la lógica computacional se ha generado sobre el middleware ROS 2, garantizando la modularidad del código y el procesamiento asíncrono de los flujos de datos en tiempo real. Se han utilizado técnicas de procesamiento de imagen y teorías clásicas de control para solventar los problemas físicos descubiertos en las pruebas de vuelo.

El sistema completo ha sido validado de forma iterativa, comenzando en un entorno virtual de simulación (Webots) y finalizando con éxito en pruebas dinámicas de laboratorio. Los resultados demuestran que la arquitectura propuesta es robusta y escalable, logrando que el UAV compense activamente sus propias limitaciones físicas de hardware para ejecutar misiones espaciales y cruzar obstáculos de forma íntegramente autónoma.

Palabras clave: UAV, Robótica móvil, Navegación autónoma, Navegación sin GPS, ROS 2, Visión artificial, Estimación de pose, Problema PnP, Control PID, Seguimiento de trayectorias.

Abstract

This Bachelor's Thesis presents the design, development, and validation of a distributed software architecture for the autonomous navigation of Unmanned Aerial Vehicles (UAVs) in GPS-denied indoor environments. Utilizing a low-cost commercial drone (DJI Tello), the system is capable of detecting doors represented by ArUco markers, estimating their three-dimensional position, and generating flight routes to achieve autonomous navigation.

All computational logic has been developed using the ROS 2 middleware, ensuring code modularity and real-time asynchronous data flow processing. Image processing techniques and classical control theories have been employed to address the physical challenges encountered during flight tests.

The complete system has been iteratively validated, starting in a virtual simulation environment (Webots) and successfully concluding with dynamic laboratory tests. The results demonstrate that the proposed architecture is robust and scalable, enabling the UAV to actively compensate for its own physical hardware limitations to execute spatial missions and navigate obstacles fully autonomously.

Keywords: UAV, Mobile robotics, Autonomous navigation, GPS-denied navigation, ROS 2, Computer vision, Pose estimation, PnP problem, PID control, Path following.

Análisis de Impacto y ODS

Los resultados de este Trabajo Fin de Grado tienen una aplicación directa en el sector industrial, presentando implicaciones reales en la evolución de la robótica móvil y su integración en la sociedad. La capacidad de dotar a un vehículo aéreo no tripulado (UAV) de navegación autónoma en interiores desconocidos, utilizando hardware de bajo coste y software de código abierto, facilita el acceso a tecnologías de vanguardia. De este modo, el proyecto contribuye al desarrollo sostenible y al logro de la Agenda 2030 de las Naciones Unidas, alineándose de manera específica con tres Objetivos de Desarrollo Sostenible (ODS):

ODS 8: Trabajo decente y crecimiento económico. La automatización de la exploración en espacios cerrados tiene un impacto directo en la seguridad y salud laboral. El sistema desarrollado permite que un dron se adentre en infraestructuras previamente desconocidas, sin depender de señal GPS, mapeando el entorno a través de referencias visuales. Esta tecnología es directamente extrapolable a tareas de inspección industrial, minería o evaluación de edificios colapsados. Con ello, se evita la exposición de trabajadores humanos a entornos de alto riesgo, a la par que se fomenta la creación de empleo de alto valor añadido centrado en la programación, mantenimiento y supervisión de estos sistemas autónomos.

ODS 9: Industria, innovación e infraestructura. El proyecto se fundamenta en la investigación tecnológica y la innovación aplicada. Al utilizar una arquitectura modular basada en el estándar industrial ROS 2 y librerías de visión artificial (OpenCV, `solvePnP`), es posible desarrollar sistemas de navegación complejos sin requerir sensores altamente costosos, como cámaras RGB-D o LiDAR. Esto no solo optimiza el uso de recursos, sino que impulsa un desarrollo tecnológico accesible y escalable, permitiendo que pequeñas y medianas empresas puedan integrar soluciones robóticas en sus infraestructuras.

ODS 11: Ciudades y comunidades sostenibles. A medida que los núcleos urbanos crecen en densidad y complejidad, la gestión inteligente de los espacios interiores (hospitales, almacenes logísticos o grandes superficies) resulta vital. Este proyecto sienta las bases algorítmicas para que flotas de drones puedan realizar tareas de monitorización, transporte de pequeño material o asistencia en emergencias dentro del ecosistema urbano cerrado. Esta capacidad de respuesta contribuye a la creación de infraestructuras ciudadanas más resilientes, seguras y eficientes.

Índice general

Resumen	I
Abstract	II
Análisis de impacto y ODS	III
1. Introducción	1
1.1. Contexto y motivación	1
1.2. Alcance del proyecto	1
1.3. Estructura de la memoria	2
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Estado del arte	5
3.1. El desafío de la navegación en interiores	5
3.2. Plataformas aéreas para la investigación y educación	6
3.3. Visión artificial y localización espacial	8
3.3.1. Odometría visual y flujo óptico	8
3.3.2. Estimación de profundidad mediante Deep Learning	10
3.3.3. Sistemas SLAM visual	10
3.3.4. Marcadores visuales en robótica industrial	13
3.3.5. Algoritmos geométricos y estimación de pose	15
3.4. Control cinemático y planificación de trayectorias	18
3.4.1. Estrategias de control de vuelo	18
3.4.2. Navegación espacial	19
3.5. Ecosistemas de desarrollo en robótica	20
3.5.1. Middleware robótico: ROS 2	21
3.5.2. Herramientas de visualización y análisis	22
3.6. Entornos de simulación robótica	23
4. Metodología	26
4.1. Plataforma hardware: DJI Tello	26
4.2. Entorno de simulación: Webots	28
4.3. Entorno de software: ROS 2 y Python	29
4.3.1. Herramientas de monitorización y depuración	29
4.4. Ciclo de desarrollo y pruebas	30

4.4.1. Secuenciación de tareas y diagrama de Gantt	31
5. Diseño e implementación de la arquitectura	33
5.1. Arquitectura global del sistema	33
5.2. Módulo de telemetría	35
5.2.1. El enfoque inicial: DJITelloPy y ORB-SLAM3	35
5.2.2. Limitaciones físicas del SLAM monocular y alternativas	36
5.2.3. Traslado a TelloPy y arquitectura basada en eventos	40
5.2.4. El problema del origen: <code>nodoIntermedioPose</code>	42
5.3. Módulo de detección	44
5.3.1. Detección 2D en la imagen	44
5.3.2. Estimación de pose relativa	47
5.3.3. Álgebra espacial y cambios de sistema de referencia	50
5.4. Módulo de navegación lógica	55
5.4.1. Máquina de estados y gestión de objetivos	56
5.4.2. Navegación punto a punto y control proporcional (P)	57
6. Resultados	59
6.1. Validación de los módulos de telemetría y detección	59
6.1.1. Estabilización de odometría	59
6.1.2. Condicionantes del entorno físico	60
6.1.3. Precisión de la detección espacial	61
6.2. Resultados en simulación	63
6.3. Limitaciones físicas del DJI Tello	65
6.4. Evolución del control: PID y seguimiento de trayectorias	66
6.4.1. Controlador PID (proporcional, integral y derivativo)	66
6.4.2. Seguimiento de trayectoria	67
6.4.3. Análisis comparativo de la respuesta cinemática	69
6.5. Evaluación final del vuelo físico autónomo	71
7. Conclusiones y trabajo futuro	72
7.1. Conclusiones del proyecto	72
7.2. Líneas de trabajo futuro	72
7.2.1. Procesamiento a bordo para evitar cortes de red	72
7.2.2. Detección de obstáculos mediante <i>Deep Learning</i>	73
7.2.3. Navegación basada en segmentación semántica	73
7.2.4. Diseño de un UAV personalizado	73
7.3. Viabilidad comercial y aplicaciones industriales	73
Bibliografía	75

Índice de figuras

3.1. Visualización de una nube de puntos 3D de alta resolución capturada por un sistema LiDAR profesional [1].	6
3.2. Fotografía de un dron profesional equipado con un sensor LiDAR YellowScan, un hardware de alto peso y coste [2].	6
3.3. Kit de desarrollo PX4 X500 V2, un ejemplo de UAV de arquitectura abierta con alta capacidad de carga para ordenadores de a bordo [3]. . .	7
3.4. El micro-dron Bitcraze Crazyflie 2.1, ampliamente utilizado en investigación por su tamaño ultra-compacto y seguridad en interiores [4]. . . .	8
3.5. Esquema del flujo óptico: el desplazamiento físico del dron en una dirección provoca un desplazamiento inverso de los píxeles en el plano de la cámara [5].	9
3.6. Módulo PX4Flow, un sensor de odometría visual inteligente muy utilizado en robótica aérea de código abierto [5].	9
3.7. Ejemplo de una cámara de profundidad RGB-D [6].	10
3.8. Comparativa de inferencia de profundidad utilizando diferentes versiones de la arquitectura neuronal MiDaS a partir de una única imagen RGB (esquina superior izquierda) [7].	11
3.9. Visualización del sistema ORB-SLAM3 operando en modo estéreo (dos cámaras). A la derecha, la extracción de características en los dos flujos de vídeo simultáneos; a la izquierda, la nube de puntos 3D y la trayectoria generada [8].	12
3.10. Ejemplo de marcador ArUco.	13
3.11. Ejemplo de marcador AprilTag.	13
3.12. Esquema de transformación de coordenadas: al detectar las esquinas del marcador 2D, el dron calcula los vectores de traslación y rotación tridimensionales [9].	14
3.13. Diferentes ejemplos de uso industrial y académico de marcadores visuales [9].	15
3.14. Esquema geométrico del problema PnP [10].	16
3.15. Visualización del árbol de transformaciones en un robot humanoide (PR2). Cada articulación y sensor opera con su propio sistema de referencia local, requiriendo un seguimiento continuo para lograr coherencia global [11].	17
3.16. Curva de respuesta de un controlador PID. Se observa cómo la sintonización correcta de los parámetros permite alcanzar el objetivo de forma rápida, mitigando el <i>overshoot</i> y estabilizando el sistema en el tiempo [12].	19

3.17. Ilustración de los errores de navegación. El algoritmo de seguimiento de trayectorias se centra en minimizar el error lateral (<i>cross-track error</i>) para forzar al vehículo a regresar a la ruta planificada [13].	20
3.18. Arquitectura de comunicación en ROS 2 [14].	21
3.19. Comparativa arquitectónica. A la izquierda, la topología centralizada de ROS 1, dependiente del <i>ROS Master</i> . A la derecha, la topología distribuida de ROS 2 operando sobre el bus de datos estándar DDS [15]. . .	22
3.20. Interfaz gráfica de RViz2 representando un ejemplo industrial [16]. . . .	23
3.21. Captura de un entorno de simulación de UAVs desarrollado en Gazebo [17].	24
4.1. Dron comercial DJI Tello utilizado para las pruebas de vuelo en el entorno físico.	27
4.2. Entorno de simulación en Webots mostrando el modelo virtual del Mavic 2 Pro y las puertas creadas.	29
4.3. Visualización en RViz2 durante las pruebas físicas, mostrando la pose del dron, la estimación en el mapa 3D de la puerta detectada, y a la izquierda, la imagen de la cámara modificada en tiempo real.	30
4.4. Diagrama de Gantt mostrando la planificación temporal.	32
5.1. Diagrama de la arquitectura de software en ROS 2, mostrando los cuatro nodos principales y el flujo de datos a través de topics.	34
5.2. Detección geométrica y reproyección de esquinas internas durante el proceso de calibración de la cámara del dron.	37
5.3. Captura del algoritmo ORB-SLAM3 procesando el flujo de vídeo monocular y generando la nube de puntos en tiempo real.	37
5.4. Pérdida de referencia de ORB-SLAM3 en el eje Z.	38
5.5. Escasez de puntos de interés detectados por el algoritmo durante la navegación a través de un pasillo.	39
5.6. Efecto túnel debido a disponer solo de puntos de referencia lejanos en la siguiente sala.	39
5.7. Detección de los vértices de la puerta en el simulador Webots.	46
5.8. Diagrama que detalla los sistemas de coordenadas y las transformaciones espaciales.	52
6.1. Captura de la terminal donde se muestra a la derecha los valores extraídos directamente de la lectura del Tello y a la izquierda, la lectura correcta de la pose del dron, publicada por <code>nodoIntermedioPose</code>	60
6.2. Entorno del laboratorio preparado para el vuelo.	60
6.3. Ejemplo de un pasillo carente de textura e iluminación.	60
6.4. Fallo del filtrado de color por reflejos en la escena real.	61
6.5. Anclaje exitoso de un único marcador ArUco en RViz2.	61
6.6. Proyección de los tres puntos de navegación.	61
6.7. Validación de la distancia a la puerta calculada utilizando referencias métricas.	62
6.8. Lectura de las mediciones a través de la terminal y <code>rqt</code>	62
6.9. Validación de la detección del ángulo de <i>Yaw</i>	63

6.10. Validación de la navegación en simulación. Se observa el entorno 3D en Webots junto con la lectura de la máquina de estados en la terminal y la detección de las puertas en <code>rqt</code>	64
6.11. Representación en RViz2 del seguimiento de trayectoria. Se observa la puerta detectada (verde), posición del dron (amarillo) y la trayectoria calculada a seguir (azul).	68
6.12. Escenario A: vuelo simulado.	69
6.13. Escenario B: vuelo real	70
6.14. Escenario C: vuelo real.	70

Índice de tablas

3.1. Comparativa técnica entre los sistemas de marcadores visuales más extendidos.	14
5.1. Protocolos de seguridad y comandos especiales codificados en el array de velocidades.	42
6.1. Error en la estimación de distancia a la puerta.	63
6.2. Error en la estimación del ángulo de rotación.	64

Capítulo 1

Introducción

El presente Trabajo Fin de Grado desarrolla un sistema de software que dota a un vehículo aéreo no tripulado (UAV) de la capacidad de detectar, localizar y navegar de forma autónoma en un entorno físico a priori desconocido.

1.1. Contexto y motivación

En el ámbito de la robótica móvil, uno de los mayores desafíos es la navegación autónoma en interiores sin depender de sistemas de posicionamiento global (GPS). Cuando un dron accede a un escenario desconocido, es necesario que sea capaz de construir una representación del entorno y ubicarse dentro de él de manera dinámica.

La motivación principal de este proyecto es desarrollar un sistema de navegación autónoma basado en visión artificial monocular. Dado que se utiliza una única cámara, el sistema no es capaz de calcular profundidad. Para solventar esta limitación y calcular las distancias, el dron utiliza referencias visuales con dimensiones físicas conocidas.

En este caso de estudio, se han utilizado puertas como referencias para navegar, ya que son fáciles de visualizar y de probar en el laboratorio. La principal ventaja del sistema, es que basta con que la cámara detecte la puerta una única vez para que esta quede registrada permanentemente en el mapa tridimensional del dron, permitiéndole navegar de forma autónoma sin necesidad de mantener una línea de visión constante con el objetivo.

1.2. Alcance del proyecto

Para abordar la complejidad técnica del problema, el alcance de este desarrollo se plantea mediante una estructura dividida en tres fases principalmente:

1. **Fase de detección:** Identificación de referencias visuales a través del flujo de vídeo en tiempo real. Se plantea desarrollar un sistema de visión artificial capaz de extraer puntos de interés de la imagen. Inicialmente, se abordará mediante la detección de colores en un entorno simulado, para posteriormente evolucionar hacia sistemas más robustos basados en marcadores visuales, debido a su fiabilidad en un entorno real.

2. **Fase de ubicación 3D:** Proyección y transformación geométrica de las detecciones bidimensionales (2D) obtenidas por la cámara hacia coordenadas tridimensionales (3D) relativas al mapa. El objetivo en esta fase es resolver matemáticamente el problema de estimación de pose para dotar al vehículo de memoria espacial.
3. **Fase de navegación:** Ejecución de la lógica de vuelo autónomo. Se diseñará un sistema de toma de decisiones que gestione una cola de tareas y permita la navegación punto a punto (*waypoints*) hacia las coordenadas espaciales previamente calculadas.

A nivel de arquitectura de software, el proyecto requiere el diseño de un sistema modular y distribuido que permita la comunicación asíncrona entre los diferentes procesos necesarios (gestión de sensores, visión artificial y control de navegación).

Asimismo, el ciclo de validación del proyecto se contempla en dos grandes etapas: una fase de simulación inicial en un entorno virtual tridimensional para garantizar la seguridad de los algoritmos y afianzar la lógica, seguida de un despliegue en el mundo real utilizando un dron comercial de bajo coste. Debido a las previsibles limitaciones de capacidad de cómputo interno en este tipo de vehículos, se prevé que la carga de procesamiento pesado (visión y control) se ejecute en un ordenador externo, enviando las órdenes de vuelo al dron de forma remota (arquitectura *offboard*).

1.3. Estructura de la memoria

Para facilitar la comprensión del trabajo realizado, la presente memoria se ha estructurado en los siguientes capítulos:

- **Capítulo 2. Objetivos:** Se definen las metas generales y los hitos específicos.
- **Capítulo 3. Estado del arte:** Se analizan las tecnologías actuales relacionadas con este proyecto.
- **Capítulo 4. Metodología:** Se describen las herramientas de software, el entorno de trabajo y la planificación seguida durante el proyecto.
- **Capítulo 5. Diseño e implementación de la arquitectura:** Se detalla en profundidad la arquitectura del software, los nodos desarrollados y los algoritmos implementados para las diferentes fases de este trabajo.
- **Capítulo 6. Resultados:** Se exponen las métricas y el rendimiento del sistema, tanto en el simulador como en las pruebas físicas.
- **Capítulo 7. Conclusiones:** Se realiza una valoración crítica del proyecto y se proponen futuras líneas de investigación y mejora.

Capítulo 2

Objetivos

El propósito de este capítulo es definir las metas que guían el desarrollo de este Trabajo Fin de Grado, dividiéndolas en un objetivo principal que engloba la finalidad del proyecto, y una serie de objetivos específicos que detallan los hitos técnicos necesarios para lograr esa meta general.

2.1. Objetivo general

El objetivo principal de este proyecto es diseñar, desarrollar y validar una arquitectura de software integral, basada en ROS 2, que permita a un vehículo aéreo no tripulado (UAV) con cámara monocular explorar un entorno a priori desconocido, detectando referencias visuales para ubicarlas en un mapa tridimensional y utilizar dicha información para la navegación autónoma.

2.2. Objetivos específicos

Para alcanzar el objetivo general propuesto, se han establecido los siguientes objetivos específicos:

- **Integración y control de hardware UAV:** Establecer una comunicación bidireccional fiable con un dron comercial (DJI Tello), para lograr extraer la telemetría completa y la estimación de pose en seis grados de libertad (x , y , z , $roll$, $pitch$, yaw) necesarios para la odometría.
- **Desarrollo del entorno de simulación:** Diseñar un escenario realista en el simulador Webots, utilizando un modelo proporcionado por el simulador (Mavic 2 Pro) y modelando los elementos necesarios del entorno.
- **Detección y procesamiento bidimensional (2D):** Implementar algoritmos de visión artificial que procesen el flujo de vídeo en tiempo real para identificar los puntos de interés.
- **Transformación y mapeo tridimensional (3D):** Desarrollar el modelo matemático necesario para proyectar las coordenadas 2D obtenidas en el plano de la cámara hacia un sistema de coordenadas espaciales 3D, dotando así al dron de memoria espacial.

- **Planificación y navegación autónoma:** Diseñar un sistema de control de vuelo capaz de gestionar una cola de tareas (puertas detectadas y puertas atravesadas) para dirigir al dron mediante un sistema de navegación por *waypoints* hacia los objetivos ubicados.
- **Validación integral del sistema:** Comprobar el correcto funcionamiento de toda la arquitectura de software en la fase de simulación (Webots) y, posteriormente, transferir y validar el sistema en un entorno físico real con el DJI Tello.

Capítulo 3

Estado del arte

Para contextualizar adecuadamente las decisiones tomadas a lo largo de este Trabajo Fin de Grado, es necesario analizar el panorama actual de las tecnologías implicadas y establecer un marco teórico sólido. El diseño de un sistema de navegación autónoma requiere el uso de múltiples disciplinas, abarcando desde las limitaciones del hardware físico hasta la optimización de los algoritmos de control de vuelo.

En este capítulo se desglosa el estado actual de la industria robótica y la investigación académica. En primer lugar, se expone la problemática acerca de la navegación en entornos sin GPS y se evalúan las plataformas aéreas disponibles para su estudio. Posteriormente, se profundiza en las técnicas de visión artificial enfocadas a la localización espacial, comparando algoritmos de alto coste computacional frente a soluciones geométricas ligeras. Por último, se examinan las estrategias de control de vuelo y se analizan los ecosistemas de software de código abierto, evidenciando el nicho tecnológico que este proyecto pretende resolver mediante el diseño de una arquitectura integral.

3.1. El desafío de la navegación en interiores

En el mundo de los drones, volar al aire libre es un problema relativamente resuelto gracias a los sistemas GPS, que proporcionan coordenadas exactas en todo momento. Sin embargo, cuando un vehículo aéreo entra en un edificio, un túnel o un almacén cerrado, la señal de los satélites rebota con las paredes o se pierde por completo. A esta situación se le conoce en la robótica como un entorno *GPS-denied* (sin acceso a GPS). En el instante en que se pierde la señal, el dron se queda completamente “ciego” respecto a su ubicación global y no sabe si está quieto, avanzando o yéndose contra un obstáculo.

Para evitar colisiones, el dron está obligado a deducir su posición utilizando exclusivamente los sensores que lleva a bordo. En la industria robótica profesional, la solución estándar para mapear interiores es el uso de escáneres láser tridimensionales, conocidos como LiDAR¹. Esta tecnología permite que el vehículo tenga un modelo tridimensional exacto del espacio que lo rodea, como se ilustra en la Figura 3.1.

Esta tecnología es extremadamente potente y funciona a la perfección para crear mapas precisos en entornos complejos. Sin embargo, el hardware necesario para capturar esta información no es viable para micro-drones comerciales de bajo coste. Los

¹LiDAR (*Light Detection and Ranging*): Tecnología de telemetría que emite pulsos láser para medir distancias exactas y construir nubes de puntos 3D de alta resolución.

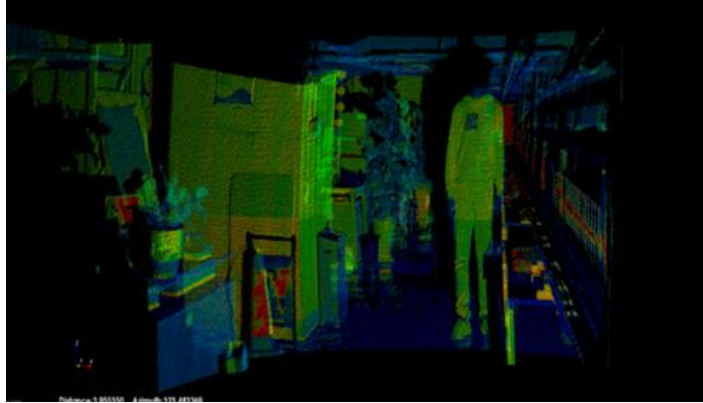


Figura 3.1: Visualización de una nube de puntos 3D de alta resolución capturada por un sistema LiDAR profesional [1].

sensores LiDAR profesionales son dispositivos caros, consumen demasiada energía y, sobre todo, pesan demasiado. La Figura 3.2 muestra un dron profesional equipado con un voluminoso sensor LiDAR, un tamaño y complejidad imposibles de replicar en una plataforma educativa.



Figura 3.2: Fotografía de un dron profesional equipado con un sensor LiDAR YellowScan, un hardware de alto peso y coste [2].

Un micro-dron de dimensiones reducidas es físicamente incapaz de levantar este tipo de hardware. Por lo tanto, la única alternativa viable es utilizar una cámara de vídeo convencional (visión monocular) y suplir la falta de sensores caros con algoritmos de visión artificial que sean capaces de estimar las distancias matemáticas a partir de los píxeles de la imagen. Este es uno de los desafíos que aborda este proyecto: reconstruir el mundo 3D a partir de una simple imagen plana.

3.2. Plataformas aéreas para la investigación y educación

A la hora de seleccionar una plataforma física para investigar la navegación autónoma en interiores, el mercado actual ofrece soluciones muy diversas. En la robótica aérea de investigación, los vehículos se dividen principalmente en dos grandes categorías, cada una con sus propias ventajas y cuellos de botella técnicos.

Por un lado, se encuentran los drones de arquitectura abierta y hardware personalizado. Estas plataformas suelen construirse en laboratorios montando piezas a medida sobre controladores de vuelo industriales de código abierto, como *Pixhawk*. Un ejemplo representativo de esta categoría es el *PX4 Development Kit X500 V2* (Figura 3.3). Su gran ventaja es la capacidad de carga útil. Al ser drones más grandes y potentes, permiten llevar a bordo ordenadores (como una Raspberry Pi o una NVIDIA Jetson) para procesar algoritmos pesados en pleno vuelo, además de soportar multitud de sensores periféricos.



Figura 3.3: Kit de desarrollo PX4 X500 V2, un ejemplo de UAV de arquitectura abierta con alta capacidad de carga para ordenadores de a bordo [3].

Sin embargo, esta ventaja computacional tiene un precio físico. Su tamaño, el peso de las baterías y la fuerza de rotación de sus grandes hélices, fabricadas típicamente en CFRP², los convierten en aparatos potencialmente peligrosos para volar en aulas, espacios cerrados reducidos o cerca de personas sin utilizar redes de seguridad perimetrales.

Por otro lado, destaca el segmento de los micro-UAVs³ comerciales y de investigación, donde reinan modelos como el *Bitcraze Crazyflie* (Figura 3.4) o el *DJI Tello* [18]. El principal atractivo de estos vehículos es su peso ultraligero (el Crazyflie apenas pesa 27 gramos) y sus dimensiones compactas. Esto los hace seguros para operar en cualquier laboratorio sin riesgo de causar daños en caso de colisión o fallo del software.

La desventaja de este tipo de drones es la restricción de hardware. Para mantener el peso bajo y maximizar la autonomía, estos drones carecen de procesadores capaces de ejecutar visión artificial avanzada o cálculos de trayectorias complejas de forma interna. Esta limitación obliga a los desarrolladores a cambiar de paradigma: en lugar de procesar a bordo (*onboard*), se debe diseñar una arquitectura externa (*offboard*). En este modelo operativo, el dron actúa como un simple sistema de adquisición de datos que envía su telemetría y el flujo de vídeo a un ordenador central a través de una red

²CFRP (*Carbon Fiber Reinforced Polymer*): Polímeros reforzados con fibra de carbono.

³UAV (*Unmanned Aerial Vehicle*): Vehículo aéreo no tripulado, término técnico y estandarizado en la industria para referirse a los drones.



Figura 3.4: El micro-dron Bitcraze Crazyflie 2.1, ampliamente utilizado en investigación por su tamaño ultra-compacto y seguridad en interiores [4].

Wi-Fi; es este ordenador el cerebro que ejecuta los algoritmos pesados y devuelve las órdenes de movimiento a los motores.

3.3. Visión artificial y localización espacial

Dotar a un vehículo aéreo de la capacidad de comprender su entorno es uno de los mayores retos en la robótica autónoma. Cuando se utiliza una única cámara, el sistema se enfrenta al problema de la pérdida de la dimensión de profundidad. Al proyectar el mundo tridimensional real sobre el sensor bidimensional plano de la cámara, resulta imposible determinar a simple vista si un objeto en la imagen es pequeño y está muy cerca, o si es grande y se encuentra a varios metros de distancia.

Para que el dron pueda navegar de forma segura hacia un objetivo, es necesario recuperar esta tercera dimensión perdida. A lo largo de los años, la comunidad investigadora y la industria han desarrollado diferentes estrategias algorítmicas para resolver este reto computacional. A continuación, se analizan las principales técnicas de localización espacial, abarcando desde los métodos de estabilización integrados en el hardware y los recientes avances en redes neuronales, hasta llegar a los sistemas de mapeo y las soluciones geométricas de bajo consumo que permiten calcular distancias exactas.

3.3.1. Odometría visual y flujo óptico

Para que un vehículo aéreo pueda navegar de forma autónoma, el primer requisito indispensable es que sea capaz de mantenerse estable en el aire. A falta de coordenadas satélite, la inmensa mayoría de micro-drones resuelven este problema equipando una cámara secundaria orientada verticalmente hacia el suelo, combinada habitualmente con un sensor de distancia (infrarrojo o ultrasonido) que actúa como altímetro.

El procesador del vehículo utiliza una técnica clásica de visión artificial conocida

como flujo óptico (*optical flow*). Este algoritmo analiza el desplazamiento espacial de los píxeles entre dos fotogramas consecutivos capturados por la cámara inferior. Como se ilustra en el esquema de la Figura 3.5, si el dron se mueve hacia la derecha, el algoritmo detecta que el patrón visual del suelo se mueve hacia la izquierda en la imagen. Con esta información, el controlador de vuelo calcula la velocidad relativa y por lo tanto consigue tanto estabilizarse en el aire, como calcular su pose.

Example:

- Drone moves to the right
- Pixel pattern moves to the left



Figura 3.5: Esquema del flujo óptico: el desplazamiento físico del dron en una dirección provoca un desplazamiento inverso de los píxeles en el plano de la cámara [5].

En la industria de los drones, esta tecnología está tan estandarizada que se comercializa en forma de módulos independientes listos para integrar, como el popular sensor PX4Flow (Figura 3.6), que incorpora su propia lente y un procesador dedicado exclusivamente a calcular estos desplazamientos a alta velocidad.



Figura 3.6: Módulo PX4Flow, un sensor de odometría visual inteligente muy utilizado en robótica aérea de código abierto [5].

Aunque este sistema de odometría visual es vital para lograr un vuelo estático estable, presenta deficiencias físicas para la navegación. Al tratarse de un sistema de medición puramente relativo (mide el cambio posicional desde el fotograma anterior, no desde un origen espacial global), acumula un error matemático progresivo en la estimación de la posición. Este fenómeno se conoce en la robótica como deriva (*drift*).

Además, la fiabilidad del flujo óptico es dependiente de las condiciones del entorno físico. Si el vehículo vuela sobre un suelo carente de textura (como baldosas lisas o superficies muy reflectantes) o si la iluminación de la sala es deficiente, el algoritmo es incapaz de encontrar puntos de interés estables en los píxeles. Ante esta situación,

el sistema pierde su referencia espacial, lo que suele derivar en la pérdida total de la estabilización.

3.3.2. Estimación de profundidad mediante Deep Learning

La solución más directa y estandarizada en la industria para recuperar la tercera dimensión es sustituir la cámara monocular por un sensor tridimensional físico. Las cámaras de profundidad o RGB-D (como la familia *Intel RealSense* o *Microsoft Kinect* [6]) incorporan proyectores infrarrojos, que son utilizados para generar un mapa de profundidad en tiempo real.



Figura 3.7: Ejemplo de una cámara de profundidad RGB-D [6].

Sin embargo, es complicado integrar este tipo de hardware en plataformas aéreas de bajo coste. Estas cámaras requieren conexiones de alto ancho de banda (como USB 3.0), fuentes de alimentación dedicadas y poseen un peso que supera la capacidad de carga útil de los micro-UAVs.

Ante la imposibilidad de utilizar hardware 3D dedicado, la comunidad investigadora ha logrado emular este comportamiento mediante software, utilizando Inteligencia Artificial. El objetivo es entrenar redes neuronales capaces de calcular la geometría tridimensional de una escena a partir de una simple imagen plana (RGB).

Uno de los mayores proyectos es la familia de modelos **MiDaS** (Monocular Depth Estimation), desarrollada por investigadores del *Intelligent Systems Lab* [7]. Han conseguido que al procesar el flujo de vídeo de una cámara monocular, esa red neuronal sea capaz de predecir un mapa de profundidad relativo. Es fundamental destacar que, al carecer de una escala métrica absoluta, este mapa no proporciona distancias físicas reales, por lo tanto, el uso exclusivo de esta red resulta insuficiente para tareas de posicionamiento o localización espacial del vehículo. En las representaciones visuales generadas por MiDaS, las zonas cálidas representan los objetos más cercanos a la lente, mientras que los tonos fríos indican el fondo del entorno (Figura 3.8).

Aunque este enfoque elimina la necesidad de cargar con hardware pesado en el dron, traslada el cuello de botella al ámbito del software. El cálculo de mapas de profundidad densos requiere una gran potencia de cálculo, teniendo que usar procesadores gráficos dedicados (GPUs) de alto rendimiento para lograr la baja latencia necesaria para navegar en tiempo real.

3.3.3. Sistemas SLAM visual

Para que un vehículo autónomo pueda navegar en un entorno completamente desconocido, se enfrenta a un problema cíclico porque para saber dónde está, necesita un mapa del entorno; pero para construir ese mapa, necesita saber exactamente dónde

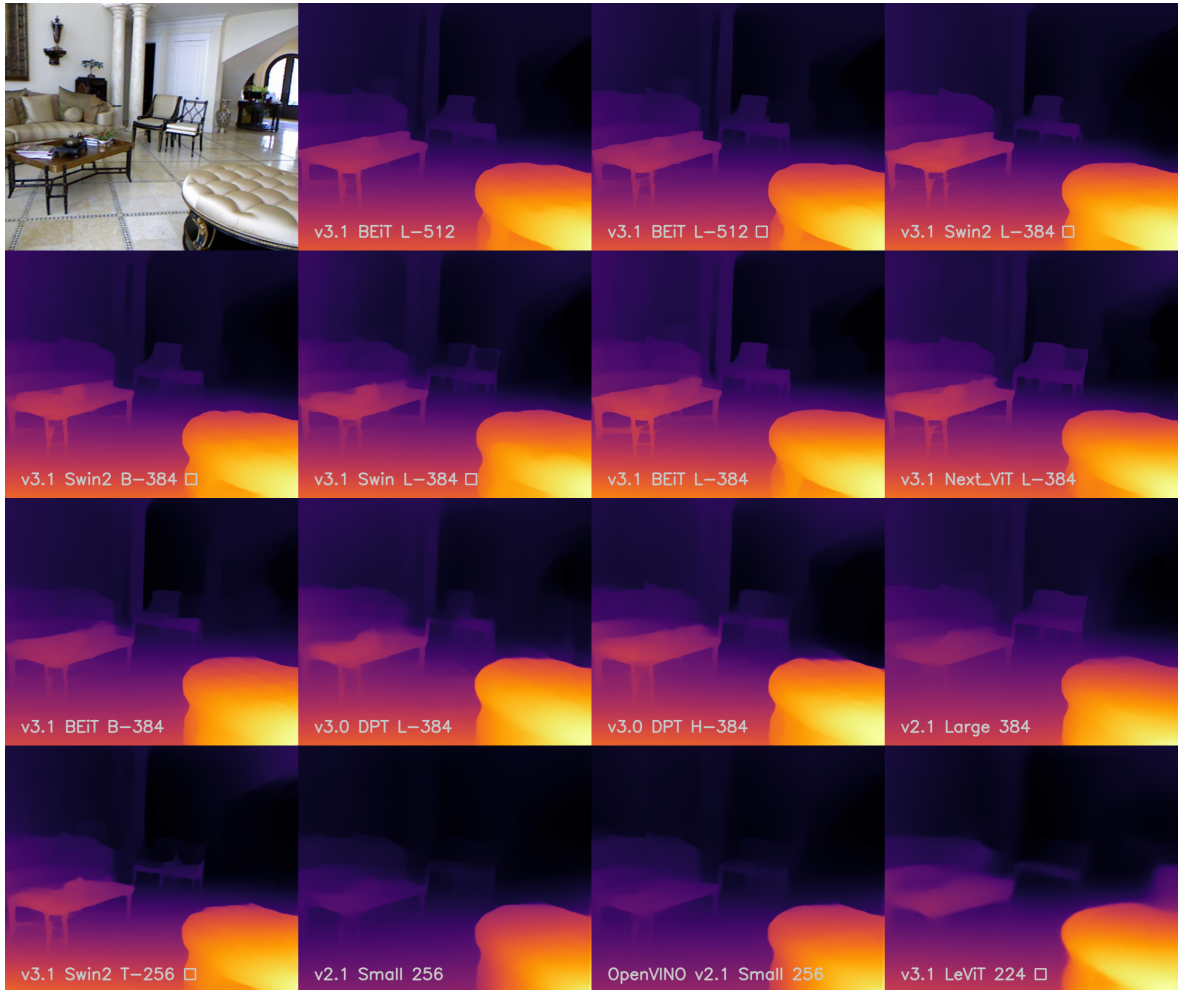


Figura 3.8: Comparativa de inferencia de profundidad utilizando diferentes versiones de la arquitectura neuronal MiDaS a partir de una única imagen RGB (esquina superior izquierda) [7].

está. La robótica lo resuelve mediante algoritmos de Localización y Mapeo Simultáneo (SLAM).

Cuando esta técnica se realiza utilizando exclusivamente cámaras, se denomina SLAM Visual (vSLAM). El algoritmo extrae miles de puntos característicos de los fotogramas de vídeo (como esquinas de muebles o bordes de ventanas) y rastrea su movimiento a lo largo del tiempo. Triangulando estos puntos desde diferentes ángulos a medida que la cámara se mueve, el software es capaz de construir una nube de puntos tridimensional del entorno mientras calcula la trayectoria exacta de la lente dentro de esa misma nube.

El estándar en la comunidad investigadora de código abierto es la familia de algoritmos **ORB-SLAM**⁴. Su iteración más reciente y avanzada, **ORB-SLAM3** [8], representa el estado del arte actual en la localización robótica visual.

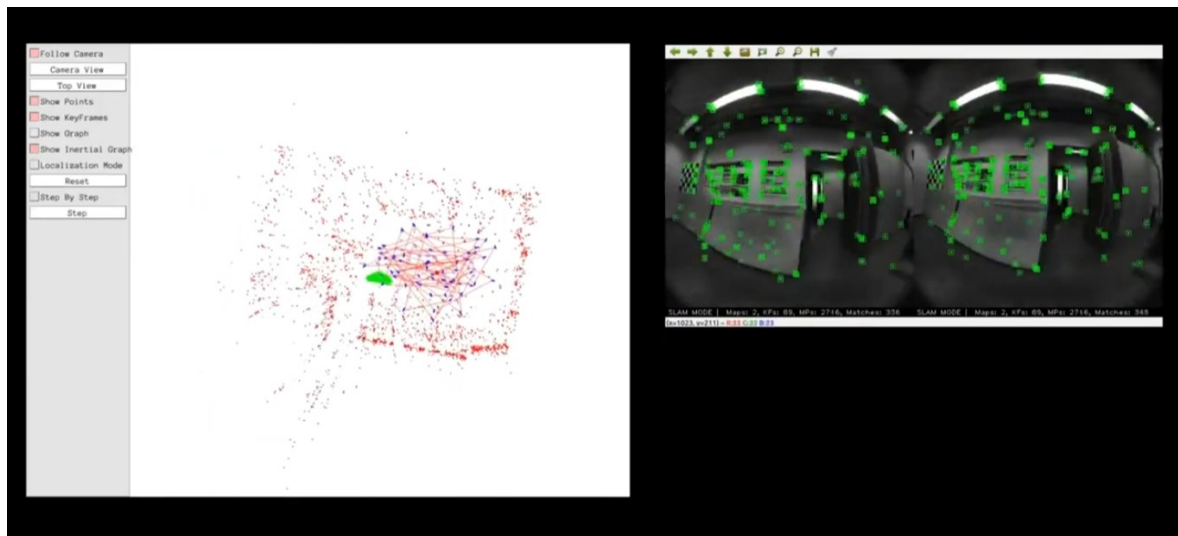


Figura 3.9: Visualización del sistema ORB-SLAM3 operando en modo estéreo (dos cámaras). A la derecha, la extracción de características en los dos flujos de vídeo simultáneos; a la izquierda, la nube de puntos 3D y la trayectoria generada [8].

A nivel arquitectónico, ORB-SLAM3 está diseñado para procesar información de una amplia gama de sensores. El algoritmo alcanza su máximo rendimiento cuando se alimenta con cámaras estéreo (como se aprecia en los dos flujos de vídeo de la Figura 3.9) o cámaras de profundidad (RGB-D), ya que estas configuraciones proporcionan la escala métrica real del entorno de forma nativa.

Para el caso de operar con una cámara de una sola lente, la falta de profundidad exige la integración estricta de una Unidad de Medición Inercial (IMU) de alta precisión. La fusión de los datos visuales con la aceleración física del IMU⁵ es el único mecanismo matemático que permite al algoritmo monocular deducir la escala tridimensional real y evitar errores de traslación cuando ocurren movimientos bruscos o rotaciones sobre su propio eje.

⁴ORB (*Oriented FAST and Rotated BRIEF*): Es un algoritmo de visión artificial extremadamente rápido y eficiente utilizado para detectar y describir puntos de interés en una imagen, siendo libre de las patentes que limitaban a sus predecesores como SIFT o SURF.

⁵IMU (*Inertial Measurement Unit*): Dispositivo electrónico compuesto generalmente por un acelerómetro y un giroscopio, que mide e informa acerca de la fuerza específica, la tasa angular y la orientación del vehículo.

El salto hacia el mapeo colaborativo

Una de las innovaciones más importantes de la tecnología actual de SLAM, es su capacidad de *Multimap* o mapeo colaborativo. En el ámbito industrial moderno, los almacenes o fábricas no se mapean con un único dron. La industria despliega flotas o enjambres de robots simultáneamente.

Los algoritmos actuales permiten que varios robots exploren distintas zonas de un edificio al mismo tiempo. Cuando el software detecta que dos drones están mirando al mismo sitio, fusiona instantáneamente las nubes de puntos individuales de ambos vehículos para crear un único “mapa global” compartido. Esto sienta las bases para arquitecturas distribuidas, donde la información espacial que descubre un vehículo puede ser utilizada inmediatamente por el resto del enjambre para navegar de forma segura.

3.3.4. Marcadores visuales en robótica industrial

En los apartados anteriores se ha expuesto cómo los algoritmos de odometría visual y las redes neuronales dependen de la textura del entorno o de un alto poder computacional para deducir el espacio tridimensional. Sin embargo, en entornos industriales reales (como almacenes logísticos o cadenas de montaje), las paredes suelen ser lisas, monocolor y con iluminación artificial cambiante, lo que provoca que los algoritmos visuales convencionales fracasen.

Para resolver este problema sin recurrir a hardware costoso, la industria robótica estandarizó el uso de marcadores visuales. Estos marcadores son como balizas artificiales diseñadas para ser detectadas por una cámara de la forma más rápida y robusta posible.

Consisten en un cuadrado negro que contiene una matriz binaria interna (una cuadrícula de píxeles blancos y negros). El borde negro exterior permite al algoritmo de visión artificial localizar el cuadrado en la imagen a alta velocidad analizando los gradientes de contraste. Por su parte, la matriz interna codifica un identificador único (ID) y proporciona un patrón que indica al sistema en qué orientación está girado el marcador.



Figura 3.10: Ejemplo de marcador ArUco.



Figura 3.11: Ejemplo de marcador AprilTag.

ArUco frente a AprilTag

A simple vista, como se aprecia en las Figuras 3.10 y 3.11, los diferentes tipos de marcadores parecen idénticos. Sin embargo, en el estado del arte existen dos grandes familias que compiten por el estándar industrial: **ArUco** [19] y **AprilTag** [20]. La Tabla 3.1 resume las principales diferencias que determinan su elección en un proyecto robótico.

Tabla 3.1: Comparativa técnica entre los sistemas de marcadores visuales más extendidos.

Característica	Familia ArUco	Familia AprilTag
Origen	Universidad de Córdoba (España).	Universidad de Michigan (EE.UU.).
Integración	Nativa dentro de la librería estándar OpenCV. Muy fácil de implementar en Python/C++.	Requiere compilar librerías externas o usar adaptadores de terceros.
Rendimiento	Coste computacional extremadamente bajo. Ideal para microprocesadores o drones ligeros.	Ligeramente más pesado para la CPU debido a su algoritmo de detección de bordes.
Robustez	Excelente en condiciones controladas. Falla si el marcador no es detectado por completo.	Más robusto frente a oclusiones y mala iluminación.

Estimación geométrica y aplicaciones

El verdadero potencial de estos marcadores no reside únicamente en leer su ID (como haría un código QR comercial), sino en su geometría matemática. Al conocer previamente el tamaño exacto del marcador, el sistema de visión artificial aplica algoritmos para calcular la pose completa de la cámara respecto al marcador.

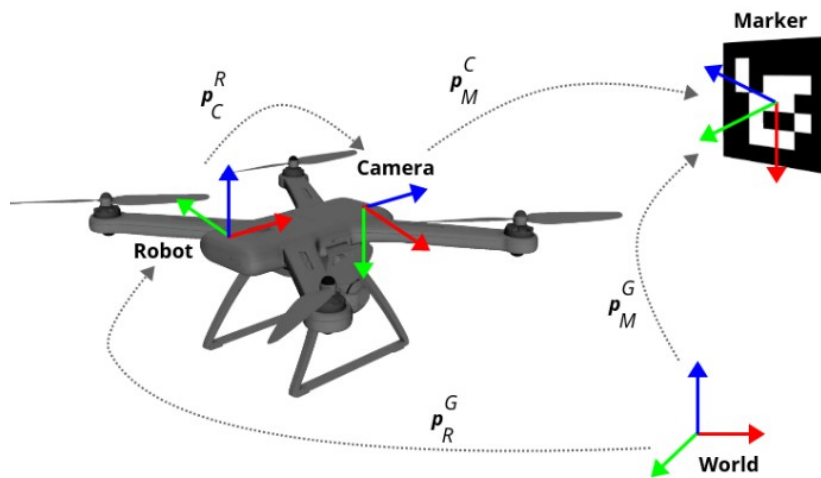


Figura 3.12: Esquema de transformación de coordenadas: al detectar las esquinas del marcador 2D, el dron calcula los vectores de traslación y rotación tridimensionales [9].

Como se observa en la Figura 3.12, una vez detectado el marcador, el vehículo proyecta un sistema de coordenadas (X, Y, Z). Esto le permite saber a cuántos metros exactos se encuentra y con qué ángulo de inclinación se está aproximando.

Debido a su bajo coste (solo requiere imprimirlo), estos marcadores se han convertido en el método preferido para dotar de referencias espaciales a entornos industriales. Su uso abarca desde plataformas de aterrizaje de precisión para drones, hasta el etiquetado de columnas y contenedores en laboratorios y almacenes logísticos (Figura 3.13).

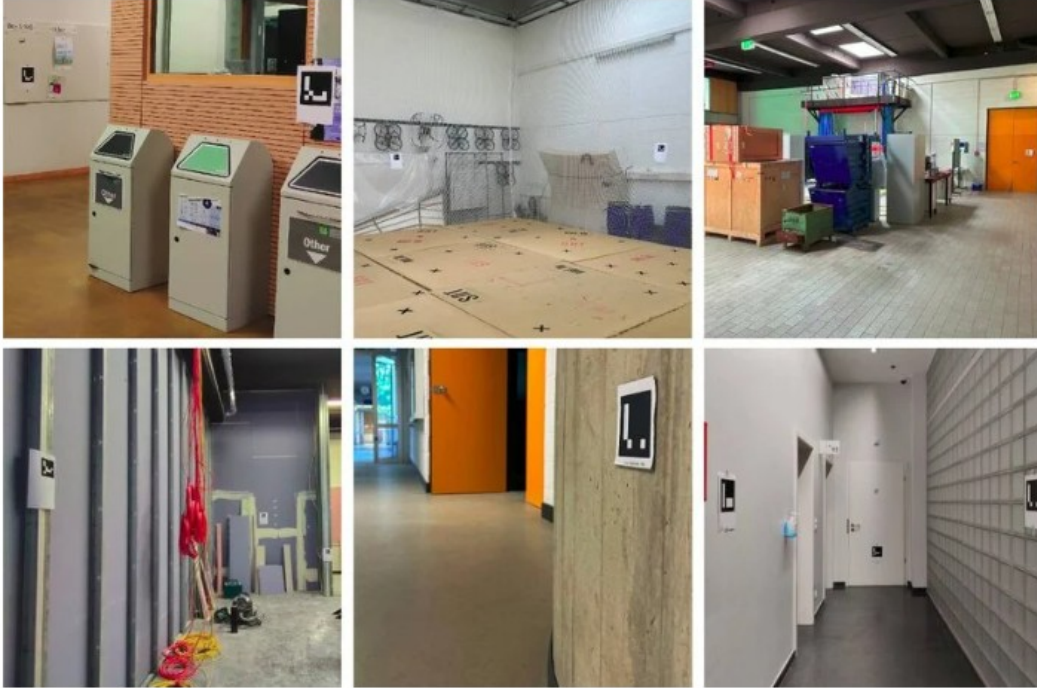


Figura 3.13: Diferentes ejemplos de uso industrial y académico de marcadores visuales [9].

3.3.5. Algoritmos geométricos y estimación de pose

Una vez que el sistema de localización ha detectado los puntos de interés en la imagen, el reto técnico se traslada al mundo matemático: proyectar esos píxeles hacia el mundo tridimensional real. Este proceso se conoce como el problema PnP (*Perspective-n-Point*).

El objetivo del problema PnP es calcular la pose exacta de una cámara calibrada respecto a un objeto tridimensional conocido. Para resolver esta incógnita, la industria robótica estandarizó el uso de la librería de código abierto **OpenCV** (Open Source Computer Vision Library), apoyándose en su algoritmo iterativo `solvePnP` [10].

Como se ilustra en la Figura 3.14, el modelo matemático requiere relacionar los puntos del mundo físico con sus proyecciones en la lente. Si se conoce la matriz de parámetros intrínsecos de la cámara (K) —que define su distancia focal y centro óptico— y se introducen un mínimo de cuatro puntos, el algoritmo es capaz de devolver el vector de traslación ($\vec{t}$) y el vector de rotación ($\vec{r}$) que indican a cuántos metros y con qué ángulo exacto se encuentra el objeto respecto a la lente óptica del dron.

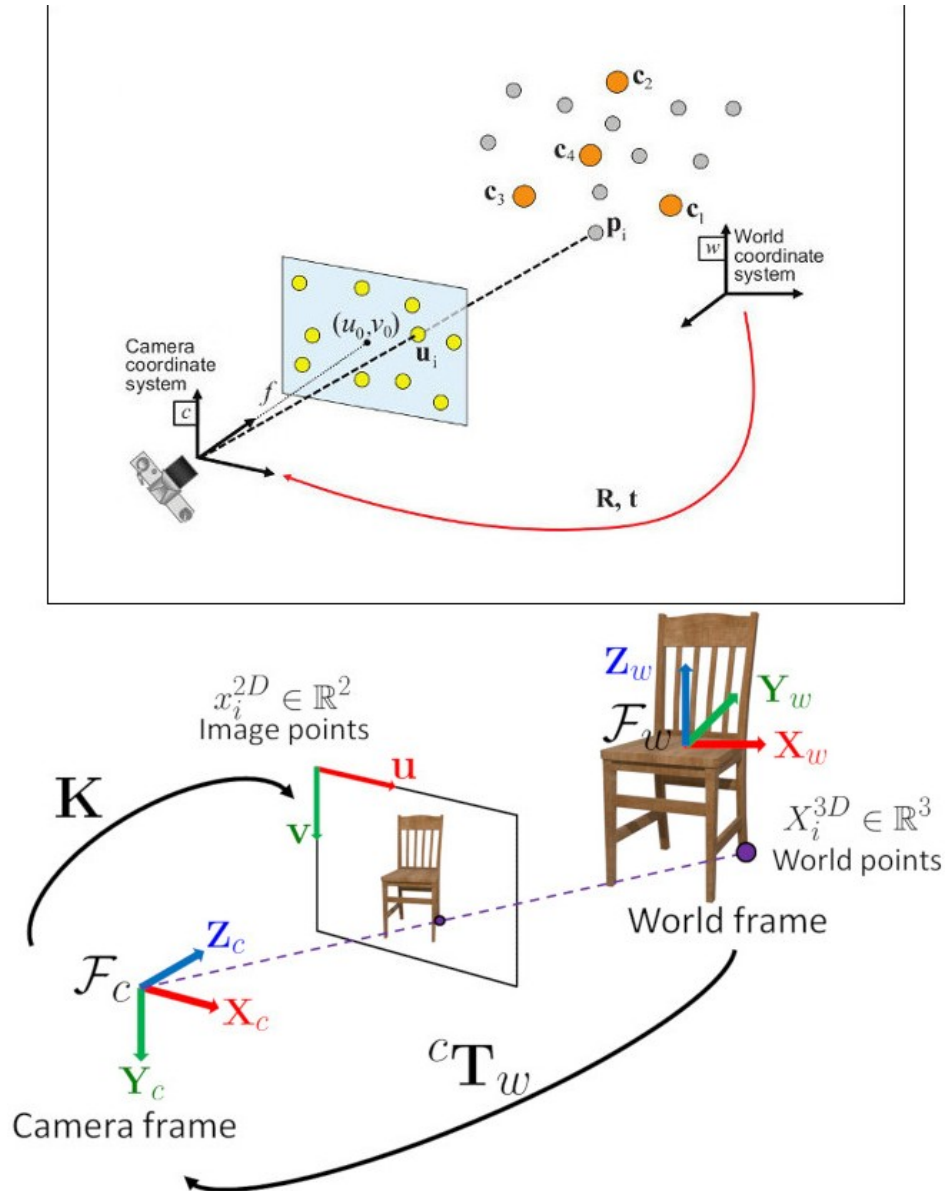


Figura 3.14: Esquema geométrico del problema PnP [10].

El árbol de transformaciones y la librería tf2

Aunque el algoritmo `solvePnP` soluciona la ubicación del objeto respecto a la cámara, esto introduce un nuevo desafío: el dron no es solo una cámara voladora. En un vehículo autónomo funcionan múltiples sensores simultáneamente, y cada uno de ellos posee su propio origen de coordenadas.

Si la cámara detecta una puerta a 2 metros en su eje Z (hacia delante), pero el controlador de vuelo espera las instrucciones relativas a su propio centro de masas (cuyo eje Z apunta hacia arriba), los comandos de navegación no funcionarían. Este problema de sincronización espacial se vuelve aún más complejo al introducir redes *Multi-mapa* con múltiples vehículos en la industria.

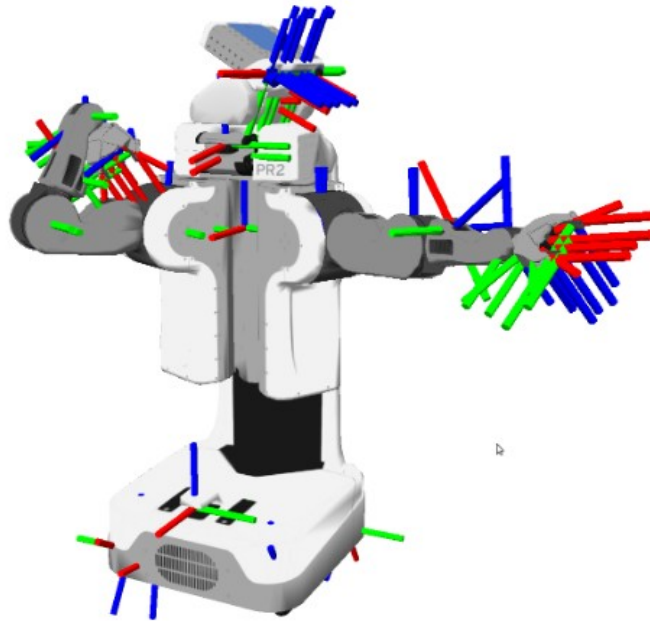


Figura 3.15: Visualización del árbol de transformaciones en un robot humanoide (PR2). Cada articulación y sensor opera con su propio sistema de referencia local, requiriendo un seguimiento continuo para lograr coherencia global [11].

Para resolver este sistema matemático, se utilizan árboles cinemáticos de transformación. En el ecosistema ROS 2, esta tarea se realiza con la librería `tf2` [11]. Como se visualiza en la Figura 3.15, esta herramienta permite mantener un registro temporal de la relación geométrica entre todos los sistemas de referencia del robot (llamados *frames*).

Gracias a `tf2`, se puede hacer que las coordenadas tridimensionales de la puerta, originalmente calculadas desde la perspectiva de la lente (*camera_frame*), se transformen automáticamente al sistema de referencia estático y global del mapa (*map_frame*).

3.4. Control cinemático y planificación de trayectorias

Una vez resuelto el problema de la localización espacial —es decir, lograr que el sistema comprenda dónde se ubica el vehículo y dónde están sus objetivos—, el siguiente eslabón en la arquitectura de un sistema autónomo es la actuación. El gran desafío radica en cómo transformar esas coordenadas en comandos de movimiento que controlen los motores del dron de forma segura.

A diferencia del entorno simulado, donde los movimientos pueden ser inmediatos y perfectos, en el mundo físico se dan restricciones mecánicas y aerodinámicas, como la inercia, la fricción del aire o las imperfecciones del propio vehículo. En esta sección se exponen las bases teóricas necesarias para controlar el vuelo, analizando en primer lugar las estrategias de control que permiten a un robot alcanzar una velocidad o posición precisa, y en segundo lugar, las lógicas de navegación utilizadas para trazar el camino más eficiente hacia su destino.

3.4.1. Estrategias de control de vuelo

En la navegación robótica, el cálculo de la ruta hacia un objetivo produce continuamente una métrica conocida como *vector de error* ($e(t)$). Este valor representa la diferencia entre la posición real del vehículo en un instante determinado y la posición de destino deseada. El objetivo de un sistema de control es calcular la señal de actuación necesaria para reducir ese error a cero.

En el estado del arte de la industria, existen diferentes enfoques para abordar este problema físico, siendo los controladores P y PID los más extendidos [21].

Control proporcional (P)

La estrategia más elemental e intuitiva es el control puramente Proporcional. En este modelo, la velocidad enviada a los motores es directamente proporcional a la magnitud del error actual, multiplicada por una constante de ganancia (K_p):

$$u(t) = K_p \cdot e(t) \quad (3.1)$$

Si el dron está muy lejos de la puerta, viaja muy rápido; a medida que se acerca, la velocidad disminuye. Aunque este enfoque es matemáticamente correcto y funciona, puede fracasar debido a diferentes factores, como la fricción aerodinámica o la inercia cuando el error es muy pequeño, porque la señal de velocidad generada es tan pequeña que no logra vencer la resistencia física del dron. Esto provoca que el vehículo se quede estancado cerca del objetivo sin llegar nunca a alcanzarlo. Si se intenta solucionar aumentando la constante K_p , el sistema se vuelve inestable y comienza a oscilar alrededor de la meta.

Controlador PID (Proporcional-Integral-Derivativo)

Para resolver las deficiencias del control P puro, la industria estandarizó el uso de controladores PID. Este algoritmo es ampliamente utilizado, porque no solo reacciona

al error actual, sino que tiene en cuenta el pasado y predice el futuro [21]. Matemáticamente, se define como la suma de tres acciones de control distintas:

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (3.2)$$

Cada componente asume un rol físico fundamental en la estabilización del vuelo:

- **Acción Proporcional (K_p):** Proporciona la fuerza de empuje principal basada en el error presente.
- **Acción Integral (K_i):** Acumula el error a lo largo del tiempo. Si el dron se queda estancado a 10 cm del objetivo por falta de fuerza, esta componente va sumando ese pequeño error, aumentando progresivamente la orden de velocidad hasta forzar a los motores a vencer la fricción aerodinámica.
- **Acción Derivativa (K_d):** Actúa como un amortiguador predictivo. Calcula la tasa de cambio del error (a qué velocidad se acerca el dron a la meta). Si el dron se acerca demasiado rápido, genera una fuerza de frenado contraria para evitar que se pase de largo (*overshoot*).

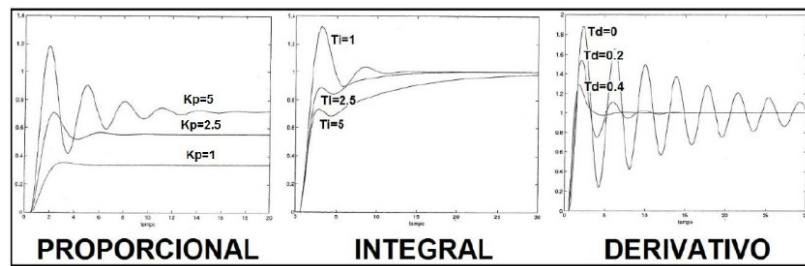


Figura 3.16: Curva de respuesta de un controlador PID. Se observa cómo la sintonización correcta de los parámetros permite alcanzar el objetivo de forma rápida, mitigando el *overshoot* y estabilizando el sistema en el tiempo [12].

Como se ilustra en la Figura 3.16, la correcta sintonización de estas tres constantes (K_p , K_i , K_d) permite que un sistema físico alcance su objetivo de la forma más rápida y estable posible.

3.4.2. Navegación espacial

El controlador PID detallado en el apartado anterior es el encargado de ejecutar los movimientos, pero necesita saber hacia dónde dirigirse. La navegación espacial es la capa lógica encargada de generar los comandos de referencia espaciales, y en la robótica móvil se aborda habitualmente desde dos paradigmas distintos [22].

Navegación punto a punto

Es la estrategia de navegación más elemental. En este modelo, el sistema recibe una coordenada espacial absoluta en el espacio tridimensional (X, Y, Z) y el controlador calcula un vector directo entre la posición actual del vehículo y dicho destino.

En un entorno virtual perfecto, el dron rotaría hacia el objetivo y avanzaría en una línea recta. Sin embargo, en un entorno físico, si el dron sufre una perturbación lateral, el sistema recalcula un nuevo vector directo desde su nueva posición. El resultado es que el vehículo nunca recupera la trayectoria original, lo que hace que en espacios cerrados aumente el riesgo de colisión.

Seguimiento de trayectorias

Para entornos donde la precisión es importante, la industria adopta algoritmos de seguimiento de trayectoria. En lugar de limitarse a alcanzar un punto final, este enfoque exige que el vehículo se desplace a lo largo de un “carril” geométrico predefinido.

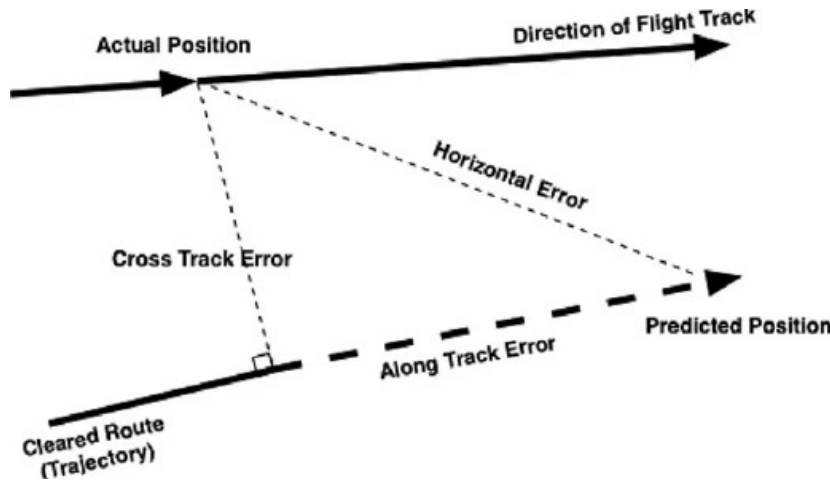


Figura 3.17: Ilustración de los errores de navegación. El algoritmo de seguimiento de trayectorias se centra en minimizar el error lateral (*cross-track error*) para forzar al vehículo a regresar a la ruta planificada [13].

La principal diferencia radica en cómo se calcula el error. Como se puede observar en la Figura 3.17, en el seguimiento de trayectorias, el algoritmo calcula de forma continua el *cross-track error* (error lateral), es decir, la distancia perpendicular entre el centro de masas del dron y la línea imaginaria trazada en el mapa tridimensional. Por lo tanto, el controlador PID calculará comandos de velocidad para forzar al dron a regresar al vector original, garantizando un avance estable y predecible a lo largo del trayecto.

3.5. Ecosistemas de desarrollo en robótica

A medida que los sistemas robóticos aumentan en complejidad —integrando control de vuelo espacial, procesamiento de visión artificial y toma de decisiones simultáneas—, resulta insostenible programar el vehículo mediante un único bloque de código monolítico. Para gestionar esta concurrencia de procesos, la industria se apoya en infraestructuras de software especializadas que manejan la comunicación entre los diferentes componentes del robot de forma segura y eficiente.

3.5.1. Middleware robótico: ROS 2

El estándar global en la robótica moderna es **ROS** (*Robot Operating System*). A pesar de su nombre, no es un sistema operativo tradicional como Windows o Ubuntu, sino un *middleware*: una capa de abstracción de software que se instala por encima del sistema operativo del ordenador y proporciona herramientas, librerías y convenciones para escribir código de robótica de forma modular.

Bajo el paradigma de ROS, el cerebro del robot no se programa como un único archivo monolítico, sino que se divide en pequeños programas independientes con una función muy específica, denominados **Nodos**. Por ejemplo, un dron puede ejecutar un nodo exclusivo para leer las imágenes de la cámara, otro nodo dedicado a calcular el error PID, y un tercer nodo encargado de enviar comandos a los motores.

Para que estos nodos independientes puedan colaborar, ROS establece una arquitectura de comunicaciones basada en los siguientes elementos [14]:

- **Topics (Temas):** Son canales de comunicación de flujo continuo, ideales para datos que cambian constantemente, como el vídeo de una cámara o la telemetría. Utilizan una arquitectura de *Publicador/Suscriptor*. Un nodo (publicador) envía datos al canal sin saber quién los leerá, y cualquier otro nodo (suscriptor) puede “escuchar” ese canal para recibir los datos.
- **Services (Servicios):** A diferencia de los topics, se utilizan para interacciones bajo una arquitectura de *Cliente/Servidor*. Un nodo solicita una acción concreta (como encender los LEDs) y el sistema espera hasta recibir una respuesta de confirmación antes de continuar.

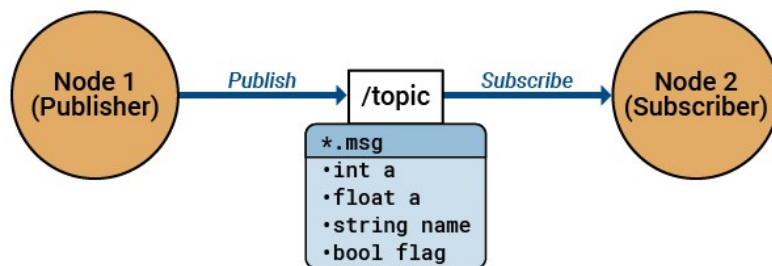


Figura 3.18: Arquitectura de comunicación en ROS 2 [14].

El salto evolutivo: De ROS 1 a ROS 2

Durante más de una década, la primera versión del sistema (ROS 1) dominó el mercado. Sin embargo, presentaba una vulnerabilidad arquitectónica para la industria: dependía de un nodo maestro central (conocido como **roscore**). Si este ordenador central se bloqueaba o perdía la conexión momentáneamente, la comunicación entera se paralizaba.

Para resolver este cuello de botella y mejorar su funcionamiento, se desarrolló **ROS 2**. El mayor avance tecnológico de esta nueva versión fue la eliminación total del nodo maestro, sustituyendo su núcleo de red por el estándar de comunicaciones **DDS** (*Data Distribution Service*) [15].

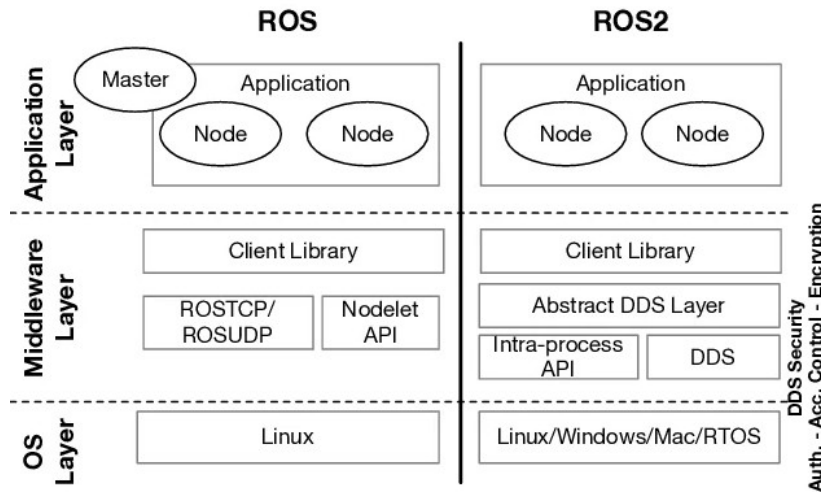


Figura 3.19: Comparativa arquitectónica. A la izquierda, la topología centralizada de ROS 1, dependiente del *ROS Master*. A la derecha, la topología distribuida de ROS 2 operando sobre el bus de datos estándar DDS [15].

Como se observa en la Figura 3.19, el protocolo DDS proporciona una arquitectura completamente descentralizada. Los nodos ahora pueden crearse y negociar conexiones entre sí de forma autónoma (*peer-to-peer*) mediante un bus de datos compartido globalmente.

Aunque el presente proyecto se enfoca en el desarrollo y control de un único vehículo aéreo, la elección de ROS 2 frente a otras alternativas más simples responde a un criterio de ingeniería fundamental: la **escalabilidad**. Gracias al estándar DDS, la arquitectura de software desarrollada en este proyecto está preparada de forma nativa para el despliegue de múltiples vehículos (enjambres o *swarms*).

3.5.2. Herramientas de visualización y análisis

El desarrollo de algoritmos de navegación autónoma implica que el vehículo tomará decisiones en tiempo real basándose en su percepción matemática del entorno. Programar y depurar un sistema de este tipo observando únicamente el vuelo exterior del dron resulta ineficiente e inseguro, ya que es imposible detectar a simple vista si un choque se debe a que la cámara detecta mal, a un retraso en la red Wi-Fi o a un cálculo erróneo de la controladora.

Para abordar este desafío, el ecosistema ROS 2 proporciona de forma nativa el siguiente conjunto de herramientas de análisis:

Herramientas de interfaz de línea de comandos

La forma mas rápida de analizar un vuelo en tiempo real se realiza a través de la terminal del sistema. Comandos de monitorización nativos como `ros2 topic list` o `ros2 topic echo` permiten auditar la red en tiempo real. Por ejemplo, es posible verificar si los fotogramas de la cámara están llegando al ordenador con la frecuencia (FPS) esperada, o imprimir en pantalla los valores numéricos que el algoritmo de navegación está calculando y mandando a los motores, todo ello sin interrumpir la ejecución del código principal.

RViz2: El estándar de visualización tridimensional

Cuando el análisis numérico resulta insuficiente para entender el comportamiento espacial del vehículo, la industria recurre a **RViz2** (*ROS Visualization*) [23]. Es importante destacar que RViz2 no es un simulador físico, sino un entorno de visualización 3D que puede proyectar en tiempo real tanto los datos físicos obtenidos de los sensores (como la pose espacial o capturas de vídeo) como los cálculos que realizan los algoritmos (como la planificación de rutas).

La principal ventaja técnica que justifica su uso frente a otros motores gráficos (como Unity o Unreal Engine) es su integración con el protocolo de red. RViz2 actúa como un nodo más de ROS 2 que se suscribe a los *topics* ya existentes, renderizando la información instantáneamente.

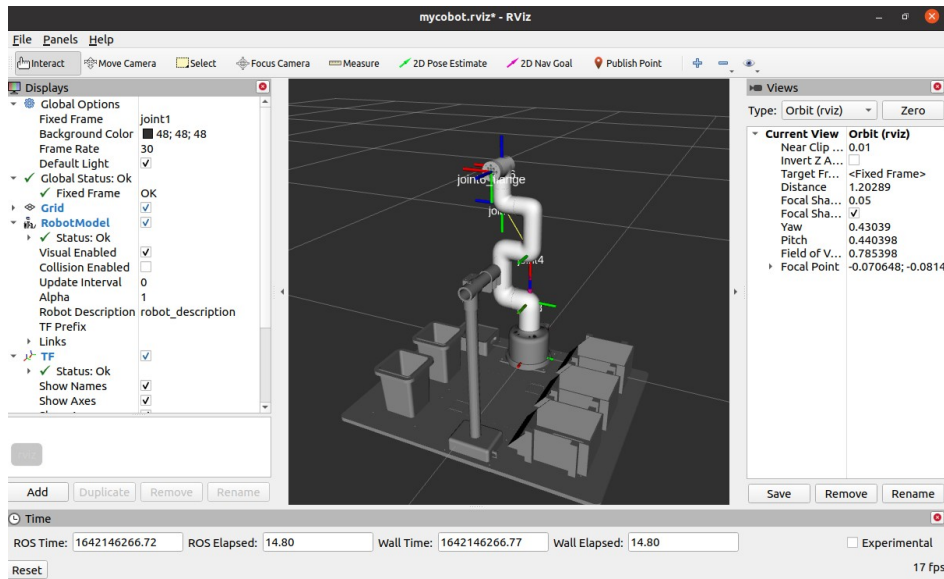


Figura 3.20: Interfaz gráfica de RViz2 representando un ejemplo industrial [16].

Como se aprecia en el ejemplo de la Figura 3.20, RViz2 permite renderizar los *topics* de forma muy sencilla, seleccionándolos del listado de *topics* detectados.

3.6. Entornos de simulación robótica

Validar software en el ámbito de la robótica aérea conlleva un riesgo físico y económico, ya que un simple error puede provocar la colisión del vehículo. Para reducir este riesgo durante las fases de desarrollo, se emplean entornos de simulación robótica. Estas plataformas permiten ejecutar el código en un gemelo digital⁶ antes de transferirlo al hardware físico.

En el estado del arte actual, destacan tres grandes herramientas de simulación, cada una con un enfoque arquitectónico distinto:

⁶Un gemelo digital (*Digital Twin*) es una réplica virtual exacta de un producto, proceso o sistema físico, que permite realizar pruebas y simulaciones de comportamiento bajo condiciones reales sin poner en riesgo el hardware original.

Gazebo

Históricamente, Gazebo se ha consolidado como el simulador estándar de código abierto en la comunidad robótica. Destaca respecto a sus rivales por la alta fidelidad que ofrece su simulación física, además de tener una integración nativa con ROS2.

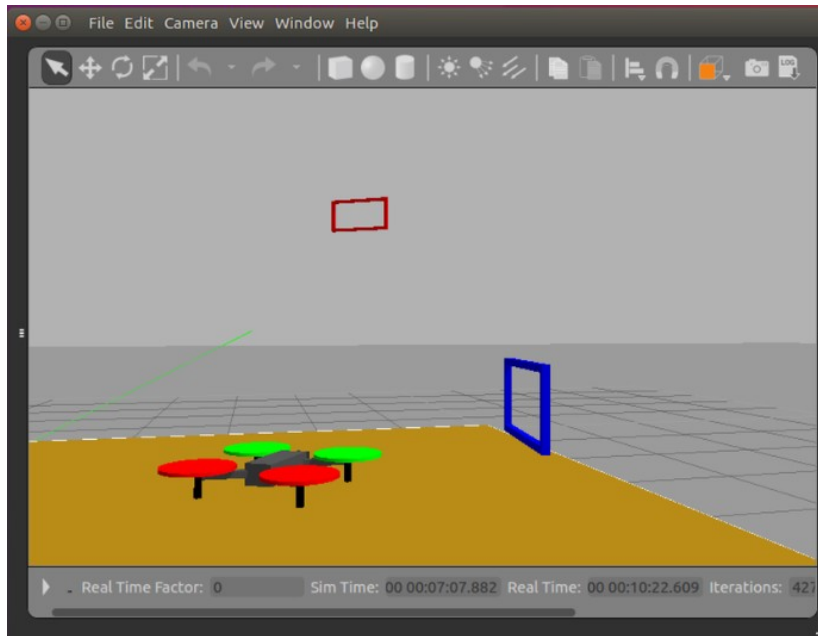


Figura 3.21: Captura de un entorno de simulación de UAVs desarrollado en Gazebo [17].

Sin embargo, como se observa en la Figura 3.21, modelar mundos complejos o escenarios personalizados en Gazebo resulta complicado. Requiere desarrollar archivos de descripción de hardware (SDF o URDF)⁷ y su interfaz de usuario es muy simple, lo que eleva significativamente la curva de aprendizaje y el tiempo de despliegue para nuevos proyectos.

Webots

Desarrollado originalmente por Cyberbotics, Webots se presenta como una alternativa altamente eficiente y ágil. Destaca por resolver las mayores deficiencias de Gazebo: ofrece una interfaz gráfica mucho más moderna que facilita el diseño y la modificación de mundos 3D.

Además, cuenta con dos ventajas clave: una integración fluida y nativa con la arquitectura de nodos de ROS 2, y un extenso catálogo de modelos preconfigurados. Esta librería incluye multitud de sensores, actuadores y robots comerciales listos para ser instanciados, lo que reduce el tiempo necesario para poner en marcha una simulación compleja.

⁷SDF (*Simulation Description Format*) y URDF (*Unified Robot Description Format*) son formatos basados en XML que describen la cinemática, la dinámica y las propiedades visuales de un robot u objetos, para que el simulador o ROS 2 puedan interpretarlo.

NVIDIA Isaac Sim

Representa una nueva generación impulsada por Inteligencia Artificial [24]. Mientras que Gazebo y Webots equilibran el cálculo físico y el rendimiento general, Isaac Sim está diseñado específicamente para la validación de algoritmos de visión artificial avanzados.

Utilizando renderizado por trazado de rayos (*Ray Tracing*), genera mundos fotorealistas con reflejos, sombras dinámicas y físicas de iluminación idénticas a las del mundo real. Este nivel de detalle es crítico para entrenar redes neuronales, pero conlleva un coste: requiere una alta potencia computacional, siendo obligatorio el uso de tarjetas gráficas (GPUs) dedicadas de gama alta para mantener la simulación en tiempo real.

Capítulo 4

Metodología

Para desarrollar un sistema autónomo de navegación hay que elegir tanto el hardware físico sobre el que operará el vehículo, como el entorno de software utilizado para su programación y simulación. En este capítulo se describen las plataformas utilizadas, los entornos virtuales y el flujo de trabajo adoptado para llevar el proyecto desde una fase conceptual hasta su validación en el mundo real.

4.1. Plataforma hardware: DJI Tello

Para la validación física de los algoritmos desarrollados, se ha seleccionado el cuadricóptero comercial DJI Tello. Tras el análisis de las plataformas aéreas realizado en la Sección 3.2, se determinó que este vehículo representa un equilibrio óptimo para este proyecto. A diferencia de las plataformas de arquitectura abierta como PX4 (Figura 3.3), que resultan excesivamente voluminosas y peligrosas para un entorno de laboratorio, el DJI Tello ofrece un marco de operación seguro gracias a sus protecciones de hélices y su peso reducido de apenas 80 gramos. Asimismo, frente a opciones como el *Crazyflie* (Figura 3.4), el Tello cuenta con una cámara frontal integrada de mayor resolución nativa, lo cual es determinante para el correcto funcionamiento de los algoritmos de detección. Sin embargo, más allá de estas ventajas técnicas, la elección final se basó en que era la plataforma disponible en el laboratorio de la Universidad.

Especificaciones técnicas

A nivel de hardware, el DJI Tello es un micro-UAV diseñado con restricciones físicas para maximizar su portabilidad y bajo coste. Su limitación más crítica para el ámbito de la robótica autónoma es su nula capacidad de carga útil (*payload*). A diferencia de drones de investigación mayores, intentar añadir unos gramos extra (como una Raspberry Pi Zero o sensores adicionales) desestabiliza su centro de masas y reduce su autonomía teórica de 13 minutos, haciéndolo incapaz de despegar con seguridad.

En cuanto a sensores nativos, el vehículo contiene una cámara frontal (720p) a 30 fotogramas por segundo, que será utilizada en la fase de detección. Adicionalmente, el dron incorpora el sistema de posicionamiento inferior analizado en la Sección 3.3.1, compuesto por una cámara de flujo óptico y un sensor infrarrojo de tiempo de vuelo o ToF¹. Esta combinación permite al controlador de vuelo interno mantener la estabilidad

¹ToF (*Time-of-Flight*): Sistema de telemetría que determina la distancia entre el emisor y un



Figura 4.1: Dron comercial DJI Tello utilizado para las pruebas de vuelo en el entorno físico.

en vuelo estático (*hovering*) de forma autónoma.

Cabe destacar que, como se detalló en el estado del arte, este sistema presenta limitaciones físicas que condicionan directamente la metodología de este trabajo:

- **Deriva espacial (*Drift*):** Al carecer de un módulo GPS, el error acumulado por la odometría visual impide que el dron mantenga coordenadas perfectas a largo plazo.
- **Dependencia lumínica y de superficie:** El flujo óptico requiere suelos con textura y una buena iluminación, mientras que el sensor ToF puede fallar sobre superficies reflectantes o materiales oscuros que absorban el espectro infrarrojo. Estos factores técnicos se han tenido en cuenta al configurar el escenario físico de pruebas para evitar la pérdida de referencia.

Arquitectura *Offboard* e interfaz de comunicación

Debido a la limitación de carga útil del vehículo, que imposibilita embarcar hardware de cómputo adicional, y al diseño cerrado de su sistema integrado, cuyo procesador no permite la ejecución de ningún tipo de código o script personalizado por parte del usuario, se ha adoptado una arquitectura de procesamiento estrictamente *offboard*².

Para interactuar con los motores y los sensores del vehículo, el DJI Tello expone de forma nativa una interfaz de red abierta basada en el protocolo UDP³. Toda la lógica

obstáculo midiendo el tiempo que tarda una señal lumínica en ir y volver tras rebotar en la superficie.

²*Offboard computing*: Arquitectura donde el procesamiento computacional se delega a una estación base externa, en lugar de ejecutarse en el hardware integrado del propio vehículo.

³UDP (*User Datagram Protocol*): Protocolo de red rápido y sin conexión, ideal para transmisiones

de visión, el controlador matemático y la planificación de trayectorias se ejecutan en una estación externa (ordenador portátil) que se conecta a la red Wi-Fi local generada por el propio dron a 2.4 GHz.

A través de esta conexión inalámbrica por UDP, el dron es capaz de emitir su telemetría y flujo de vídeo, así como recibir comandos de vuelo. La accesibilidad de este protocolo ha propiciado que la comunidad de desarrolladores de código abierto haya creado diversas librerías de control que facilitan su integración con lenguajes como Python. La selección de la librería a utilizar, se detallará en el Capítulo 5.

4.2. Entorno de simulación: Webots

Desarrollar algoritmos de navegación autónoma directamente sobre una plataforma física conlleva un alto riesgo de colisiones. Para evitar daños en el hardware durante las primeras fases del proyecto, la validación inicial de la lógica matemática y de control se llevó a cabo en **Webots**, un simulador robótico de código abierto.

Como se expuso en la Sección 3.6, existen diversas herramientas de simulación. Aunque Gazebo es el estándar más extendido en la comunidad, modelar escenarios personalizados en él exige la manipulación de archivos descriptivos (SDF o URDF) y presenta una curva de aprendizaje pronunciada. Para este Trabajo Fin de Grado, se optó por Webots debido a que su interfaz gráfica facilita la creación rápida de mundos tridimensionales. Esta decisión permitió reducir drásticamente el tiempo dedicado al modelado 3D para poder centrar los esfuerzos del proyecto en la programación algorítmica.

A nivel de arquitectura, la conexión entre el simulador y el código se realizó mediante el paquete oficial *webots_ros2*⁴. Este paquete funciona como un puente bidireccional: extrae los datos de los sensores virtuales (como la cámara del dron) y los publica nativamente como *topics* de ROS 2, a la vez que lee los comandos de velocidad generados por el código y los aplica a los motores del simulador.

El propio paquete incluye modelos de drones comerciales preconfigurados, como el *Crazyflie* o el *DJI Mavic 2 Pro*. Puesto que modelar y simular las dinámicas de vuelo del DJI Tello desde cero quedaba fuera del alcance de este trabajo, se decidió utilizar el modelo virtual del **Mavic 2 Pro**. Esta elección se justifica en que su interfaz de control es análoga a la del Tello: permite extraer su pose espacial y acepta comandos de velocidad lineal (x, y, z) y rotación (yaw). Utilizar este modelo garantizó que el código diseñado en simulación pudiera transferirse al dron físico sin reescribir la estructura de los nodos.

Finalmente, aprovechando la sencillez del entorno, se diseñó un escenario de pruebas virtual básico. Se modelaron estructuras a modo de puertas (Figura 4.2). De esta forma, la cámara del dron simulado percibía un entorno equivalente al que se montaría posteriormente en el laboratorio.

en tiempo real como el flujo de vídeo (*streaming*), donde la velocidad prima sobre la garantía de entrega de todos los paquetes.

⁴Repositorio oficial del puente entre Webots y ROS 2: https://github.com/cyberbotics/webots_ros2

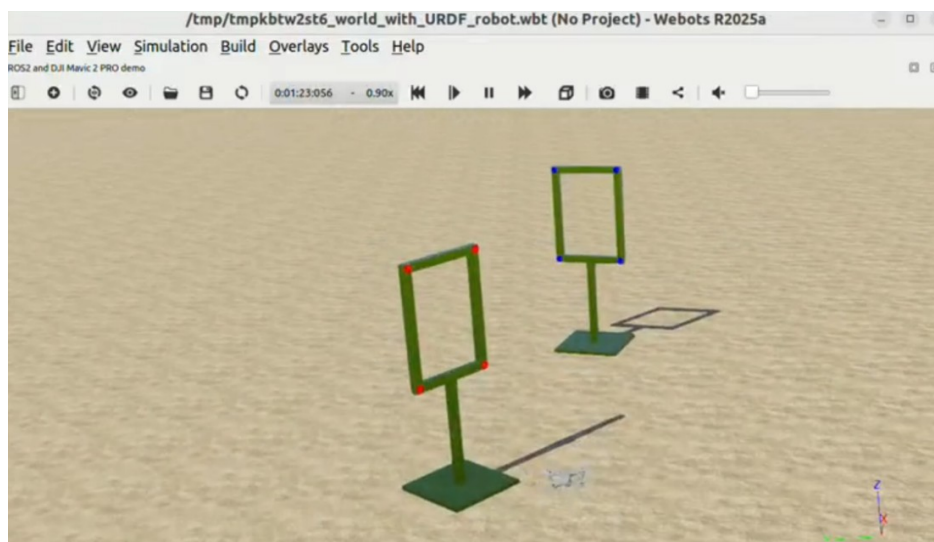


Figura 4.2: Entorno de simulación en Webots mostrando el modelo virtual del Mavic 2 Pro y las puertas creadas.

4.3. Entorno de software: ROS 2 y Python

El ecosistema de software se ha desarrollado íntegramente utilizando **Python** como lenguaje principal. Esta decisión se fundamenta en su agilidad para el prototipado algorítmico y su integración directa con las dos librerías esenciales del proyecto: *OpenCV* para el procesamiento de visión artificial y *NumPy* para la resolución matricial de las transformaciones espaciales.

Como infraestructura de comunicaciones se ha empleado el *middleware* **ROS 2**. La decisión técnica de no programar el sistema como un único archivo monolítico, sino de aislar las diferentes lógicas (visión, control y hardware) en distintos programas independientes de Python, responde a criterios de ingeniería del software: mejora el rendimiento, facilita la legibilidad del código y permite sustituir o actualizar módulos en el futuro sin afectar al resto del sistema.

Al fragmentar el código de esta manera, surgió el reto técnico de mover un gran volumen de datos (como la señal de video, o datos de sensores) a muy alta velocidad entre los distintos programas. Para resolver esta necesidad, se integró ROS 2. Su función en este proyecto es gestionar el paso continuo de mensajes mediante *topics*, y favorecer así también la visualización. Toda esta división modular se alinea con las directrices arquitectónicas oficiales establecidas por el proyecto ROS 2 [25], las cuales fundamentan el uso del estándar DDS para aislar procesos y garantizar comunicaciones de baja latencia en robótica móvil. El diseño de esta división y la función de cada módulo se defenderán en profundidad en el Capítulo 5.

4.3.1. Herramientas de monitorización y depuración

El desarrollo de un sistema autónomo requiere verificar continuamente que el flujo de datos procesado es correcto. Para ello, se integraron en el ciclo de trabajo dos mecanismos de supervisión nativos de ROS 2:

En primer lugar, la interfaz de línea de comandos. Herramientas estándar como

`ros2 topic list` y `ros2 topic echo` se utilizaron para verificar en tiempo real los datos de los sensores, o verificar los datos publicados tras los correspondientes cálculos.

En segundo lugar, para la validación espacial, se empleó el entorno **RViz2** (detallado en la Sección 3.5.2). La herramienta se configuró para renderizar en tiempo real el origen de coordenadas global, la pose del dron, la posición calculada de las puertas, etc. Observar esta reconstrucción 3D de forma simultánea al vuelo físico del vehículo permitió confirmar empíricamente que los cálculos matemáticos coincidían con la realidad (Figura 4.3).

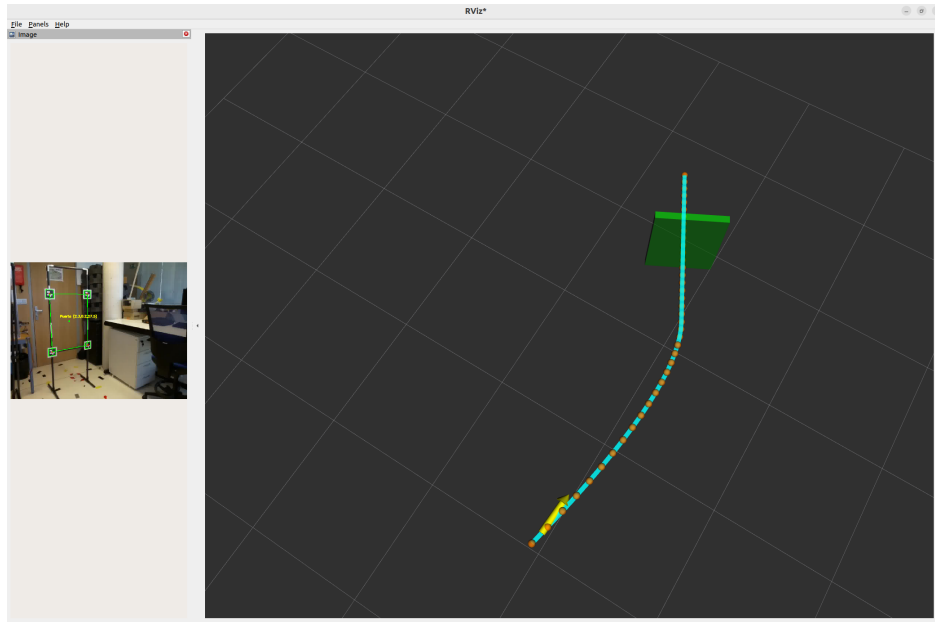


Figura 4.3: Visualización en RViz2 durante las pruebas físicas, mostrando la pose del dron, la estimación en el mapa 3D de la puerta detectada, y a la izquierda, la imagen de la cámara modificada en tiempo real.

4.4. Ciclo de desarrollo y pruebas

Para garantizar la seguridad del hardware y facilitar la detección de errores durante la programación, la metodología de trabajo se estructuró siguiendo un enfoque iterativo e incremental. El protocolo de validación para cualquier nuevo algoritmo desarrollado se dividió en tres fases operativas:

1. **Fase de simulación:** Toda la lógica de visión y navegación se programó y depuró primero en el entorno virtual de Webots. Esta fase permitió comprobar que las transformaciones matemáticas eran correctas y aislar los errores lógicos del código sin poner en riesgo la integridad física del vehículo aéreo.
2. **Fase de pruebas estáticas:** Una vez validada la lógica en el simulador, el código se conectaba al Tello real. En esta etapa, el dron se mantenía sujeto manualmente o posado en el suelo. El objetivo era verificar de forma segura que los cálculos se realizaban correctamente.

3. **Fase de vuelo dinámico:** La etapa final consistió en el vuelo autónomo dentro del laboratorio. Esta fase sirvió para revelar problemas que el simulador no contemplaba. De hecho, estrategias matemáticas que funcionaron perfectamente en la Fase 1 (como la navegación puramente proporcional o punto a punto), fracasaron en esta etapa, obligando a iterar el diseño y volver a la fase de simulación para desarrollar soluciones más robustas.

4.4.1. Secuenciación de tareas y diagrama de Gantt

Siguiendo una metodología iterativa y ágil, el desarrollo del Trabajo Fin de Grado se estructuró desde su inicio en ocho paquetes de trabajo secuenciales:

- **Fase 1. Estado del arte y tecnologías:** Revisión bibliográfica de soluciones robóticas, estudio de las limitaciones de navegación en interiores y selección del *stack* tecnológico.
- **Fase 2. Setup de simulación Webots y ROS 2:** Despliegue de la arquitectura base en Python, configuración de la comunicación y diseño del escenario tridimensional.
- **Fase 3. Detección visual:** Calibración de la cámara del DJI Tello y programación de los algoritmos de detección para extraer coordenadas tridimensionales.
- **Fase 4. Navegación:** Programación de la máquina de estados y desarrollo de la lógica de control proporcional.
- **Fase 5. Primeras pruebas físicas:** Despliegue del código inicial sobre el hardware real en el laboratorio.
- **Fase 6. Iteración y optimización:** Fase dedicada a la corrección de errores detectados en la primera interacción con el entorno real.
- **Fase 7. Validación y vuelo autónomo definitivo:** Ejecución de nuevos ensayos físicos para certificar el funcionamiento del sistema y validar el cumplimiento de los objetivos.
- **Fase 8. Redacción de la memoria del TFG:** Tarea de carácter continuo que abarca la documentación y justificación del proyecto.

Para ilustrar la planificación temporal de estas tareas, se presenta el diagrama de Gantt en la Figura 4.4.



Figura 4.4: Diagrama de Gantt mostrando la planificación temporal.

Capítulo 5

Diseño e implementación de la arquitectura

Este capítulo detalla el núcleo tecnológico del Trabajo Fin de Grado. Para facilitar la comprensión del sistema, la estructura de este capítulo refleja la metodología de trabajo empleada durante la fase de programación: el problema general de la navegación autónoma se dividió en módulos o programas independientes. A continuación, se expone la arquitectura global que los interconecta y, en los subapartados posteriores, se explica en detalle desde el origen de cada problema técnico hasta su solución final.

5.1. Arquitectura global del sistema

El software de este proyecto se ha diseñado bajo una arquitectura distribuida basada en el *middleware* ROS 2. El sistema se ha dividido en cuatro programas independientes (nodos) programados en Python. Esta fragmentación responde a dos necesidades importantes de ingeniería: el rendimiento de las comunicaciones y la escalabilidad a futuro.

En primer lugar, respecto al rendimiento, el DJI Tello solo permite mantener una única conexión de red activa. Si la lectura de los sensores, el procesamiento pesado de visión artificial y el cálculo de trayectorias se ejecutaran secuencialmente en un mismo programa, el retardo generado por el análisis de imagen bloquearía el envío de velocidades a los motores, provocando que el dron se volviera inestable o se estrellara. Al separar los programas mediante ROS 2, el nodo de comunicación se ejecuta a máxima velocidad, leyendo sensores y publicando en los *topics* sin esperar a que el nodo de visión termine de analizar su fotograma.

En segundo lugar, respecto a la **escalabilidad y modularidad**, esta arquitectura garantiza que el código sea completamente reutilizable. Si en el futuro se desea cambiar el dron por otro modelo comercial (o por uno montado a medida), bastaría con programar un nuevo nodo de comunicación que sustituya al actual. De igual forma, si el objetivo de la misión dejara de ser “cruzar puertas” y pasara a ser por ejemplo, inspeccionar válvulas en un edificio, bastaría con modificar exclusivamente el nodo de detección para que busque válvulas.

La interacción y el flujo de información entre estos cuatro programas se ilustra en el diagrama de la arquitectura (Figura 5.1):

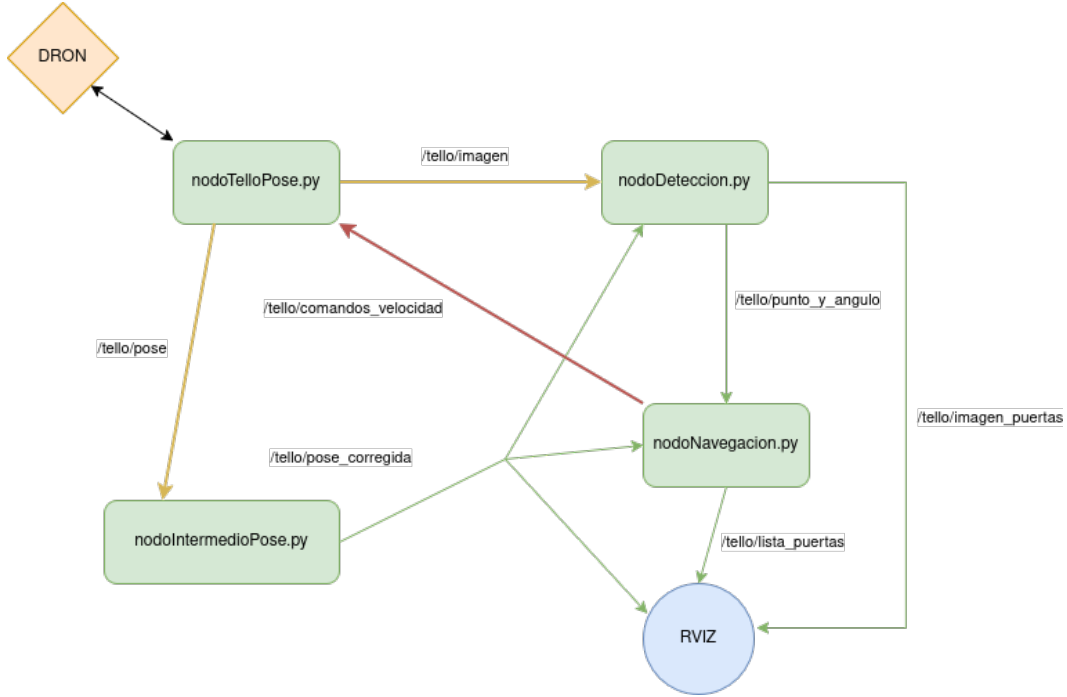


Figura 5.1: Diagrama de la arquitectura de software en ROS 2, mostrando los cuatro nodos principales y el flujo de datos a través de topics.

Como se observa en el diagrama, cada nodo cumple el principio de responsabilidad única, asumiendo tareas muy delimitadas dentro del ciclo de detección y actuación:

- **nodoTelloPose:** Es el “driver” de bajo nivel. Su única función es mantener viva la conexión UDP con el dron, leer la telemetría y el flujo de vídeo para inyectarlos en la red lo más rápido posible, y recibir los comandos de velocidad para aplicarlos a los motores. Al no tener lógica de decisión, su tiempo de ejecución es mínimo (Apartado 5.2).
- **nodoIntermedioPose:** Actúa como un filtro de corrección para el *optical flow* (Apartado 5.2).
- **nodoDeteccion:** Es el motor de visión artificial. Se encarga de procesar el *topic* de imagen y calcular las coordenadas tridimensionales de los objetivos (Apartado 5.3).
- **nodoNavegacion:** Representa el cerebro lógico del sistema. Recibiendo los datos de la telemetría y de los objetivos detectados, gestiona una máquina de estados para calcular las velocidades de vuelo (Apartado 5.4).

Para garantizar la robustez de las comunicaciones entre estos nodos, no se utilizaron las configuraciones por defecto de los *topics* de ROS 2, sino que se utilizaron políticas de Calidad de Servicio (QoS, *Quality of Service*) específicas según la necesidad de los datos. Como se observa en el siguiente fragmento del `nodoDeteccion.py` (Código 5.1, para la transmisión del flujo de vídeo se configuró una política `BEST_EFFORT` con un historial de profundidad 1, priorizando la menor latencia posible y permitiendo descartar fotogramas si la red se satura. Por el contrario, para la recepción de la pose

espacial, se exigió una política **RELIABLE**, asegurando a nivel de protocolo de red que ninguna coordenada se pierda:

```

1  # QoS para video: Priorizar latencia (se pueden perder paquetes
   )
2  qos_profile_sub = QoSProfile(
3      reliability=ReliabilityPolicy.BEST_EFFORT,
4      history=HistoryPolicy.KEEP_LAST, depth=1)
5  self.suscripcion_imagen = self.create_subscription(
6      Image, ROS_TOPIC_IMAGEN_RAW_INPUT,
7      self.callback_procesamiento_imagen, qos_profile_sub)
8
9  # QoS para pose: Priorizar fiabilidad (no perder datos)
10 qos_profile_pose = QoSProfile(
11     reliability=ReliabilityPolicy.RELIABLE,
12     history=HistoryPolicy.KEEP_LAST, depth=10)
13 self.suscripcion_pose = self.create_subscription(
14     Float32MultiArray, ROS_TOPIC_POSE_ANGLES,
15     self.callback_pose, qos_profile_pose)

```

Código 5.1: Configuración de perfiles QoS diferenciados para vídeo y telemetría.

5.2. Módulo de telemetría

El primer gran desafío técnico de este proyecto fue dotar al sistema de una estimación de pose tridimensional (X, Y, Z, yaw) fiable. En entornos de interiores, la ausencia de señal GPS obliga al vehículo a depender exclusivamente de sus sensores o de la visión artificial para determinar su posición. Esta sección documenta el proceso iterativo de investigación llevado a cabo: desde una primera aproximación basada en SLAM visual, hasta el descubrimiento y desarrollo de la solución definitiva.

Para la interpretación de los siguientes ensayos, se anticipa que el sistema toma como referencia general la convención aeronáutica estándar de ROS 2 (eje X para el avance frontal, Y para el desplazamiento lateral izquierdo y Z para la altitud positiva hacia arriba).

5.2.1. El enfoque inicial: DJITelloPy y ORB-SLAM3

En la fase inicial del desarrollo, la comunicación con el dron se estableció a través del nodo `Tello`, utilizando la librería estándar `DJITelloPy` [26]. Aunque esta herramienta ofrecía una interfaz de programación accesible basada en métodos directos (*getters*), presentaba una limitación para la navegación autónoma: no proporcionaba las coordenadas espaciales absolutas (X, Y), limitándose a entregar una estimación de la altura y rotaciones.

Para solventar esta ausencia de odometría, se planteó extraer la posición tridimensional procesando el flujo de vídeo monocular mediante **ORB-SLAM3**, un algoritmo de localización y mapeo simultáneo avanzado.

El despliegue de ORB-SLAM3 supuso un reto de integración importante. El repositorio original está diseñado para versiones antiguas de Linux y para el ecosistema

ROS 1, lo que generaba severos conflictos de dependencias. Para superar esta barrera, se recurrió a una versión *dockerizada* compatible con ROS 2¹. Dado que esta imagen base estaba orientada a cámaras estéreo, se modificó su código fuente para habilitar la función monocular y se mandó el *topic* de imagen (`/tello/imagen`) directamente hacia el contenedor Docker.

Para que el algoritmo de SLAM pudiera inicializarse y proyectar los píxeles en un mundo 3D correctamente, fue un requisito indispensable realizar una rigurosa calibración intrínseca previa de la cámara del DJI Tello. Se desarrolló un script específico encargado de extraer fotogramas del flujo de vídeo a 30 FPS. Posteriormente, mediante la librería OpenCV, se procesaron las capturas utilizando un patrón de tablero de ajedrez de 6x4 esquinas internas.

A través de la función `findChessboardCorners`, el algoritmo detectó las intersecciones geométricas (Figura 5.2) y calculó la **Matriz de Intrínsecos** (K) de la cámara. Esta matriz es el núcleo matemático que relaciona las coordenadas 3D del mundo con el plano 2D de la imagen (de 960×720 píxeles), y proporcionó los siguientes valores para el Tello:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 921,17 & 0 & 474,31 \\ 0 & 920,53 & 362,48 \\ 0 & 0 & 1 \end{bmatrix} \quad (5.1)$$

Donde f_x, f_y representan la distancia focal y c_x, c_y el centro óptico de la imagen. Asimismo, el proceso de calibración devolvió los coeficientes de distorsión radial y tangencial ($D = [k_1, k_2, p_1, p_2]$), parámetros vitales para corregir computacionalmente la deformación esférica de la lente antes de inyectar las imágenes al algoritmo SLAM. En la Figura 5.3 se ilustra la ejecución del contenedor Docker en tiempo real, mostrando cómo el sistema extrae los puntos de interés del vídeo monocular para generar la nube de puntos del entorno.

5.2.2. Limitaciones físicas del SLAM monocular y alternativas

A pesar de conseguir integrar y ejecutar el contenedor Docker, las pruebas en el laboratorio demostraron que usar solo una cámara monocular no era práctico para navegar en interiores. Durante los vuelos, se identificaron tres problemas principales que hacían fallar la estimación de la posición:

1. **Ambigüedad de escala:** ORB-SLAM3 necesita extraer puntos clave (*keypoints*) de la imagen para crear el mapa. Al usar una sola lente, el mapa no tiene medidas reales en metros. Se intentó obtener un factor de conversión para pasar su escala a la del mundo real, pero dependía de dónde arrancaba el dron. Si despegaba mirando a una zona con muchos puntos de interés, la escala era distinta que si arrancaba frente a una pared más limpia visualmente.
2. **Falsa traslación al rotar (*Yaw*):** Para buscar las puertas o al navegar, el dron puede girar sobre sí mismo sin moverse del sitio. Si se daba ese caso, al no disponer ORB-SLAM de lectura de la IMU, el algoritmo fallaba creyendo que el dron se estaba desplazando lateralmente, generando error, normalmente, en el eje Y .

¹Repositorio ORB-SLAM3-ROS2-Docker:
ORB-SLAM3-ROS2-Docker

<https://github.com/suchetanrs/>

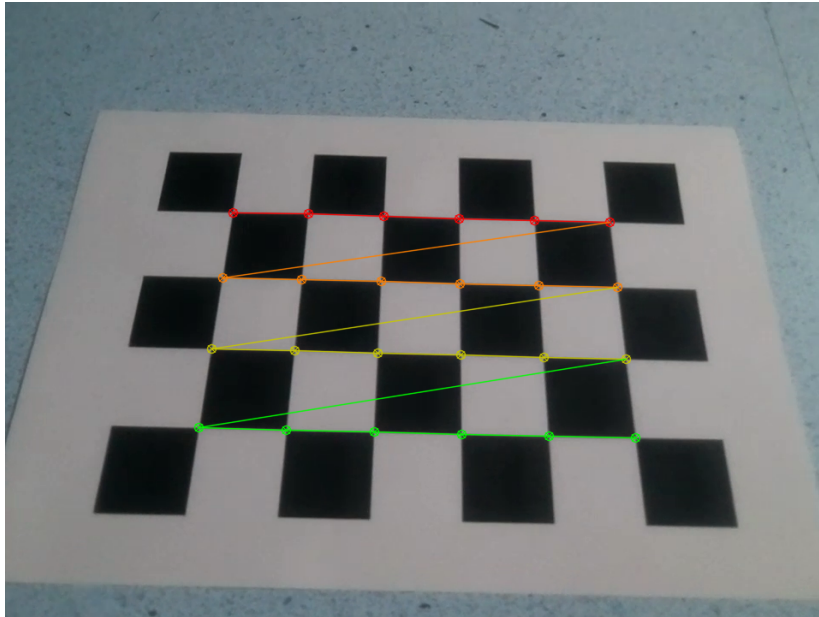


Figura 5.2: Detección geométrica y reproyección de esquinas internas durante el proceso de calibración de la cámara del dron.

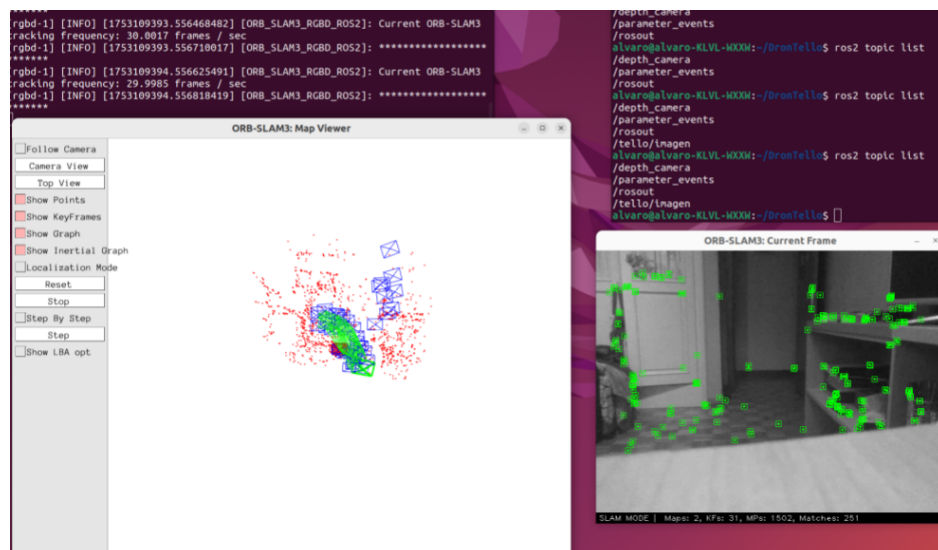


Figura 5.3: Captura del algoritmo ORB-SLAM3 procesando el flujo de vídeo monocular y generando la nube de puntos en tiempo real.

3. **Falta de texturas y efecto túnel:** Navegar por interiores es difícil para la visión artificial si hay zonas lisas sin referencias. Esto da lugar a tres situaciones problemáticas:

- **Fallos en la altura (Z):** Como se ve en la Figura 5.4, el algoritmo es capaz de extraer puntos de la zona inferior (la línea de las mesas, los ordenadores y los objetos), pero de ahí hacia arriba la pared es lisa. Si el dron asciende y pierde de vista los objetos de abajo, se queda sin referencias y el algoritmo falla.

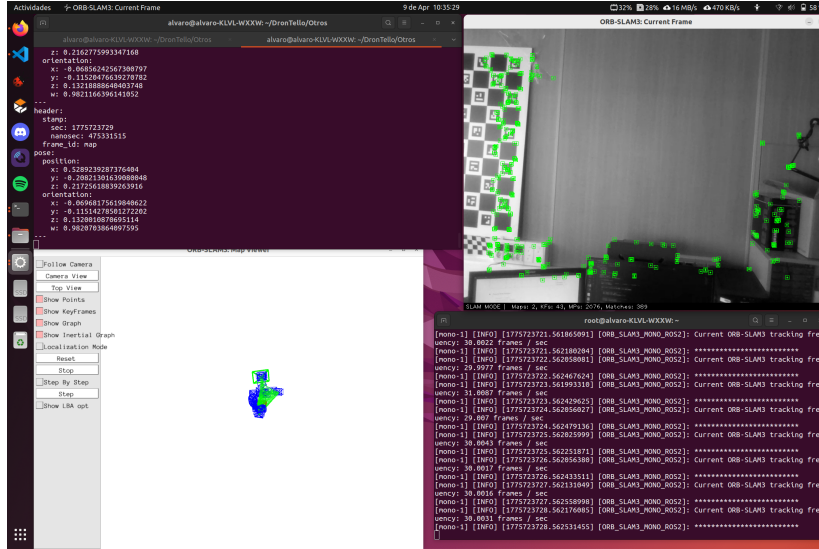


Figura 5.4: Pérdida de referencia de ORB-SLAM3 en el eje Z.

- **Pasillos con pocas referencias:** En la Figura 5.5 se observa al dron intentando mapear un pasillo del edificio. Al disponer de pocos puntos de referencia en las paredes laterales, el sistema falla al tratar de calcular la profundidad real, dificultando así la navegación.
- **El efecto túnel en las puertas:** La Figura 5.6 está tomada sin cruzar la puerta del laboratorio, a medio metro antes. Como se puede apreciar, en ese punto la cámara solo detecta puntos lejanos, como puede ser de la sala siguiente. Esto provoca que el algoritmo falle al calcular el avance: al mover el dron hacia delante, los puntos lejanos apenas cambian de tamaño en la imagen, por lo que el SLAM asume un avance mucho menor al realizado.

Intentos de solución y descarte de SLAM visual

Para implementar SLAM visual con una cámara monocular, es necesario fusionar las imágenes con los datos de una unidad de medición inercial (IMU) para obtener una estimación de pose robusta. Dado que el DJI Tello dispone de ambos sensores, la primera estrategia consistió en utilizar la modalidad monocular-inercial de ORB-SLAM3. Sin embargo, el docker utilizado carecía de soporte nativo para esta sincronización. Desarrollar un módulo de bajo nivel para sincronizar temporalmente los fotogramas del vídeo con la aceleración de la IMU superaba el alcance del proyecto. Por ello, se

modificó el código para operar en modo puramente monocular, pero la estimación de posición resultó inestable.

A continuación, se modificó la configuración de ORB-SLAM3 para forzar al cálculo de un mayor número de puntos de interés y evitar pérdidas de seguimiento en paredes lisas. Aunque el reconocimiento de la escena mejoró ligeramente, el coste computacional aumentó hasta el punto de reducir la tasa de fotogramas por segundo, introduciendo una latencia incompatible con el control en tiempo real.

Finalmente, se intentó utilizar la red neuronal MiDaS para inferir mapas de profundidad a partir de la cámara RGB y emular un sensor RGB-D. Esta aproximación se descartó por dos motivos: el elevado consumo de recursos necesario para generar mapas densos con la frecuencia que exige ORB-SLAM3, y la complejidad matemática de alinear la escala de la red con la del sistema SLAM.

Al comprobar que el SLAM visual no ofrecía una localización fiable bajo las restricciones de hardware y procesamiento del sistema, se descartó el uso de ORB-SLAM3.

5.2.3. Traslado a TelloPy y arquitectura basada en eventos

La inmensa mayoría de la documentación académica recomienda utilizar la librería DJITelloPy para controlar el Tello, la cual no proporciona la pose del dron. Sin embargo, la observación del comportamiento del dron en el laboratorio contradecía las limitaciones de software de dicha librería. Al volar, el Tello es capaz de mantener una posición estática en el aire de forma estable. Para lograrlo, el dron utiliza una cámara orientada hacia el suelo que analiza el desplazamiento de los píxeles fotograma a fotograma; una técnica conocida como **Optical Flow** (cuyos fundamentos teóricos se detallaron en el estado del arte).

Para que este flujo óptico logre el vuelo estacionario, resultaba evidente que el procesador interno del dron debía estar calculando y manejando coordenadas espaciales precisas. El reto consistía en encontrar una vía para extraer esos datos.

La investigación condujo a descubrir TelloPy ², una librería de más bajo nivel. A diferencia de la anterior, TelloPy opera mediante una arquitectura orientada a eventos asíncronos (*callbacks*), lo que permitió programar el `nodoTelloPose`, el *driver* definitivo del dron, estructurado en tres bloques clave.

Suscripción a eventos de vuelo

El núcleo de la telemetría se resolvió programando suscripciones a los paquetes de datos internos que el dron emite continuamente por Wi-Fi. El nodo reacciona a tres eventos principales:

- `EVENT_FLIGHT_DATA`: Proporciona la telemetría de alto nivel, de donde el código extrae la altura directa del altímetro y barómetro.
- `EVENT_VIDEO_FRAME`: Captura los paquetes del flujo de vídeo H.264 emitidos por la cámara principal.

²Repositorio de código abierto disponible en <https://github.com/hanyazou/TelloPy/tree/develop-0.7.0>.

- **EVENT_LOG_DATA**: Proporciona directamente los registros del módulo MVO³ y de la IMU.

A través del **EVENT_LOG_DATA**, se logró extraer por primera vez las métricas espaciales **mvo.pos_x**, **mvo.pos_y** y **mvo.pos_z**. Sin embargo, la orientación del dron no se recibe en ángulos, sino en **cuaterniones** (q_0, q_1, q_2, q_3). Un cuaternión es una estructura matemática de cuatro dimensiones utilizada en aeronáutica para representar orientaciones espaciales, evitando el problema del *Gimbal Lock* (la pérdida de un grado de libertad al girar 90 grados).

Para transformar esta representación en los ángulos de Euler (*roll*, *pitch*, *yaw*), se implementó el siguiente sistema de ecuaciones trigonométricas:

$$\begin{cases} \text{roll} = \arctan\left(\frac{2,0 \cdot (w \cdot x + y \cdot z)}{1,0 - 2,0 \cdot (x^2 + y^2)}\right) \\ \text{pitch} = \begin{cases} \text{sign}(2,0 \cdot (w \cdot y - z \cdot x)) \cdot \frac{\pi}{2,0} & \text{si } |2,0 \cdot (w \cdot y - z \cdot x)| \geq 1 \\ \arcsin(2,0 \cdot (w \cdot y - z \cdot x)) & \text{en otro caso} \end{cases} \\ \text{yaw} = \arctan\left(\frac{2,0 \cdot (w \cdot z + x \cdot y)}{1,0 - 2,0 \cdot (y^2 + z^2)}\right) \end{cases} \quad (5.2)$$

Decodificación de vídeo

El flujo de vídeo del dron se recibe codificado en H.264. Debido a que la decodificación en el hilo principal de ROS 2 introducía una latencia crítica para el control, se optó por una arquitectura de procesamiento externa basada en tuberías (*pipes*). Esta solución está inspirada en las implementaciones de la comunidad para el protocolo de Tello (especialmente en el repositorio *hanyazou/TelloPy*).

El nodo gestiona una instancia de **ffmpeg** mediante **subprocess**, configurada con parámetros de baja latencia para suprimir el uso de búferes internos. El flujo crudo se inyecta por la entrada estándar (*stdin*) y se extrae decodificado en formato MJPEG por la salida estándar (*stdout*):

```

1  ff_cmd = [
2      'ffmpeg', '-hide_banner', '-loglevel', 'error',
3      '-fflags', 'nobuffer', '-flags', 'low_delay',
4      '-f', 'h264', '-i', 'pipe:0',
5      '-r', str(FPS),
6      '-f', 'image2pipe', '-vcodec', 'mjpeg', 'pipe:1'
7  ]
8  self.ffmpeg = subprocess.Popen(
9      ff_cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE
10 )
    
```

Código 5.2: Configuración de **ffmpeg** para decodificación de baja latencia.

Un hilo secundario escanea continuamente el flujo de bytes en memoria buscando los delimitadores de inicio (`\xff\xd8`) y fin (`\xff\xd9`) de cada imagen JPEG. Una vez reconstruido el fotograma, se publica en el *topic* correspondiente, minimizando el retardo visual respecto a los métodos de decodificación estándar.

³Siglas empleadas en el protocolo interno de DJI para hacer referencia a las variables de odometría visual calculadas en el chip de visión Intel Movidius (*Movidius/Main Visual Odometry*). Este sistema estima el desplazamiento métrico del dron analizando el flujo óptico de la cámara inferior.

Recepción de comandos

Finalmente, para cerrar el ciclo de control, el nodo escucha constantemente el *topic* `/tello/comandos_velocidad`. La máquina de estados de navegación (que se explicará en el apartado 5.4) le manda un array de 4 valores de velocidad: `[Left/Right, Forward/Backward, Up/Down, Yaw]`.

No obstante, como mecanismo de seguridad ante la falta de un mando físico de radiocontrol, se diseñó un protocolo de interrupciones dentro del *callback* de recepción. Se reservaron combinaciones específicas de valores para poder generar esos comandos de seguridad, los cuales se recogen en la Tabla 5.1:

Array [lr, fb, ud, yv]	Acción Ejecutada	Descripción Técnica
[2.0, 0.0, 0.0, 0.0]	<code>land()</code>	Aterrizaje inmediato.
[2.0, 2.0, 0.0, 0.0]	<code>emergency()</code>	Corte inmediato de corriente a los motores.
[2.0, 2.0, 2.0, 0.0]	<code>takeoff()</code>	Despegue y estabilización a 1 metro de altura.
[0.0, 0.0, 0.0, 0.0]	Estabilización	Establece todas las velocidades a 0, cediendo el control al flujo óptico para estabilizar la posición.

Tabla 5.1: Protocolos de seguridad y comandos especiales codificados en el array de velocidades.

5.2.4. El problema del origen: `nodoIntermedioPose`

Una vez extraída la telemetría mediante la librería `TelloPy`, surgió un problema inesperado que hacía imposible la navegación directa. El sistema de Odometría Visual (MVO) del Tello no inicializa su posición en $(0, 0, 0)$ al despegar. Este comportamiento se debe a que la cámara del *optical flow* se encuentra situada en la parte inferior del vehículo. Al colocar el dron sobre una superficie para su posterior despegue, el sensor queda obstruido y pierde su referencia visual, lo que provoca que el sistema de odometría se inicialice con valores arbitrarios al despegar (como, por ejemplo, $X = 25,4$ e $Y = -10,2$).

Para solucionar este desfase, se desarrolló el `nodoIntermedioPose`, cuya función es actuar como un filtro de corrección dinámico.

Estrategia de estabilización y temporización

El primer paso para obtener un origen fiable es garantizar que los sensores se han estabilizado, por ello se implementó una lógica de seguridad basada en un temporizador de 11 segundos, definido por la constante `ESPERA_TAKEOFF = 11.0`. Este intervalo fue determinado de manera experimental mediante despegues en el laboratorio. El nodo permite el despegue, detecta la primera pose recibida y detiene cualquier procesamiento durante ese intervalo para permitir que el vehículo alcance un estado estacionario (*hovering*) limpio.

Cálculo matemático del offset

Transcurrida la espera, el nodo captura una única lectura de posición y ángulos, almacenándola como el *offset*. A partir de ese instante, todas las coordenadas que el dron publique son corregidas en tiempo real restando dicho valor inicial. La lógica matemática implementada se define mediante el siguiente sistema de ecuaciones de traslación:

$$\begin{cases} X_{corregida} = X_{raw} - X_{offset} \\ Y_{corregida} = Y_{raw} - Y_{offset} \\ Z_{corregida} = Z_{raw} - Z_{offset} \end{cases} \quad (5.3)$$

Cabe destacar que el error se limita al punto de inicialización. Una vez aplicado el *offset*, el *optical flow* mantiene la escala de las medidas, permitiendo una correcta navegación.

Lógica de implementación en Python

Esta operación se realiza de forma asíncrona mediante un hilo de ejecución (*Thread*) para no bloquear la recepción de datos. Una vez fijado el origen (`origin_fijado = True`), el nodo comienza a publicar en el *topic* `/tello/pose_corregida`.

```

1  # Calculo del desfase respecto al origen guardado
2  x_corr = x_raw - self.offset_pos[0]
3  y_corr = y_raw - self.offset_pos[1]
4  z_corr = z_raw - self.offset_pos[2]
5
6  out = Float32MultiArray()
7  out.data = [
8      float(x_corr),
9      float(y_corr),
10     float(z_corr),
11     float(roll_corr),
12     float(pitch_corr),
13     float(yaw_corr)
14 ]
15 self.pub_pose.publish(out)

```

Código 5.3: Cálculo de la pose corregida en `nodoIntermedioPose.py`

Lectura de la altura

Un detalle incorporado en este nodo fue la fusión de la altura. El Tello publica tres valores de *Z*: uno proveniente del barómetro, otro del altímetro y otro del sistema MVO (basado en la distancia visual al suelo). El `nodoIntermedioPose` calcula un *offset* específico para la *Z* tras despegar, leyendo el valor del altímetro, que es el más fiable en ese momento, y durante el vuelo lee la altura del MVO.

5.3. Módulo de detección

Una vez garantizada la estabilidad de la telemetría y comunicación con el dron, el siguiente gran desafío del proyecto consistió en dotar al sistema de detección espacial. El componente encargado de esta tarea es el `nodoDeteccion`, cuya función principal es procesar el flujo de vídeo en tiempo real, detectar las características visuales de las puertas y transformar esos píxeles en coordenadas tridimensionales (X, Y, Z) dentro del mapa global.

Dado que el vehículo depende exclusivamente de una única cámara sin sensores de profundidad nativos, el proceso de proyección geométrica requirió resolver diversos obstáculos físicos y matemáticos. Para ilustrar de forma clara la solución implementada, esta sección se ha estructurado siguiendo el flujo lógico de los datos en el algoritmo de visión:

- **Detección bidimensional:** Se expone el proceso de identificación en el plano de la imagen, documentando la evolución desde las primeras pruebas teóricas mediante filtrado de color (HSV), hasta la transición hacia el uso de marcadores (ArUco) debido a la baja calidad de la cámara del dron.
- **Estimación de pose relativa:** Se detalla cómo, a partir de las coordenadas de los píxeles, se aplica la geometría proyectiva para obtener la distancia y el ángulo de la puerta de forma relativa a la lente de la cámara.
- **Álgebra espacial y cambios de sistema de referencia:** Se documentan las matrices de rotación y las ecuaciones de traslación necesarias para fusionar la posición del vehículo con la visión de la cámara, logrando anclar la puerta como un objeto estático en el mapa 3D.

5.3.1. Detección 2D en la imagen

El primer paso es localizar el objeto dentro de la fotografía. El `nodoDeteccion` recibe el flujo de vídeo en tiempo real a través del `topic /tello/imagen`. El objetivo de esta fase es procesar cada fotograma para aislar visualmente la puerta y extraer las coordenadas exactas en píxeles (x, y) de sus cuatro esquinas.

Fase 1: Segmentación por color en simulación

Durante las etapas iniciales de desarrollo en el entorno virtual (Webots), se planteó resolver esta detección identificando colores específicos situados en las esquinas de las puertas. Para lograr un algoritmo robusto, el primer paso indispensable fue convertir la imagen entrante del espacio de color RGB estándar al espacio **HSV** (*Hue*, *Saturation*, *Value*).

La justificación técnica de este cambio reside en cómo se representa la luz computacionalmente. En el formato RGB, el color y el brillo están matemáticamente acoplados; si una zona de la imagen recibe más luz, sus tres valores numéricos cambian por completo. Por el contrario, en el espacio HSV, el matiz del color puro (*Hue*) está separado de la saturación (*Saturation*) y de la intensidad luminosa (*Value*). Esto permitía definir rangos de color estables para cada esquina, ignorando en gran medida las variaciones de brillo.

Una vez transformada la imagen, el algoritmo aplicaba máscaras binarias (`cv2.inRange`) para filtrar y dejar en blanco solo los píxeles correspondientes a los colores de las esquinas. Sobre estas máscaras, se buscaban los contornos y se aplicaba un filtro de área mínima para descartar el ruido visual y quedarse únicamente con las detecciones válidas. Como se observa en el siguiente fragmento del código, el sistema extraía el centro de cada contorno para obtener el punto exacto de la esquina:

```

1  # Conversion a HSV
2  hsv = cv2.cvtColor(imagen, cv2.COLOR_BGR2HSV)
3
4  # Aplicacion de la mascara binaria para el color de la esquina
5  mask = cv2.inRange(hsv, color_min, color_max)
6
7  # Busqueda de contornos tras el filtrado
8  contornos, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.
    CHAIN_APPROX_SIMPLE)
9
10 for contorno in contornos:
11     if cv2.contourArea(contorno) > AREA_MINIMA:
12         # Calcular el centro del contorno
13         M = cv2.moments(contorno)
14         if M["m00"] != 0:
15             cx = int(M["m10"] / M["m00"])
16             cy = int(M["m01"] / M["m00"])
17             # Guardar (cx, cy) como coordenada de la esquina

```

Código 5.4: Segmentación por color y extracción de centros de contornos.

En el entorno ideal del simulador Webots, donde la iluminación ambiental es uniforme, esta técnica funcionaba a la perfección. El algoritmo lograba identificar los cuatro colores, calcular sus centros y extraer las coordenadas (x, y) de los vértices con una gran estabilidad (Figura 5.7).

Fase 2: El salto al entorno físico y el problema de la luz

Al trasladar este mismo algoritmo al dron en el laboratorio, la estrategia basada en la detección de color demostró deficiencias. La calidad óptica de la cámara del DJI Tello es limitada, y el entorno real introduce variables lumínicas: luces en el techo, sombras proyectadas por las paredes o los cambios de luz natural procedentes de las ventanas.

Esta dependencia de la iluminación provocaba que la máscara HSV se rompiera o quedara incompleta. Al no detectar el color de la puerta en su totalidad, la función que calculaba el rectángulo (`boundingRect`) devolvía una anchura y altura menores a las reales, lo que resultaba en unas coordenadas (x, y) totalmente erróneas para las cuatro esquinas.

Para intentar solucionar esto, era necesario recalibrar los umbrales de color constantemente. Si se optaba por aumentar los rangos del filtro para que tolerase mejor esas variaciones de luz, el sistema fallaba porque la cámara comenzaba a captar *falsos positivos*. El algoritmo confundía fácilmente los tonos, llegando a detectar el color marrón claro de las paredes del laboratorio como si fueran la puerta.

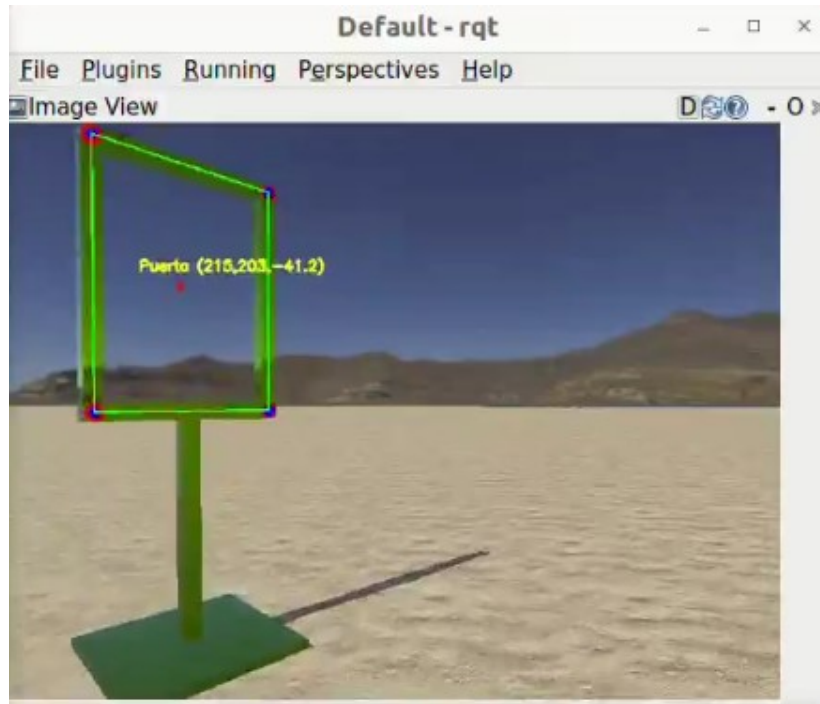


Figura 5.7: Detección de los vértices de la puerta en el simulador Webots.

Desde el punto de vista del diseño, depender de una calibración lumínica previa resulta poco eficiente para una navegación verdaderamente autónoma.

Aunque era posible desarrollar funciones extra que analizaran las proporciones geométricas de todas las marcas detectadas para descartar falsos positivos y quedarse solo con las puertas, el coste de procesamiento habría sido ineficiente para el sistema, además de haber necesitado invertir mucho tiempo para lograr una solución robusta. Esta limitación obligó a abandonar el color y adoptar una alternativa industrial que garantizase esos cuatro vértices bajo cualquier condición de iluminación.

Fase 3: La solución industrial (Marcadores ArUco)

Para resolver este problema lumínico de forma definitiva, se cambió el filtrado de color por marcadores ArUco (explicados en el Estado del Arte). Las propiedades físicas de estos marcadores hacen que sea más fiable su detección en escenarios con diferentes iluminaciones.

Para la implementación, se colocaron cuatro marcadores impresos, situando uno en cada esquina del marco de la puerta física. Al ser patrones en blanco y negro de máximo contraste, el algoritmo de detección de OpenCV es capaz de reconocerlos con una tasa de acierto prácticamente total ante distintas condiciones de iluminación ambiental, siempre que esta se mantenga dentro del rango operativo de la cámara.

Como añadido extra para la escalabilidad del sistema, se estableció una convención de identificadores muy sencilla para facilitar la futura navegación a través de múltiples puertas. Los cuatro marcadores ArUco que componen una misma puerta comparten siempre el mismo identificador numérico (ID). Esta regla de diseño permite al sistema identificar de forma unívoca qué obstáculo está cruzando en cada momento, posibilitando gestionar una ruta con múltiples puertas sin añadir ninguna complejidad

computacional extra a la visión artificial.

En este punto del desarrollo se tuvo que tomar una importante decisión de diseño. La librería ArUco dispone de una función nativa (`estimatePoseSingleMarkers`) que devuelve automáticamente la posición 3D y la orientación tridimensional de cada marcador detectado de forma independiente. Sin embargo, se optó por prescindir de ella.

El motivo de este descarte se basa en un problema clásico de fotogrametría conocido como la ambigüedad de la proyección en perspectiva. Si el dron detecta un marcador impreso pequeño (por ejemplo, de 15x15 cm) desde varios metros de distancia, el marcador abarca una cantidad muy reducida de píxeles en el sensor de la cámara. A esa escala, un error en la detección de un solo píxel se traduce en un margen de error enorme en el cálculo de su inclinación.

Para reducir esta inestabilidad, se debe incrementar la separación entre los puntos de referencia [19]. Siguiendo este principio de diseño, se tomó la idea de procesar la imagen únicamente para extraer la coordenada 2D (x, y) del centro de cada uno de los cuatro marcadores detectados.

Al unir matemáticamente estos cuatro vértices lejanos en lugar de procesarlos individualmente, el sistema construye en la práctica un “marcador virtual gigante” del tamaño de la puerta entera. Esta estrategia maximiza la precisión espacial del algoritmo.

Al concluir la ejecución de esta fase, el `nodoDeteccion` logra su objetivo inicial: obtener un array con las cuatro coordenadas exactas en píxeles de las esquinas de la puerta. Estos cuatro puntos bidimensionales constituyen la entrada de datos imprescindible para poder calcular la profundidad y orientación en la siguiente fase y poder obtener la pose completa relativa al dron de la puerta.

5.3.2. Estimación de pose relativa

Al finalizar la etapa de detección bidimensional, el sistema de detección dispone de un array con las cuatro coordenadas en píxeles (x, y) de las esquinas de la puerta. El siguiente reto matemático es la reconstrucción tridimensional: averiguar a qué distancia física se encuentra ese marco y con qué ángulo está rotado.

El objetivo final de todo el bloque de visión artificial es entregar al algoritmo de control de vuelo (`nodoNavegacion`) dos datos fundamentales: el punto central tridimensional de la puerta y su ángulo de rotación (*yaw*), para que el dron sepa hacia dónde volar y cómo alinearse.

Antes de poder ubicar esos datos en el mapa global, es obligatorio calcular primero la “pose relativa”. Es decir, resolver el problema geométrico de dónde está la puerta utilizando exclusivamente la lente de la cámara como punto de origen $(0, 0, 0)$.

Fase 1: Proyección simple y el problema de la perspectiva

Independientemente del método matemático que se utilice posteriormente, el primer paso obligatorio al recibir el array con los cuatro píxeles detectados es ordenarlos de forma determinista. El algoritmo necesita saber qué coordenada corresponde a cada esquina del marco. Para resolverlo, se implementó una función geométrica sencilla que evalúa la suma y la diferencia de las coordenadas (x, y) :

```

1 def ordenar_puntos(self, pts):
2     pts = np.array(pts, dtype="float32")
3     rect = np.zeros((4, 2), dtype="float32")
4
5     # La suma (x+y) menor es la esquina superior izquierda
6     # La suma mas grande es la esquina inferior derecha
7     s = pts.sum(axis=1)
8     rect[0] = pts[np.argmin(s)]
9     rect[2] = pts[np.argmax(s)]
10
11     # La diferencia (y-x) proporciona las otras dos esquinas
12     diff = np.diff(pts, axis=1).reshape(-1)
13     rect[1] = pts[np.argmin(diff)] # Superior derecha
14     rect[3] = pts[np.argmax(diff)] # Inferior izquierda
15
16     return rect

```

Código 5.5: Algoritmo de ordenación de vértices en el plano de la imagen.

Tras llamar a esta función, el sistema asigna de forma estructurada los vértices: **esq1** (superior izquierda), **esq2** (superior derecha), **esq3** (inferior derecha) y **esq4** (inferior izquierda).

Con los puntos correctamente etiquetados, el primer enfoque para extraer la pose fue utilizar óptica básica y trigonometría. Para calcular la profundidad (la distancia en el eje Z respecto a la cámara), el cálculo se apoyó en el principio de proyección de la lente, relacionando la altura real de la puerta (H_{real}), la distancia focal de la cámara (f) y la altura percibida en píxeles (h_{px}), según la ecuación clásica:

$$Distancia = \frac{H_{real} \cdot f}{h_{px}} \quad (5.4)$$

En código, para calcular la altura de la puerta detectada, se utilizaba la norma vectorial:

```

1 # Calculo de las alturas de los postes laterales en pixeles
2 alto_izq = np.linalg.norm(esquinas_ordenadas[0] -
3     esquinas_ordenadas[3])
4 alto_der = np.linalg.norm(esquinas_ordenadas[1] -
5     esquinas_ordenadas[2])
6 alto_px = (alto_izq + alto_der) / 2.0
7
8 if alto_px > 0:
9     distancia = (ALTO_REAL_PUERTA_M * FOCAL_PIXELS) / alto_px

```

Código 5.6: Cálculo inicial de la profundidad mediante proyección de lente.

Esta proyección directa ofrecía una estimación de profundidad aceptable. Sin embargo, el desafío crítico para la navegación consistía en calcular el ángulo de rotación (*yaw*) de la puerta.

Para calcular ese ángulo, se intentó utilizar la deformación por perspectiva (reglas de tres y proporciones). La lógica era la siguiente: si el dron mira la puerta totalmente de frente, los postes izquierdo y derecho miden exactamente lo mismo en píxeles (ángulo

$= 0^\circ$). Si la puerta está rotada, la perspectiva hace que el poste más cercano a la lente se vea más alto y el más lejano se vea más bajo. Calculando la diferencia y la proporción entre `alto_izq` y `alto_der`, generaba un ángulo estimado, deduciendo su signo (+ o -) en función de qué poste era el de mayor tamaño.

El problema de la robustez y la relatividad

Aunque esta heurística lograba devolver un ángulo de forma rápida, las pruebas de vuelo demostraron que matemáticamente era insuficiente por dos motivos:

1. **Sensibilidad a la distancia:** Cuando la puerta estaba lejos, la diferencia en píxeles entre ambos postes era minúscula. Un error de detección de apenas 2 píxeles en el ruido de la imagen se traducían en un falso salto de 5 o 10 grados en el cálculo del ángulo.

2. **Sistemas de referencia no acoplados:** El mayor problema era que este ángulo estimado era estrictamente relativo a la lente frontal de la cámara. Si el dron estaba rotado 30 grados respecto al mundo real, la cámara solo proporcionaba la inclinación relativa visual. Intentar sumar y restar este ángulo proporcional con los datos inerciales del dron (la IMU) para fijar el ángulo global de la puerta en el mapa resultaba inestable.

Se constató que tratar de reconstruir una orientación tridimensional completa usando proporciones simples no era un enfoque correcto. Dicho obstáculo obligó a investigar cómo se resolvía este reto en la robótica profesional, descubriendo que se trata de un problema clásico (PnP), lo que dio paso a la implementación de la solución definitiva.

Fase 2: Geometría proyectiva (Algoritmo solvePnP)

Para obtener una pose matemáticamente robusta, se recurrió a la resolución del problema de la **Perspectiva de N Puntos (PnP)**. El PnP es un estándar clásico en fotogrametría, visión artificial y robótica espacial. Su propósito es estimar la pose relativa de un objeto tridimensional a partir de su proyección en una única imagen bidimensional. Para resolver este problema, la librería OpenCV proporciona la función `cv2.solvePnP`.

Para que este algoritmo pueda operar, requiere que se le proporcionen cuatro conjuntos de datos que asocien el mundo real con la cámara del dron:

1. `puntos_imagen_2d`: El array con los cuatro píxeles (x, y) de las esquinas previamente extraídas y ordenadas en la Fase 1.
2. `puntos_objeto_3d`: Un modelo matemático ideal del objeto a calcular, representado por un array que contiene las coordenadas tridimensionales (X, Y, Z) .
3. `camera_matrix`: La Matriz de Intrínsecos (K) de la lente.
4. `dist_coeffs`: Los coeficientes de distorsión de la lente.

Una vez preparados estos parámetros, la ejecución en el código de Python es directa:

```

1 puntos_imagen_2d = np.array([esq1, esq2, esq3, esq4], dtype=np.
    float32)
2
3 # Ejecucion del algoritmo PnP
4 success, rvec, tvec = cv2.solvePnP(
5     self.puntos_objeto_3d,
```

```

6 puntos_imagen_2d ,
7 self.camera_matrix ,
8 self.dist_coeffs
9 )

```

Código 5.7: Resolución de la pose mediante el algoritmo solvePnP.

Si la función encuentra una solución geométrica válida (**success** = **True**), el algoritmo devuelve los dos vectores fundamentales de la pose espacial:

- **Vector de traslación (tvec):** Un vector de tres dimensiones (X, Y, Z) que indica la distancia física, en metros, desde el centro de la lente de la cámara hasta el origen de la puerta.
- **Vector de rotación (rvec):** Describe cómo está inclinada y girada la puerta respecto a la cámara. Por eficiencia computacional, OpenCV no devuelve este dato en los clásicos ángulos de Euler (*Roll, Pitch, Yaw*), sino en formato **Rodrigues**. Esta es una representación matemática compacta (vector eje-ángulo) muy útil para el cálculo interno, pero difícil de manipular algebraicamente.

Llegados a este punto, el sistema ya sabe exactamente a qué distancia y con qué inclinación se encuentra el obstáculo. Sin embargo, antes de poder enviar estos datos al módulo de navegación, surge el último obstáculo matemático: **tvec** y **rvec** son valores estrictamente *relativos a la lente de la cámara*. Para poder ubicar la puerta en el mapa global, es necesario extraer estos parámetros y aplicar una serie de transformaciones espaciales que se detallan en la siguiente sección.

5.3.3. Álgebra espacial y cambios de sistema de referencia

Tras la ejecución del algoritmo *solvePnP*, el sistema dispone de la pose relativa al dron de la puerta (representada en los vectores **tvec** y **rvec**). Sin embargo, es necesario transformarlas en coordenadas dentro del mapa global.

El mayor desafío de esta fase —y uno de los retos de diseño más complejos de todo el proyecto— es la diferencia de los sistemas de coordenadas. En robótica, los sensores (cámara), el vehículo (dron) y el entorno físico (mundo) no entienden el espacio tridimensional de la misma manera. Por lo tanto, es necesario uniformar los sistemas de referencia implicados.

Para que el álgebra espacial funcione correctamente, los ejes deben cumplir una ley geométrica: todos los sistemas deben ser **ortogonales** (formando ángulos de 90° entre sí) y **dextrógiros**⁴.

A continuación, se describen los sistemas de referencia que existen en este proyecto:

⁴Un sistema de coordenadas es dextrógiro cuando la disposición relativa de sus ejes cumple la “regla de la mano derecha”: si el dedo índice apunta en la dirección del eje X y el dedo medio en la del eje Y, el pulgar indicará obligatoriamente la dirección positiva del eje Z. Esta regla define la relación espacial entre los tres ejes, no su posición absoluta; por lo tanto, un sistema con el eje Z apuntando hacia abajo puede ser perfectamente dextrógiro siempre que X e Y mantengan esta configuración. Si no se respeta esta convención geométrica, el determinante de las matrices de rotación será -1 en lugar de +1, invalidando el álgebra espacial estándar.

- **1. Plano de la imagen 2D:** Es la imagen capturada por la cámara. Por convención, su origen $(0, 0)$ se ubica en la esquina superior izquierda de la imagen. El eje $+X$ avanza hacia la derecha (columnas) y el eje $+Y$ avanza hacia abajo (filas).
- **2. Sistema de la cámara 3D:** Es el espacio tridimensional nativo en el que trabaja *solvePnP*. Su origen $(0, 0, 0)$ está en el centro óptico de la lente. Hereda la disposición geométrica del plano 2D, por lo que el eje $+X$ apunta a la derecha y el $+Y$ hacia abajo. El eje restante, el $+Z$, representa la profundidad, proyectándose hacia adelante.
- **3. Sistema del cuerpo del dron:** Para la lógica de control y navegación, se adopta la convención aeronáutica estándar recomendada por ROS 2, conocida como FLU (*Forward, Left, Up*). En este marco de coordenadas, el origen es el centro de masas del vehículo. El eje $+X$ indica el avance frontal, el $+Y$ apunta hacia su izquierda y el $+Z$ indica la altitud, positivo hacia arriba.
- **4. Telemetría de la librería:** La librería de control del DJI Tello funciona con un sistema aeronáutico clásico tipo NED (*North, East, Down*), donde el eje Y es positivo hacia la derecha, y el Z hacia abajo. Esta discrepancia obligó a que al recibir la odometría del dron, se inviertan los signos de los ejes Y y Z, así como el ángulo de rotación (*Yaw*), garantizando que toda la capa de navegación trabaje exclusivamente bajo el estándar FLU.
- **5. Sistema global del mundo:** Constituye el entorno tridimensional estático donde se mapean las puertas y se calculan las trayectorias. Hereda la misma convención FLU que el dron ($+X$ adelante, $+Y$ izquierda, $+Z$ arriba). Su origen de coordenadas $(0, 0, 0)$ será el punto exacto donde el dron se inicializa y despegue.

A modo de resumen visual de todos los sistemas y operaciones definidos, la Figura 5.8 presenta el árbol completo de transformaciones cinemáticas.

Una vez definidos los sistemas de referencia, a continuación se explica la cadena secuencial de transformaciones algebraicas aplicada. El proceso consiste en extraer la pose relativa calculada mediante PnP, rotarla para alinearla con el cuerpo del UAV, componer matricialmente el instante del vuelo y trasladar el vector resultante para proyectar las coordenadas en el sistema global del mundo.

Paso 1: Expansión matricial (Fórmula Rodrigues)

El primer obstáculo matemático lo presenta el propio resultado de OpenCV. Por motivos de eficiencia computacional, *solvePnP* no devuelve la orientación espacial en forma de ángulos de Euler (*Roll, Pitch, Yaw*) ni como cuaterniones, sino como un vector de rotación compacto (*rvec*).

Este vector está basado en la representación **eje-ángulo**. La dirección del vector tridimensional indica el eje sobre el cual la puerta está rotando, y su módulo (la longitud del vector) indica la magnitud de dicha rotación en radianes. Si bien es computacionalmente ligero, es un formato con el que resulta imposible realizar multiplicaciones matriciales de cambio de base.

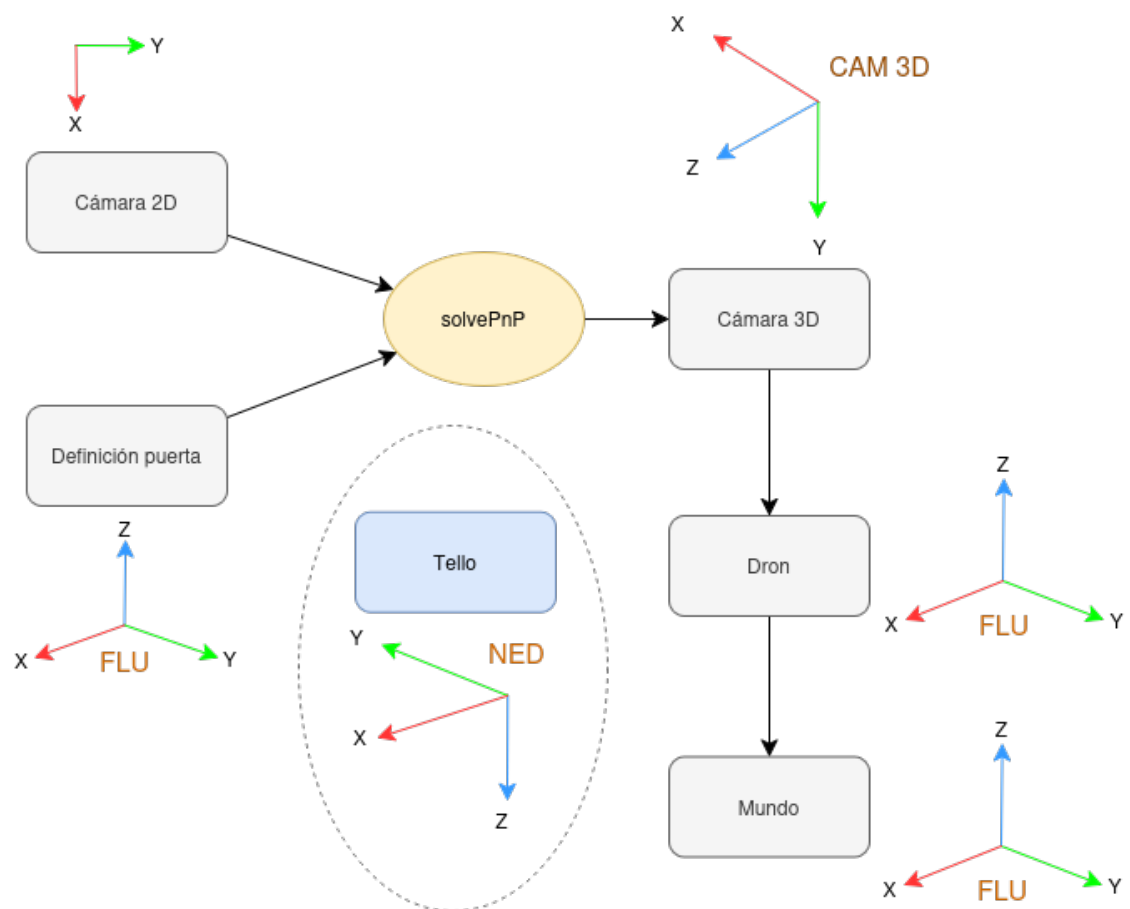


Figura 5.8: Diagrama que detalla los sistemas de coordenadas y las transformaciones espaciales.

Para poder operar algebraicamente, es obligatorio aplicar la fórmula de Olinde Rodrigues⁵, obtenida de la implementación nativa de la librería OpenCV. Esta operación convierte el vector eje-ángulo en una matriz de rotación ortogonal cuadrada de 3×3 , que describe perfectamente la actitud de la puerta relativa a la lente de la cámara:

```
1 # rvec es un vector (3,1). R_puerta_a_camara sera una matriz
   (3,3)
2 R_puerta_a_camara, _ = cv2.Rodrigues(rvec)
```

Código 5.8: Transformación del vector de rotación a matriz mediante la función de Rodrigues.

Con la matriz $R_{puerta_a_camara}$ calculada, el sistema está preparado para comenzar la corrección de perspectiva entre la cámara y el cuerpo del dron.

Paso 2: Alineación de ejes (Cámara a cuerpo)

Como se expuso en la introducción de los sistemas, la cámara entiende el mundo de forma distinta al estándar aeronáutico FLU.

Para alinear el Sistema 2 (Cámara) con el Sistema 3 (Dron), es necesario mapear cada eje geoméricamente:

- El avance frontal del dron ($+X_{dron}$) equivale a la profundidad visual de la cámara ($+Z_{cam}$).
- La izquierda del dron ($+Y_{dron}$) equivale al negativo de la derecha de la cámara ($-X_{cam}$).
- La altitud del dron ($+Z_{dron}$) equivale al negativo del eje inferior de la cámara ($-Y_{cam}$).

Esta relación directa conforma una matriz de rotación constante, que se define en el código de la siguiente manera:

$$R_{camara_a_dron} = \begin{pmatrix} 0 & 0 & 1 \\ -1 & 0 & 0 \\ 0 & -1 & 0 \end{pmatrix} \quad (5.5)$$

Paso 3: Odometría absoluta (Cuerpo a mundo)

El dron no es estático; durante el vuelo se inclina para avanzar o frenar. Para anclar la puerta en un mapa global (Sistema 4), es fundamental inyectar en el cálculo la actitud del dron en ese instante preciso.

Utilizando la telemetría recibida ($Roll, Pitch, Yaw$), el sistema convierte los ángulos a radianes y construye las tres matrices de rotación elementales sobre los ejes espaciales (R_x, R_y, R_z). El producto algebraico de estas tres matrices proporciona la orientación absoluta del dron respecto al mapa global:

$$R_{dron_a_mundo} = R_z(yaw) \cdot R_y(pitch) \cdot R_x(roll) \quad (5.6)$$

⁵Teorema cinemático que define un algoritmo eficiente para computar la matriz de rotación espacial a partir de un eje y un ángulo, operando algebraicamente con la matriz identidad y la matriz antisimétrica del vector.

Paso 4: Cadena cinemática y cálculo del ángulo de la puerta

El siguiente paso es aplicar el principio de la cadena cinemática. Multiplicando las tres matrices en el orden riguroso de transformaciones, se obtiene finalmente la matriz que describe cómo está rotada la puerta de forma absoluta en el mapa global:

$$R_{puerta_a_mundo} = R_{dron_a_mundo} \cdot R_{camara_a_dron} \cdot R_{puerta_a_camara} \quad (5.7)$$

Computacionalmente, esta operación matricial se ejecuta de forma optimizada utilizando el operador `@` de la librería NumPy.

Una vez calculada esta matriz global, surge la necesidad de extraer de ella un único dato para la navegación: el ángulo de rotación (*yaw*) de la puerta. Geométricamente, la tercera columna de una matriz de rotación representa el vector normal del objeto, es decir, un vector perpendicular a la puerta.

Dado que el dron debe atravesarla, se invirtió el signo de este vector normal para que apunte hacia “adentro” de la puerta. Proyectando este vector sobre el plano bidimensional del suelo (X, Y) y aplicando la función arcotangente (`atan2`), el algoritmo calcula el ángulo exacto al que debe encararse el dron para cruzar sin colisionar:

```

1 # Extraer la 3a columna: vector Z
2 vector_normal_mundo = R_puerta_a_mundo[:, 2]
3
4 # Invertir el vector
5 vector_normal_mundo = -vector_normal_mundo
6
7 # Calcular el angulo Yaw
8 angulo_yaw = math.degrees(math.atan2(vector_normal_mundo[1],
9                                       vector_normal_mundo[0]))
10
11 # Normalizacion del angulo
12 angulo_yaw = (angulo_yaw + 180) % 360 - 180

```

Código 5.9: Extracción del ángulo global de cruce.

Paso 5: Traslación espacial y transformación extrínseca

Resuelta la rotación, el desafío final consiste en fijar la posición espacial (X, Y, Z). El vector de traslación original `tvec` devuelto por OpenCV contiene la distancia métrica a la puerta, pero referenciada a la lente óptica de la cámara.

Para llevar ese vector al centro de masas del vehículo (*Body Frame*), se aplica la misma transposición de ejes que en el Paso 2:

$$P_{cuerpo_dron} = \begin{pmatrix} Z_{tvec} \\ -X_{tvec} \\ -Y_{tvec} \end{pmatrix} \quad (5.8)$$

Llegados a este punto de la cinemática, la robótica profesional exige aplicar una **transformación extrínseca** para compensar la distancia física entre la lente de la cámara y la IMU (el centro de masas del dron). Sin embargo, dado que el DJI Tello es un micro-UAV de dimensiones reducidas, la distancia entre su lente y su centro de gravedad es de apenas 1 o 2 centímetros.

En el contexto de este proyecto, esta magnitud es despreciable frente al margen de error del algoritmo de visión a larga distancia. Por ello, se asume que el centro óptico y el centro de masas coinciden, simplificando así el cálculo. Por tanto, para situar este punto en el mapa absoluto, se rota el vector local según la inclinación del dron y se le suma su posición actual:

$$P_{\text{mundo}} = P_{\text{posicion_actual}} + (R_{\text{dron_a_mundo}} \cdot P_{\text{cuerpo_dron}}) \quad (5.9)$$

Datos de salida del módulo de detección

Para integrar la pose de la puerta calculada en la arquitectura ROS 2, el nodo utiliza un publicador de tipo `Float32MultiArray`. La configuración del publicador en el constructor del nodo se define de la siguiente forma:

```
1 self.pub_punto_a = self.create_publisher(Float32MultiArray, '/  
tello/punto_y_angulo', 10)
```

Finalmente, una vez calculado el punto en el mundo y el ángulo de rotación, el sistema llama a la función de publicación. Esta transmite un array numérico con la estructura `[x, y, z, ángulo]`, proporcionando al módulo de navegación la información necesaria para ejecutar la trayectoria de vuelo:

```
1 def publicar_punto_a(self, x, y, z, angulo):  
2     msg = Float32MultiArray()  
3     msg.data = [x, y, z, angulo]  
4     self.pub_punto_a.publish(msg)  
5  
6 self.publicar_punto_a(punto_mundo[0], punto_mundo[1],  
    punto_mundo[2], angulo_yaw)
```

Código 5.10: Publicación de la pose de la puerta.

Con la publicación de estos datos en el *topic* `/tello/punto_y_angulo`, el módulo de detección da por finalizada su tarea, dejando los objetivos localizados para que el dron pueda navegar de forma autónoma, guiado por el módulo de navegación.

5.4. Módulo de navegación lógica

El `nodoNavegacion` es el componente encargado de tomar las decisiones de vuelo dentro de la arquitectura de software. Su función principal es suscribirse a la odometría del vehículo (`nodoIntermedioPose`) y a las coordenadas espaciales de las puertas publicadas por el módulo de detección (`nodoDeteccion`). Con estos datos, planifica la ruta y calcula los comandos de velocidad para los motores.

El diseño de este módulo se estructura en dos capas conceptuales. En primer lugar, una máquina de estados finitos que gestiona la cola de objetivos y la progresión de la misión. En segundo lugar, una capa de actuación cinemática encargada de calcular matemáticamente la velocidad necesaria para que el dron ejecute la ruta planificada por los estados.

5.4.1. Máquina de estados y gestión de objetivos

El comportamiento del vehículo se gestiona mediante una máquina de estados que evalúa en tiempo real las variables del entorno a través de la función `bucle_de_control()`, ejecutada a una frecuencia fija de 20 Hz. El ciclo de vuelo se divide en tres estados principales:

Estado 1: ESPERANDO_PUERTA (Búsqueda y planificación) El sistema se inicia en este estado. Mientras no existan objetivos fiables, el dron se mantiene en vuelo estacionario enviando comandos de velocidad nulos y utilizando el flujo óptico interno del DJI Tello para estabilizarse.

En paralelo, el nodo recopila las detecciones de las puertas. Teóricamente, cuanto más cerca de la puerta se encuentra el vehículo, más precisa es su localización, por lo tanto, se implementó un filtro de confianza.

Cuando se recibe la coordenada de una puerta, el sistema comprueba si está dentro del radio (`DISTANCIA_NUEVA_PUERTA`) de alguna puerta previamente detectada. Si es así, aplica una media ponderada para estabilizar la posición. El peso histórico aumenta en función del número de muestras (n) obtenidas previamente, según las siguientes ecuaciones:

$$W_{hist} = \frac{n \cdot (n + 1)}{2}$$

$$W_{nuevo} = n + 1$$

$$P_{filtrado} = \frac{P_{hist} \cdot W_{hist} + P_{nuevo} \cdot W_{nuevo}}{W_{hist} + W_{nuevo}}$$

Para iniciar una ruta, la máquina de estados requiere que la puerta haya sido detectada en más de tres fotogramas distintos (`n_muestras > 3`).

Una vez validados los objetivos, el sistema selecciona la puerta más cercana y, antes de iniciar el movimiento, **planifica la ruta de aproximación**. Para asegurar que el dron cruce el objetivo de forma perpendicular y no colisione con los laterales, el algoritmo no apunta directamente al centro de la puerta. Utilizando la coordenada central y el ángulo de rotación (*yaw*) de la puerta, calcula un vector normal perpendicular al marco y proyecta una recta virtual tridimensional compuesta por múltiples puntos.

Esta recta se define generando un Punto de Aproximación (`p_aprox`) situado antes de la puerta, y un Punto de Salida (`p_salida`) ubicado varios centímetros después de cruzar el umbral. De esta manera, se crea un vector que guía al dron de forma segura a través del obstáculo. Tras generar esta recta de puntos espaciales, el sistema pasa al estado `SEGUIR_TRAYECTORIA`.

Estado 2: SEGUIR_TRAYECTORIA (Ejecución) En este estado, el sistema abandona el vuelo estacionario y comienza a evaluar iterativamente el error entre la posición actual del dron y los puntos que componen la recta planificada, alimentando a la capa de control cinemático para enviar velocidad a los motores.

El dron avanza secuencialmente a través de los puntos de la recta generada. Al aproximarse al umbral físico de la puerta, la cámara pierde la visión de los marcadores laterales, pero el vehículo continúa navegando de forma autónoma gracias al mapa generado, hacia el `p_salida` proyectado en el estado anterior.

El sistema monitoriza constantemente la distancia respecto al último punto de la ruta planificada (`p_salida`). Cuando la distancia euclídea entre el dron y este punto de salida final es inferior a un umbral de aceptación, el algoritmo considera que la puerta ha sido cruzada con éxito en su totalidad. En ese instante, se llama a la función `finalizar_puerta()`, que saca el objetivo de la lista de `puertas_pendientes`, lo guarda en `puertas_visitadas`, reinicia los valores del control de vuelo y devuelve la máquina al estado `ESPERANDO_PUERTA`.

Estado 3: FIN_MISION Si el vehículo se encuentra en estado de espera, la lista de puertas pendientes está vacía y la variable `minimo_una_cruzada` es afirmativa, se pasa al estado terminal. En este punto, se detiene el envío de velocidades (`detener_dron()`), finalizando la navegación autónoma y aterrizando.

5.4.2. Navegación punto a punto y control proporcional (P)

Para la ejecución del movimiento, el diseño original se basó en una estrategia clásica de control de lazo cerrado⁶ : un Controlador Proporcional (P) guiado por navegación punto a punto (*Waypoints*)⁷.

La elección de esta arquitectura básica, considerada un estándar introductorio en la robótica móvil [22], se justificaba por las condiciones operativas ideales del proyecto. Al tratarse de un vuelo en interiores, se asumió un entorno ideal sin viento. Además, dado que el DJI Tello posee un sistema de flujo óptico interno capaz de mantener el vuelo estacionario de forma autónoma, a priori resultaba lógico deducir que un simple comando direccional sería suficiente para alcanzar la meta.

El principio de funcionamiento de un controlador proporcional establece que la acción de control —en este caso, el vector de velocidad tridimensional comandado a los motores— debe ser directamente proporcional a la magnitud del error espacial actual [21]. En cada iteración del bucle de control, el sistema calculaba el vector de error posicional ($\mathbf{e}(t)$) restando la coordenada actual del vehículo ($\mathbf{P}_{actual}$) a la coordenada de la meta ($\mathbf{P}_{objetivo}$):

$$\mathbf{e}(t) = \mathbf{P}_{objetivo} - \mathbf{P}_{actual}(t) \quad (5.10)$$

Posteriormente, este vector de error se multiplicaba por una constante escalar de ganancia (K_p), ajustada empíricamente, para generar el vector de velocidad final ($\mathbf{v}(t)$):

$$\mathbf{v}(t) = K_p \cdot \mathbf{e}(t) \quad (5.11)$$

Bajo este modelo matemático, el comportamiento cinemático del vehículo es fácilmente predecible. Cuando el dron se encuentra lejos de la puerta, la norma del vector de error es grande y, en consecuencia, la velocidad de avance demandada es alta. A medida que el vehículo recorta la distancia y se aproxima a la coordenada destino, el vector de error tiende a cero.

⁶Sistema que utiliza la lectura continua de los sensores (realimentación o *feedback*) para medir el estado actual del vehículo y compararlo con el estado deseado, ajustando dinámicamente la respuesta.

⁷Estrategia espacial donde el vehículo calcula un vector directo hacia una coordenada de destino aislada, preocupándose únicamente por llegar a ese punto y no por la trayectoria exacta seguida.

Validación en simulación y transición al entorno físico

Esta arquitectura inicial de control proporcional se probó en primer lugar en el simulador Webots. En este entorno virtual, el sistema logró guiar al dron hasta la meta de forma autónoma, validando la lógica del código. Sin embargo, ya en la simulación se observó una limitación práctica: no era posible fijar una velocidad de avance alta. Al intentarlo, la inercia provocaba que el vehículo sobrepasara las coordenadas del punto objetivo (*overshoot*), obligando al controlador a recalcular un error negativo para obligarle a retroceder. Aunque el dron terminaba llegando a la meta, el movimiento resultaba oscilatorio y poco eficiente. Este comportamiento se ilustrará en detalle en el Capítulo 6.

Al trasladar este mismo algoritmo al hardware físico del DJI Tello en el laboratorio, los problemas cinemáticos aumentaron. Para atravesar una puerta con confianza, y con el escaso margen de error disponible, el dron debe volar a velocidades bajas al acercarse a la puerta, para estar completamente alineado. El problema descubierto empíricamente fue que, a baja velocidad, la respuesta de los motores del Tello es imprecisa, desviándose de la dirección calculada por el vector de error. Incluso la velocidad que aplica el Tello puede ser nula, es decir, en el momento de alinear el dron, el `nodoNavegacion` genera velocidades bajas, para aceptar con precisión un punto y cruzar la puerta. El Tello al recibir velocidades de movimiento muy bajas, no se moverá, quedando así el dron estancado en una posición, lo que implicará que el controlador proporcional P empleado no reaccionará.

El alcance original de este Trabajo Fin de Grado concluía teóricamente en este punto, habiendo diseñado con éxito la detección tridimensional y un modelo de navegación básico. No obstante, al analizar estos fallos —que representan problemas clásicos documentados en la teoría de control automático [21, 22]—, se comprobó que la solución sería implementar un **Controlador PID**⁸ acoplado a una estrategia de **Seguimiento de Vector** (*Path Following*)⁹ suavizada mediante curvas de **Bézier**¹⁰.

Dado que la arquitectura de nodos distribuida que se diseñó en ROS 2 permitía modificar y ampliar la lógica matemática del `nodoNavegacion` sin afectar a los nodos de visión o telemetría, se tomó la decisión de programar e integrar estas mejoras en el código, a pesar de quedar fuera del alcance inicial planteado.

La explicación paso a paso de por qué el control proporcional puro fallaba en el Tello real, junto con el desarrollo matemático y la evaluación de esta nueva arquitectura PID que logró finalmente el vuelo autónomo en un entorno real, se detallan en el **Capítulo 6: Resultados**

⁸Algoritmo de control de lazo cerrado que calcula la corrección necesaria basándose en el error actual (Proporcional), la acumulación de errores pasados (Integral) y la predicción o tasa de cambio del error (Derivativo).

⁹Técnica de navegación en la que el vehículo no apunta directamente al destino, sino que minimiza continuamente su desviación lateral respecto a una línea o carril virtual predefinido en el espacio.

¹⁰Ecuaciones paramétricas utilizadas en geometría computacional para trazar trayectorias curvas y fluidas entre varios puntos de control, evitando giros mecánicos bruscos.

Capítulo 6

Resultados

Este capítulo evalúa el rendimiento y la viabilidad de la arquitectura de software diseñada. En primer lugar, se demuestra la fiabilidad de los datos de entrada, validando los valores de los sensores recibidos del dron, y las detecciones de las puertas en un entorno real. Posteriormente, se compara el comportamiento del algoritmo de navegación en el entorno ideal de simulación frente a su despliegue sobre el hardware real. Finalmente, se documentan los fallos descubiertos durante las pruebas físicas y se evalúa el rendimiento de las soluciones de control avanzado implementadas (PID y seguimiento de trayectorias) que lograron el vuelo autónomo del DJI Tello.

6.1. Validación de los módulos de telemetría y detección

Como se documentó durante el diseño de la arquitectura, el DJI Tello presenta dependencias críticas respecto a la iluminación del entorno y un error de inicialización en su odometría visual. Esta sección expone las pruebas realizadas para validar la estabilización del origen de coordenadas y las detecciones de las puertas, para posteriormente poder realizar pruebas de navegación.

6.1.1. Estabilización de odometría

Las pruebas revelaron que el sistema MVO del Tello no se inicializa correctamente, es decir, tras despegar la pose del dron debería ser $(0, 0, Z_{takeoff})$.

Como se observa en la Figura 6.1 (derecha), al leer la pose del dron directamente, proporcionada por `nodoTelloPose`, las variables arrojaban valores arbitrarios ($X = 13,00$, $Y = 5,09$, $Z = 5,62$). Además, se comprobó que la lectura de altitud (Z) procedente del MVO era muy imprecisa, con errores de unos 50 centímetros.

Para solucionar este fallo de hardware, se validó el comportamiento del `nodoIntermedioPose`. El nodo aplica un temporizador de seguridad (`ESPERA_TAKEOFF = 11.0 s`) que permite al dron despegar y alcanzar un vuelo estacionario. Una vez transcurrido este tiempo, el sistema toma una única instantánea de las coordenadas y las registra como *offset*.

Además, el programa lee el valor del altímetro proporcionado por `nodoTelloPose` (el primer valor de los 7 en el topic `/tello/pose`), para usarlo como altura real, debido a que la altura proporcionada por el MVO no es fiable. Como se aprecia en la Figura

```

alvaro@alvaro-KLVL-WXXW: ~
--
layout:
  dim: []
  data_offset: 0
data:
- 0.014562606811523438
- 0.02640676498413086
- 1.0179924964904785
- -1.2603293657302856
- 0.3165149986743927
- 0.09108448028564453
--
layout:
  dim: []
  data_offset: 0
data:
- 0.014562606811523438
- 0.02640676498413086
- 1.0179924964904785
- -1.325636625289917
- 0.36248141527175903
- 0.1297750473022461
--

alvaro@alvaro-KLVL-WXXW: ~
dim: []
data_offset: 0
data:
- 1.0
- 13.000543594360352
- 5.095044136047363
- 5.624749183654785
- 0.7711614370346069
- -0.05402234196662903
- 9.910039901733398
--
layout:
  dim: []
  data_offset: 0
data:
- 1.0
- 13.000543594360352
- 5.095044136047363
- 5.624749183654785
- 0.7058542370796204
- -0.008055920712649822
- 9.871349334716797
--

```

Figura 6.1: Captura de la terminal donde se muestra a la derecha los valores extraídos directamente de la lectura del Tello y a la izquierda, la lectura correcta de la pose del dron, publicada por `nodoIntermedioPose`.

6.1 (izquierda), tras esta compensación, el `topic /tello/pose_corregida` comienza a publicar una pose limpia donde el dron se encuentra estabilizado en $X = 0,0, Y = 0,0$ y su altura real.

6.1.2. Condicionantes del entorno físico

Tras validar la estabilización inicial, se procedió a configurar el entorno de pruebas físico en el laboratorio. Como se analizó en la Sección 3.3.1, la Odometría Visual del dron es susceptible a la pérdida de referencias visuales.

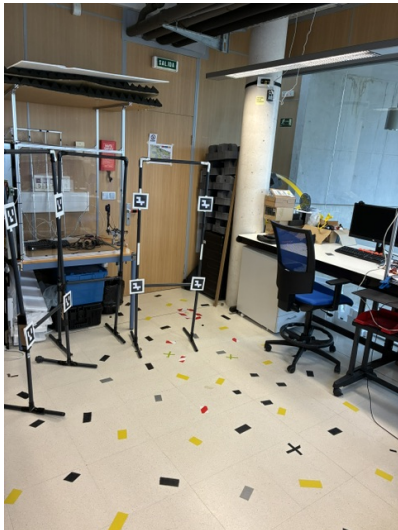


Figura 6.2: Entorno del laboratorio preparado para el vuelo.



Figura 6.3: Ejemplo de un pasillo carente de textura e iluminación.

Las pruebas de vuelo evidenciaron que, para que el vehículo pudiera mantener la estabilidad era indispensable una buena iluminación. Al intentar operar únicamente con la luz ambiental o en zonas pobremente iluminadas, como el pasillo documentado en la Figura 6.3, el flujo óptico inferior fallaba. Este fallo no impedía el despegue, pero provocaba que el dron derivara de forma agresiva hacia una única dirección al ser incapaz de calcular su velocidad relativa.

Asimismo, las superficies lisas —como el suelo del laboratorio— presentaban dificultades de mapeo para el sensor óptico inferior. Por lo tanto, se decidió colocar de manera preventiva cinta adhesiva de colores en el suelo de la zona de pruebas (Figura 6.2) para generar un suelo con texturas.

6.1.3. Precisión de la detección espacial

Para evaluar el rendimiento del `nodoDeteccion`, se estableció una metodología de pruebas apoyada en tres herramientas fundamentales del ecosistema ROS 2: `rqt` para la visualización gráfica del flujo de vídeo y los `topics` de imagen, `RViz2` para la representación tridimensional del sistema de coordenadas, y los archivos `rosvbag`.

La integración de `rosvbags` resultó de vital importancia técnica durante esta fase. Esta herramienta nativa permite grabar el tráfico continuo de la red para, posteriormente, poder reproducirlo en bucle. Esto posibilitó depurar y ajustar los parámetros matemáticos de la visión artificial en diferido, sin necesidad de volar el dron en cada iteración de código.

Evolución de las referencias visuales

Inicialmente, se intentó ubicar las esquinas de la puerta utilizando marcadores de color (esferas naranjas). Como se aprecia en la Figura 6.4, esta aproximación teórica demostró ser frágil en el mundo real: la iluminación ambiental y los reflejos impedían que el filtro HSV aislara los cuatro contornos simultáneamente con consistencia, invalidando la geometría. En ese ejemplo, a una distancia muy cercana es incapaz de detectar las 4 esferas a la vez, debido a un reflejo de luz en la esfera de abajo a la derecha.

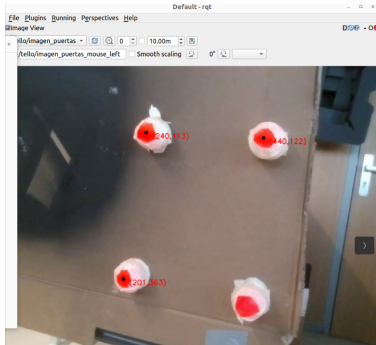


Figura 6.4: Fallo del filtrado de color por reflejos en la escena real.

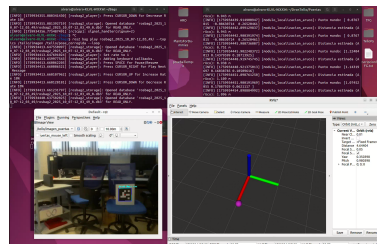


Figura 6.5: Anclaje exitoso de un único marcador ArUco en RViz2.

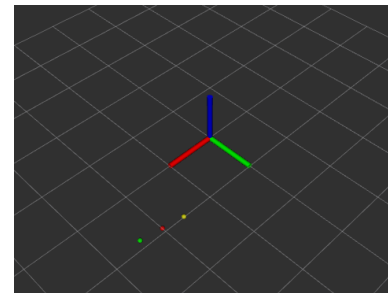


Figura 6.6: Proyección de los tres puntos de navegación.

Tras este resultado, se decidió utilizar marcadores ArUco. El proceso de validación comenzó fijando un único marcador (Figura 6.5) para corroborar en `RViz2` que el punto 3D se mantenía estático en el espacio absoluto mientras el dron orbitaba a su alrededor, demostrando que la cadena de transformaciones era correcta. Posteriormente, esta lógica se escaló a los cuatro marcadores de la puerta, lo que permitió no solo calcular el centro del marco, sino también proyectar geométricamente el punto de aproximación previo y el punto de salida posterior (Figura 6.6).

Pruebas de medición de distancia y rotación

Para cuantificar el margen de error métrico, se diseñó un experimento estático desplegando una cinta métrica en el suelo del laboratorio alineada con la puerta (Figura 6.7). El dron se hizo despegar en vuelo estacionario a diferentes distancias conocidas, monitorizando la exactitud de los cálculos. En todo momento las puertas miden 0,7 metros de alto y 0,5 metros de ancho (distancias formadas por los arucos).

En el código del sistema, la salida matemática se supervisa a través de la consola mediante la siguiente función:

```

1 def log_datos(self):
2     if self.distancia_estimada is not None:
3         self.get_logger().info("---DATOS---")
4         self.get_logger().info(f"Distancia a la puerta: {self.
5             distancia_estimada:.2f} m")
6         self.get_logger().info(f"Verificacion: La distancia
7             calculada es {self.distancia_calculada:.3f} m")
8         self.get_logger().info(f"Coordenada puerta (cuerpo): X
9             = {self.coordenada_X:.2f}, Y = {self.coordenada_Y:.2f},
10             Z = {self.coordenada_Z:.2f} m")
11     if self.punto_mundo is not None:
12         self.get_logger().info(f"Coordenadas del punto en
13             mundo: ({self.punto_mundo[0]:.2f}, {self.
14             punto_mundo[1]:.2f}, {self.punto_mundo[2]:.2f})")
15     self.get_logger().info("-----")

```

Código 6.1: Función de salida de datos en el nodo de detección para debug

En paralelo, a través del *topic* visual `/tello/imagen_puertas`, el nodo renderiza un gráfico sobre el flujo de vídeo donde enmarca el obstáculo y escribe el resultado de la coordenada global (X, Y) y la rotación (Figura 6.8).



Figura 6.7: Validación de la distancia a la puerta calculada utilizando referencias métricas.

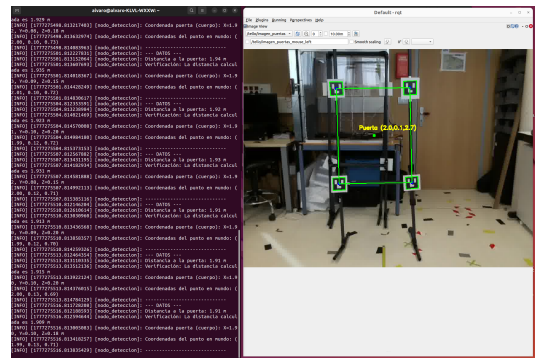


Figura 6.8: Lectura de las mediciones a través de la terminal y `rqt`.

Los resultados obtenidos fueron muy satisfactorios como se observa en la tabla

6.1. A distancias medias de 2,0 metros, el algoritmo calculó la posición con un error prácticamente nulo (2,0 metros exactos). Al incrementar la separación a 3,1 metros reales, la estimación arrojó un valor de 3,2 metros. La distancia máxima operativa se estableció en 4,6 metros reales, cota a partir de la cual el sistema calculó 4,8 metros.

Tabla 6.1: Error en la estimación de distancia a la puerta.

Distancia real (m)	Distancia estimada (m)	Error absoluto (m)
1,00	1,00	0,00
1,50	1,50	0,00
2,00	2,00	0,00
2,50	2,53	+0,03
3,10	3,20	+0,10
3,80	3,94	+0,14
4,60	4,80	+0,20

Este margen de error de 20 centímetros en el extremo operativo resulta completamente asumible considerando que los marcadores ArUco impresos tienen unas dimensiones de 9,3 cm por lado. Sin embargo, por encima de los 3 metros, el ligero movimiento inercial del dron, unido a la compresión del vídeo, dificulta la correcta extracción de las cuatro esquinas de forma simultánea. No obstante, este error en lejanía no afecta a la navegación: gracias a la arquitectura iterativa del nodo, a medida que el dron avanza físicamente, se retroalimenta con fotogramas más cercanos y certeros.

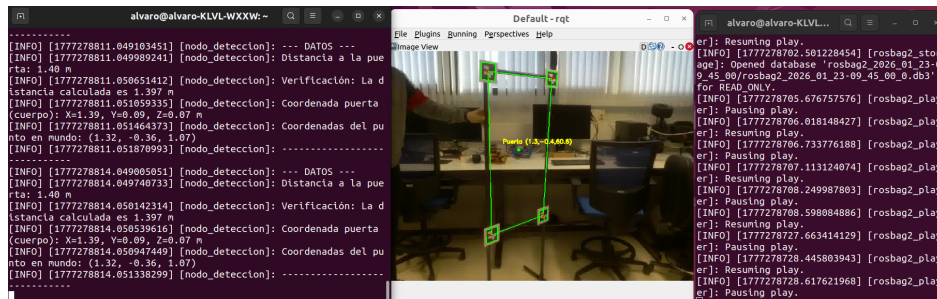


Figura 6.9: Validación de la detección del ángulo de Yaw.

Finalmente, se replicó la metodología para evaluar la fiabilidad de la orientación o Yaw de la puerta (Figura 6.9). Tras rotar la puerta con el dron fijo en una posición, y posteriormente mantener fija la puerta y rotar el dron, los resultados evidenciaron que el algoritmo genera un error de entre 5 y 10 grados como máximo (Tabla 6.2). Esta desviación direccional es lo suficientemente pequeña como para asegurar que la generación del vector normal perpendicular permita al dron alinear su entrada de forma segura.

6.2. Resultados en simulación

Antes de navegar de forma autónoma en el mundo físico, la lógica matemática del nodoNavegacion se validó en el simulador Webots. Para esta primera aproximación, se

Tabla 6.2: Error en la estimación del ángulo de rotación.

Ángulo real (°)	Ángulo estimado (°)	Error absoluto (°)
0,0	0,5	0,5
15,0	16,2	1,2
30,0	33,1	3,1
45,0	50,6	5,6
-20,0	-21,8	1,8
-40,0	-45,2	5,2
-60,0	-68,9	8,9

implementó una estrategia de navegación punto a punto (*Waypoints*) y un controlador Proporcional (P).

Como se ha explicado anteriormente, para garantizar que el vehículo cruzara las puertas de forma perpendicular, el programa calcula geoméricamente un vector normal ($\vec{n}$) a partir del ángulo de rotación (*Yaw*) del obstáculo. Sobre esta recta virtual, se proyectaban dos coordenadas tridimensionales: un punto de aproximación (P_{aprox}) a 0,6 metros antes del marco, y un punto de salida (P_{salida}) a 0,6 metros después del mismo. Estas coordenadas serían los *Waypoints* que usaría el `nodoNavegacion` para navegar uno a uno.

El controlador calculaba el error posicional en cada iteración y ordenaba velocidades proporcionales hacia estos puntos. Para dar un *waypoint* por alcanzado y transitar al siguiente, el sistema exigía que el dron entrara en una esfera de tolerancia con un radio máximo de 0,25 metros respecto a la coordenada del objetivo.

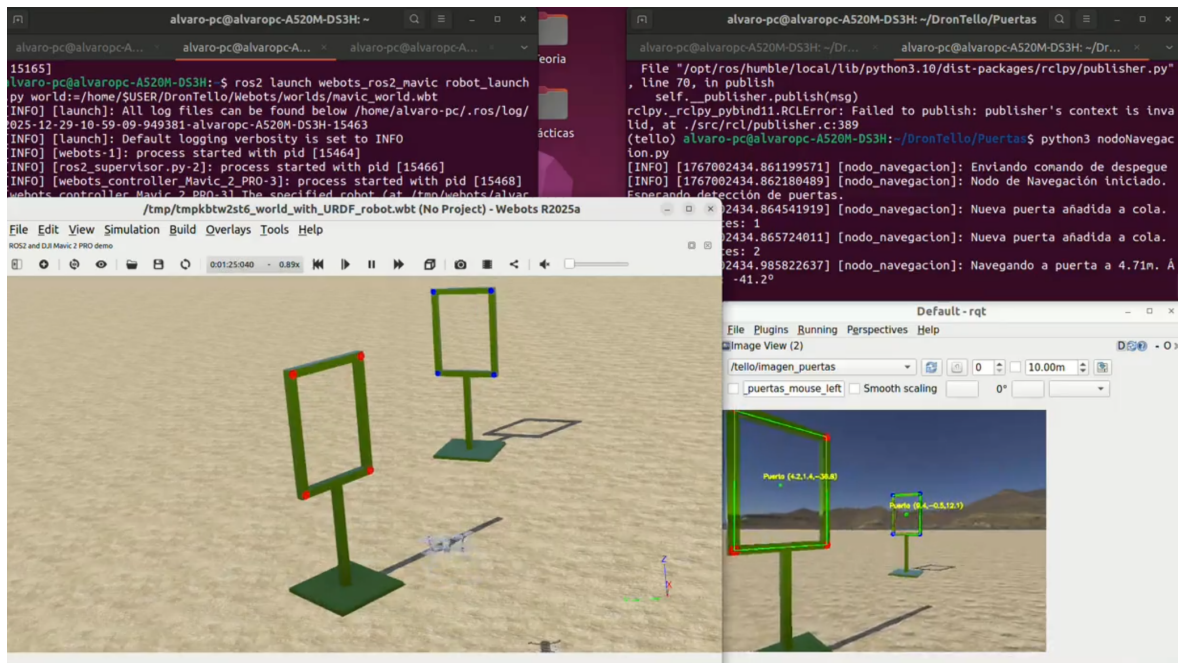


Figura 6.10: Validación de la navegación en simulación. Se observa el entorno 3D en Webots junto con la lectura de la máquina de estados en la terminal y la detección de las puertas en `rqt`.

Como se observa en la Figura 6.10, en el entorno virtual esta arquitectura básica

resultó funcional. El controlador lograba reducir la velocidad al acercarse a la meta, introduciendo el dron en el radio de 0,25 metros con la precisión requerida. A una velocidad máxima de 0,3 m/s, y con las puertas colocadas a 4 metros de separación, el vuelo fue correcto. La única anomalía detectada en simulación estaba en la gestión de la inercia: si se configuraba una velocidad de avance elevada superior, el vehículo sobrepasaba las coordenadas del punto objetivo (*overshoot*), aunque el controlador proporcional calculaba el error negativo y forzaba al dron a retroceder para corregir su posición.

6.3. Limitaciones físicas del DJI Tello

Al probar este código en la realidad, el comportamiento del DJI Tello divergió del modelo simulado. Aunque el módulo de visión funcionaba correctamente, el enfoque de navegación mediante *waypoints* y control proporcional fracasó.

Durante las pruebas, se detectaron problemas insalvables para el diseño de control inicial:

1. **Zona muerta de actuación:** La API del DJI Tello acepta comandos de velocidad normalizados en un rango de -100 a 100. En teoría, el control proporcional exige que la velocidad disminuya a medida que el error se acerca a cero. Sin embargo, el controlador de vuelo interno ignora órdenes que caigan por debajo de un umbral aproximado de ± 6 a ± 10 puntos. Es importante destacar que no se establece una constante numérica exacta en esta memoria para dicho umbral debido a su dependencia a varios factores, como la iluminación, la textura del suelo o el desgaste de los motores de cada modelo. Al no superar este umbral, el dron podía quedarse estancado antes de llegar al *waypoint*, sin alcanzarlo nunca, debido a la naturaleza del control proporcional diseñado.
2. **Deriva y necesidad de seguimiento de vector:** El vehículo demostró ser incapaz de mantener el vector de dirección comandado, sobre todo a baja velocidad. Se observó que cada unidad de DJI Tello en el laboratorio presentaba desviaciones mecánicas distintas (por ejemplo, una unidad tendía a derivar hacia la izquierda, mientras que otra lo hacía hacia la derecha). Esta variabilidad del hardware hace imposible programar compensaciones estáticas, justificando la necesidad de implementar un **Seguimiento de Trayectoria** (*Path Following*) que mida de forma genérica el error lateral en tiempo real, independientemente del dron utilizado.
3. **Inestabilidad cinemática a baja velocidad:** A velocidades altas, el dron mantenía trayectorias más estables, pero resultaba peligroso para atravesar una puerta estrecha en interiores. Al obligar al vehículo a volar a las velocidades bajas necesarias para asegurar el cruce, la falta de inercia provocaba que el sistema de flujo óptico generara vibraciones, saltos y agravara la deriva lateral mencionada en el punto anterior.
4. **Oscilación provocada por compensación proporcional:** El intento lógico de mitigar el estancamiento mencionado en el primer punto, consistió en aumentar la ganancia proporcional (K_p) para forzar al sistema a mandar velocidades

altas incluso cerca del umbral. Esta modificación resultó inviable: al carecer de un amortiguador predictivo, el exceso de velocidad provocaba un *overshoot* incontrolable, obligando a abortar las pruebas por colisión.¹

5. **Latencia Wi-Fi:** Toda la arquitectura de control y visión *offboard* depende del enlace de red Wi-Fi a 2.4 GHz generado por el propio vehículo. Durante los ensayos, se constató que la antena integrada posee una ganancia muy limitada y es altamente vulnerable a las interferencias. En entornos saturados, la comunicación sufre cortes y pérdida de paquetes UDP.

Aunque se consiguió el vuelo autónomo en el simulador, estas limitaciones evidenciaron que la arquitectura planteada en el alcance era insuficiente para una plataforma comercial de este tipo, sin lograr conseguir el vuelo autónomo en un entorno real.

Para cuantificar el rendimiento del control proporcional, se realizaron pruebas bajo condiciones favorables, colocando las puertas con la respuesta prevista del vehículo. La tasa de éxito fue de aproximadamente 3/10, siendo el estancamiento y los cortes de señal Wi-Fi las causas de fallo más recurrentes.

6.4. Evolución del control: PID y seguimiento de trayectorias

El alcance original planteado para este Trabajo Fin de Grado terminaba con la implementación del controlador Proporcional (P) mediante *waypoints*. En un principio, la corrección de estas deficiencias físicas se había dejado simplemente como una propuesta para trabajo a futuro.

Sin embargo, tras analizar el comportamiento del dron en el laboratorio, se optó por ampliar el alcance inicial e implementar soluciones de control clásicos, ampliamente documentadas en la robótica móvil. La gran ventaja de haber diseñado el sistema con una arquitectura modular es que permitía sustituir por completo la forma de navegar, es decir, el `nodoNavegacion` sin romper ni alterar los módulos de visión y telemetría, que ya estaban validados.

El objetivo de esta fase de desarrollo adicional era comprobar de forma práctica si la aplicación de estas técnicas era suficiente para solventar las carencias del hardware y consolidar, por fin, el vuelo autónomo.

6.4.1. Controlador PID (proporcional, integral y derivativo)

Para solucionar el problema de la zona muerta de actuación o la inercia, el control proporcional básico se sustituyó por un controlador PID completo para los tres ejes espaciales. Este controlador funciona como un bucle continuo a 20 Hz.

El ajuste de las ganancias se hizo realizando ensayos de vuelo en el laboratorio y analizando los datos hasta dar con el equilibrio dinámico del vehículo (cada dron puede requerir unas constantes diferentes). Tras las pruebas, se fijaron para el vehículo a usar los parámetros en $K_P = 3.2$, $K_I = 2.2$ y $K_D = 0.6$.

¹Se conoce formalmente como respuesta subamortiguada. Aparece cuando una ganancia K_p elevada sin control derivativo no logra disipar la energía cinética, provocando oscilaciones y sobreimpulsos (*hunting*) alrededor del punto de consigna.

Cada componente controla una acción diferente:

- **Acción proporcional (P):** Es el movimiento principal. Calcula un vector de movimiento base en función de la distancia del error ($P_vector = vector_total * KP$).
- **Acción integral (I):** Es la pieza fundamental que resolvió el fallo de estancamiento a baja velocidad. Mediante la acumulación temporal ($self.error_integral += vector_correccion * dt$), este término actúa como una fuerza que va creciendo progresivamente. Si el dron no consigue alcanzar un objetivo, la acción integral inyecta cada vez más fuerza a los motores ($I_vector = KI * error_integral$) hasta lograr vencer la resistencia mecánica. Para evitar que esta fuerza creciera de forma descontrolada, se limitó el sumatorio a un máximo de $MAX_INTEGRAL = 15.0$.
- **Acción derivativa (D):** Funciona como un freno predictivo. Al calcular la diferencia entre el error actual y el error previo ($derivada = (vector_total - error_previo) / dt$), el sistema reacciona a la velocidad a la que se acerca a la meta, reduciendo el impulso para mitigar el rebote y evitar que el dron se pase de largo.

6.4.2. Seguimiento de trayectoria

Aunque el PID proporcionaba la fuerza necesaria para mover el dron, en la práctica el DJI Tello era incapaz de seguir de forma exacta un vector de dirección puro. Si se le ordenaba avanzar en línea recta, inevitablemente acababa desviándose hacia los lados debido a las inercias o el comportamiento irregular de sus propios motores. Para evitar que el dron se perdiera, era necesario controlarlo en todo momento, corrigiendo su posición instante a instante. Por este motivo, se implementó una estrategia de Seguimiento de Trayectoria (*Path Following*).

La idea es intuitiva: en lugar de decirle al dron “ve directamente a este punto”, se le construye un “carril virtual” que debe seguir paso a paso. Estas trayectorias se forman de la siguiente manera:

- **El cruce de la puerta:** Para atravesar la puerta de forma segura y perpendicular, el algoritmo genera, de la misma forma que antes, una línea recta que va desde un Punto de Aproximación (P_{aprox} , a 0.6 metros antes de la puerta) hasta un Punto de Salida (P_{salida} , a 0.6 metros pasada la puerta).
- **Las curvas de enlace:** Para suavizar la trayectoria a seguir hasta el P_{aprox} , se genera una **Curva de Bézier**².
- **Navegación continua:** Una vez cruzada la primera puerta, el sistema repite el proceso: genera una nueva curva de Bézier desde el P_{salida} de la primera puerta hasta el P_{aprox} de la segunda, enlazando los objetivos de forma ininterrumpida y encadenando el vuelo.

²Una Curva de Bézier es una ecuación matemática utilizada habitualmente en diseño y geometría computacional para trazar trayectorias curvas y fluidas entre varios puntos, evitando giros mecánicos agresivos.

Para ilustrar el resultado final, la Figura 6.11 muestra la representación gráfica del sistema en pleno funcionamiento. En ella se evidencia cómo la información detectada en el plano 2D de la cámara se traduce en un entorno tridimensional donde el vehículo cuenta con un carril de aproximación para realizar su vuelo autónomo con éxito.

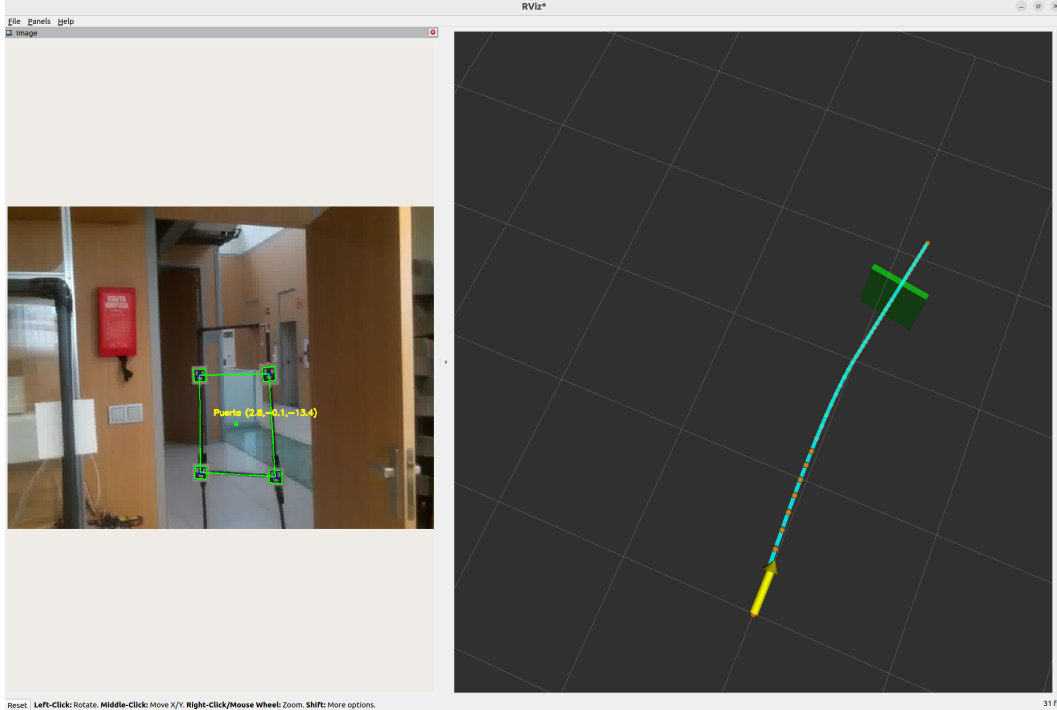


Figura 6.11: Representación en RViz2 del seguimiento de trayectoria. Se observa la puerta detectada (verde), posición del dron (amarillo) y la trayectoria calculada a seguir (azul).

El error lateral

La clave para que el dron no se salga de este carril radica en el cálculo continuo del *Cross-Track Error* o Error Lateral. En cada iteración, el algoritmo busca cuál es el punto exacto de la línea imaginaria que queda más cerca del dron. La distancia perpendicular entre la posición real del vehículo y ese punto ideal del carril es el error lateral.

Matemáticamente, si definimos la trayectoria como una recta que pasa por los puntos A y B , y la posición actual del dron como el punto P , el error lateral (e_{lat}) se calcula mediante la proyección ortogonal:

$$e_{lat} = \frac{\|(P - A) \times (B - A)\|}{\|B - A\|} \quad (6.1)$$

Donde el numerador representa el área del paralelogramo formado por los vectores y el denominador normaliza el resultado para obtener la distancia perpendicular mínima al carril.

En la programación del sistema, se le dio una prioridad absoluta a corregir este error lateral, ya que es precisamente en ese eje donde el Tello más deriva. El comportamiento diseñado funciona como un mecanismo de autoprotección: si el dron sufre una

desviación y se aleja de la línea, el sistema penaliza dinámicamente su velocidad de avance frontal y concentra la fuerza del controlador PID en empujarlo lateralmente de vuelta al carril. Solo cuando el dron vuelve a estar centrado sobre la ruta, se le permite recuperar su velocidad normal de avance hacia el objetivo. Esta corrección asimétrica y constante fue la pieza definitiva que permitió aislar los defectos físicos del hardware y lograr una navegación fluida.

6.4.3. Análisis comparativo de la respuesta cinemática

Para validar la robustez de la arquitectura final, se realizó un estudio comparativo entre el control proporcional inicial y el controlador PID diseñado. Este análisis se divide en tres escenarios que representan la evolución del sistema desde el entorno virtual hasta las condiciones más críticas del laboratorio.

Escenario A: Validación en entorno de simulación

Como metodología de trabajo, se procedió a validar cualquier cambio en la lógica de control siempre en primera instancia sobre el simulador. En la Figura 6.12 se observa esta comparativa.

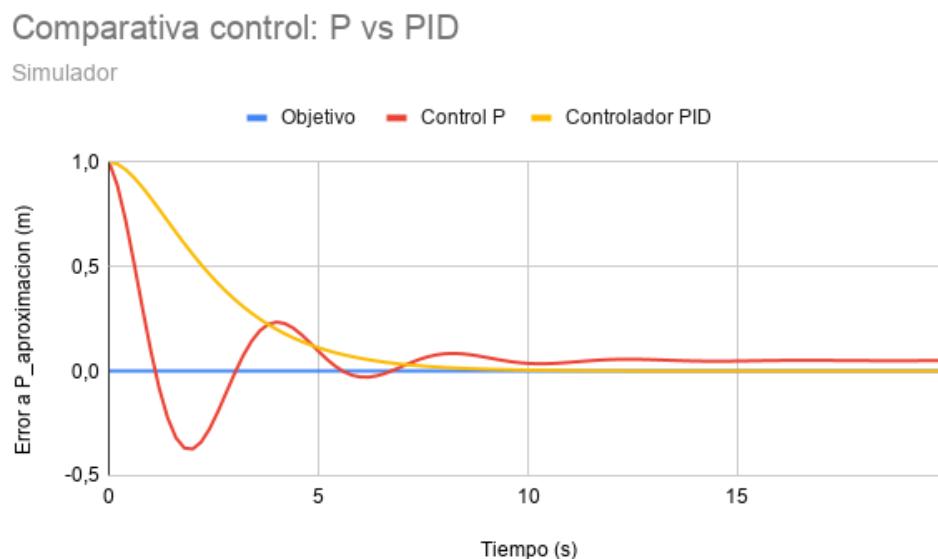


Figura 6.12: Escenario A: vuelo simulado.

Para lograr que el simulador evidenciase las carencias del control proporcional (línea roja) y generase la oscilación mostrada, fue necesario forzar al vehículo a navegar a una velocidad mayor, concretamente 0,5 m/s. En estas condiciones, se observa cómo el control P entra en un comportamiento subamortiguado, cruzando el punto objetivo y rebotando por inercia. Por el contrario, el controlador PID (línea amarilla) logra frenar a tiempo y reducir el error a cero de forma estable.

Escenarios B y C: Despliegue en el mundo real

Una vez validada la teoría, se trasladaron los ensayos al hardware real.

Comparativa control: P vs PID

Escenario real 1

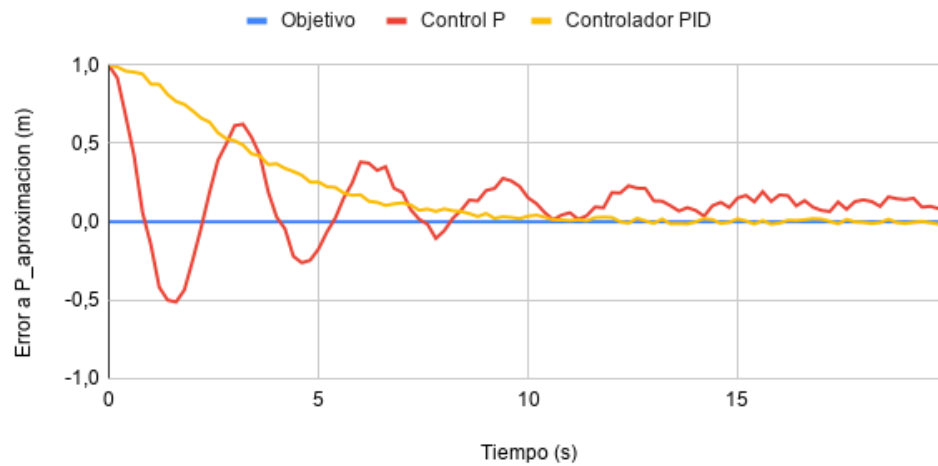


Figura 6.13: Escenario B: vuelo real

Comparativa control: P vs PID

Escenario real 2

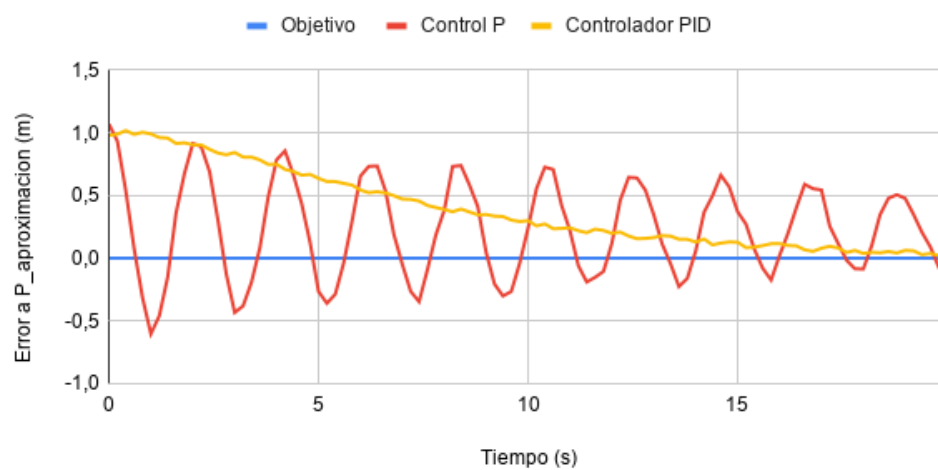


Figura 6.14: Escenario C: vuelo real.

En el Escenario B (Figura 6.13), se analizó el comportamiento del PID bajo las mismas condiciones que provocan el estancamiento del control proporcional. Como se observa en la gráfica, el control P presenta una oscilación inicial y se queda cerca del objetivo sin llegar a alcanzarlo. Por el contrario, el controlador PID bajo este mismo escenario elimina el estancamiento y alcanza el punto de consigna con éxito.

Finalmente, en el Escenario C (Figura 6.14) se evaluaron los parámetros para intentar solucionar dicho estancamiento. Al incrementar la ganancia K_p en el control proporcional, se observa que el sistema oscila con mayor agresividad y sigue sin alcanzar el objetivo. Asimismo, se probó a reducir el valor de la constante integral K_i en el PID, demostrando que el vehículo tarda significativamente más tiempo en corregir el error y alcanzar el objetivo.

6.5. Evaluación final del vuelo físico autónomo

En las pruebas de validación definitivas realizadas en el laboratorio, el DJI Tello logró ejecutar de principio a fin el ciclo completo diseñado: despegó de forma estable, inicializó su origen de coordenadas global, detectó de forma autónoma los marcadores ArUco, calculó la posición espacial y la orientación de la puerta, proyectó una ruta y atravesó la puerta de forma totalmente autónoma. Las pruebas se repitieron con dos puertas, logrando cruzarlas de forma satisfactoria, lo que revela la escalabilidad del sistema a cualquier número de puertas, con el requisito de estar formadas con arucos de diferentes IDs.

La arquitectura final alcanzó una tasa de éxito de entre el 80 % y el 90 %, permitiendo esta vez sí, una disposición arbitraria de los obstáculos. Con la implementación del PID se eliminó por completo el problema del estancamiento mecánico; de este modo, los fallos mas habituales se limitaron a pérdidas de señal Wi-Fi, un factor de hardware externo al control de la arquitectura desarrollada.

Capítulo 7

Conclusiones y trabajo futuro

7.1. Conclusiones del proyecto

El desarrollo de este Trabajo Fin de Grado ha concluido con éxito, cumpliendo no solo con los objetivos iniciales planteados, sino también con el nuevo alcance que exigió la puesta en marcha del sistema. Se ha conseguido reducir la brecha que siempre existe entre un entorno ideal, como es el simulador, y el mundo físico. Como resultado final, se ha logrado el hito principal del proyecto: un vuelo completamente autónomo en el que el dron es capaz de detectar y navegar a través de puertas.

El sistema se estructuró en diferentes programas; por un lado, un módulo encargado de comunicarse con el dron para obtener su pose completa en tiempo real y enviarle los comandos de velocidad. Por otro lado, un módulo de detección visual encargado de analizar las imágenes, con el que se consiguió detectar los puntos 2D de cada marcador ArUco y, aplicando las rotaciones y traslaciones matemáticas necesarias y el algoritmo `solvePnP`, ubicar con precisión la puerta tridimensionalmente en el mapa global.

Finalmente, para la parte de navegación, la idea inicial se basaba en una navegación punto a punto (*waypoints*) guiada por un control Proporcional (P) básico. Aunque esta estrategia funcionaba en la teoría y en el simulador, falló en la realidad, debido a las zonas muertas de los motores y las derivas del dron en el mundo real. Esta limitación motivó la evolución del sistema hacia un controlador PID completo apoyado en un seguimiento de trayectoria. Esta solución funcionó tanto en la simulación como en la realidad.

7.2. Líneas de trabajo futuro

Aunque el sistema autónomo desarrollado funciona correctamente, la gran ventaja de haberlo diseñado modularmente hace que resulte muy fácil añadirle mejoras. A partir de la base sólida que deja este Trabajo Fin de Grado, se proponen tres líneas de desarrollo para llevar el proyecto al siguiente nivel:

7.2.1. Procesamiento a bordo para evitar cortes de red

En la versión actual de este proyecto, el procesamiento se realiza en un ordenador externo. Esto significa que el vehículo debe enviar el vídeo y pose por Wi-Fi y esperar

a que el ordenador procese y le devuelva las órdenes de movimiento. Este sistema de comunicación ha provocado múltiples fallos en las pruebas, sobre todo, debido a los cortes de red.

El avance sería montar un ordenador pequeño (como una Raspberry Pi) sobre el chasis del dron. De esta forma, el propio vehículo procesaría las imágenes y calcularía las velocidades al instante, ganando autonomía y fiabilidad.

7.2.2. Detección de obstáculos mediante *Deep Learning*

Ahora mismo, el dron carece de capacidad para detectar obstáculos, es decir, al colocar las puertas hay que tener en cuenta la trayectoria que realizará el dron, para que no encuentre obstáculos por el camino.

La idea a futuro sería incorporar una red neuronal como MiDaS (analizada en el Capítulo 3), con ella, el sistema podría analizar el vídeo en tiempo real para calcular la profundidad, esto permitiría al dron detectar y esquivar obstáculos de forma autónoma, usando solo una cámara monocular, sin necesidad de incorporar otros sensores como LiDAR o cámara de profundidad.

7.2.3. Navegación basada en segmentación semántica

El uso de marcadores ArUco ha permitido validar la arquitectura, pero limita el despliegue en entornos no preparados. Una línea de investigación clave sería sustituir la detección de marcadores por una red neuronal de segmentación semántica capaz de identificar objetos geométricos por sí mismo. Esto permitiría al dron navegar en cualquier edificio sin necesidad de intervención previa en el entorno.

7.2.4. Diseño de un UAV personalizado

El DJI Tello ha sido una plataforma educativa fantástica y accesible para hacer estas primeras pruebas, pero sus limitaciones de hardware ponen un techo al rendimiento del código.

El salto definitivo sería construir un dron desde cero. Al integrar controladores de vuelo como Pixhawk y sensores de odometría de mayor calidad, se podría aprovechar al máximo toda la arquitectura desarrollada, para tratar de lograr la fiabilidad de un vuelo profesional.

7.3. Viabilidad comercial y aplicaciones industriales

La tecnología desarrollada en este proyecto tiene una utilidad muy directa en el mundo real. Hoy en día, la industria busca constantemente robots asequibles y ligeros que puedan automatizar tareas en lugares estrechos o peligrosos donde los sistemas GPS tradicionales simplemente no funcionan. Haber conseguido que un dron se ubique y navegue con seguridad usando solo una cámara abre la puerta a casos de uso muy prácticos, como por ejemplo:

- **Gestión automatizada de inventarios:** Se podrían programar enjambres de pequeños drones para que vuelen de forma autónoma a través de los pasillos de un gran almacén logístico. Podrían escanear códigos de barras o realizar inventarios de forma totalmente automática.
- **Inspección de infraestructuras críticas:** Se podrían enviar drones a revisar lugares cerrados y de difícil acceso, como minas subterráneas, túneles o edificios en obras. El dron podría recorrer toda la zona de forma segura para inspeccionarla sin poner en riesgo a ningún operario.
- **Exploración y rescate:** En situaciones de emergencia, como el colapso de un edificio, enviar a un equipo de rescate a ciegas es un riesgo enorme. Un dron podría ser el primero en entrar, utilizando su visión artificial para moverse con estabilidad por el interior de los escombros y localizar objetivos sin exponer vidas humanas al peligro inicial.

Bibliografía

- [1] LSLiDAR, “LSLiDAR - how are lidar point clouds imaged?.” <https://www.lslidar.com/es/how-are-lslidar-lidar-point-clouds-imaged/>, 2026. Accedido en 2026.
- [2] YellowScan, “YellowScan - lidar drone knowledge base.” <https://www.yellowscan.com/es/knowledge/lidar-drone/>, 2026. Accedido en 2026.
- [3] DronePrix, “Holybro PX4 Development Kit X500 V2.” <https://droneprix.es/pixhawk-holybro/713-px4-development-kit-x500-v2.html>, 2026. Accedido en 2026.
- [4] Bitcraze AB, “Crazyflie 2.1 - bitcraze.” <https://www.bitcraze.io/products/old-products/crazyflie-2-1/>, 2026. Accedido en 2026.
- [5] Dojo for Drones, “Diy optical flow drone.” <https://dojofordrones.com/diy-optical-flow-drone/>, 2026. Accedido en 2026.
- [6] G. Halmetschlager-Funek, M. Suchi, M. Kampel, and M. Vincze, “An empirical evaluation of ten depth cameras: Bias, precision, lateral noise, different lighting conditions and materials, and multiple sensor setups in indoor environments,” *IEEE Robotics & Automation Magazine*, vol. 26, no. 1, pp. 67–77, 2019.
- [7] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, 2020.
- [8] C. Campos, R. Elvira, J. J. Gómez, J. M. M. Montiel, and J. D. Tardós, “ORB-SLAM3: An accurate open-source library for visual, visual-inertial and multi-map SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [9] ResearchGate Dataset, “Dataset examples of aruco markers used to label walls and infrastructure in indoor environments.” https://www.researchgate.net/figure/Some-instances-of-the-collected-dataset-where-ArUco-markers-are-used-to-label-walls-and_fig4_381562244, 2024. Accedido en 2026.
- [10] OpenCV Organization, “OpenCV: Camera calibration and 3d reconstruction - solvePnP.” https://docs.opencv.org/3.4/d5/d1f/calib3d_solvePnP.html, 2026. Accedido en 2026.

- [11] Open Robotics, “tf2 - robot operating system documentation.” <https://wiki.ros.org/tf2>, 2026. Accedido en 2026.
- [12] Novus Automation, “Control PID: Rompiendo la barrera del tiempo.” <https://www.novusautomation.com/es/noticia/articulo-control-pid-rompiendo-la-barrera-del-tiempo>, 2026. Accedido en 2026.
- [13] ResearchGate, “Illustration of along-track, cross-track, and horizontal errors.” https://www.researchgate.net/figure/llustration-of-along-track-cross-track-and-horizontal-errors_fig6_265251751, 2026. Accedido en 2026.
- [14] MathWorks, “Exchange data with ROS 2 publishers and subscribers.” <https://it.mathworks.com/help/ros/gs/ros2-topics.html>, 2026. Accedido en 2026.
- [15] ResearchGate, “Comparison between ROS and ROS2 architecture.” https://www.researchgate.net/figure/Comparison-between-ROS-and-ROS2_fig4_335382592, 2026. Accedido en 2026.
- [16] Elephant Robotics, “Introduction and use of RViz - ROS 2 application base.” <https://docs.elephantrobotics.com/docs/gitbook-en/12-ApplicationBaseROS/12.2-ROS2/12.2.7-rviz>, 2026. Accedido en 2026.
- [17] A. Bermúdez, R. Casado, G. Fernández, M. Guijarro, and P. Olivas, “Drone challenge: A platform for promoting programming and robotics skills in k-12 education,” *International Journal of Advanced Robotic Systems*, vol. 16, no. 1, p. 1729881418820425, 2019.
- [18] Ryze Tech, “DJI Tello - especificaciones técnicas oficiales.” <https://www.ryzerobotics.com/tello/specs>, 2018. Accedido en 2026.
- [19] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.
- [20] E. Olson, “AprilTag: A robust and flexible visual fiducial system,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3400–3407, IEEE, 2011.
- [21] K. Ogata, *Ingeniería de control moderna*. Pearson Educación, 5 ed., 2010.
- [22] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to autonomous mobile robots*. MIT press, 2 ed., 2011.
- [23] Open Robotics, “RViz user guide - ROS 2 Humble documentation.” <https://docs.ros.org/en/humble/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html>, 2026. Accedido en 2026.
- [24] NVIDIA Corporation, “NVIDIA Isaac Sim - robotics simulation and synthetic data generation.” <https://developer.nvidia.com/isaac-sim>, 2026. Accedido en 2026.

- [25] Open Robotics, “ROS 2 Design: Repository of articles guiding the ROS 2 design efforts.” <https://design.ros2.org/>, 2026. Accedido en 2026.
- [26] D. Fuentes Escoté, J. Löw, *et al.*, “DJITelloPy: DJI Tello drone Python interface library.” <https://github.com/damiafuentes/DJITelloPy>, 2025. Accedido en 2025.
- [27] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.
- [28] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–123, 2000.
- [29] O. Michel, “Cyberbotics ltd. Webots™: professional mobile robot simulation,” *International Journal of Advanced Robotic Systems*, vol. 1, no. 1, pp. 39–42, 2004.